Chương 10

Căn chỉnh

Căn chỉnh (*alignment*) không phải là một khái niệm mới trong NLP, nhưng ý nghĩa của nó thay đổi theo từng lĩnh vực và theo thời gian. Trong NLP truyền thống, thuật ngữ *căn chỉnh* thường chỉ nhiệm vụ liên kết các phần tử tương ứng giữa hai tập hợp, chẳng hạn căn chỉnh các từ giữa một câu tiếng Trung và một câu tiếng Anh. Khi LLM ngày càng trở nên quan trọng trong nghiên cứu NLP, thuật ngữ này hiện được dùng rộng hơn để mô tả quá trình làm cho đầu ra của mô hình phù hợp với kỳ vọng của con người. Vấn đề mà căn chỉnh giải quyết là đầu ra của mô hình có thể không khớp với những mục tiêu hay bối cảnh cụ thể mà người dùng mong muốn. Ví dụ, các LLM đã được tiền huấn luyện có thể không làm theo được chỉ dẫn của người dùng, vì chúng không được huấn luyện để làm việc đó. Một ví dụ khác là LLM có thể sinh ra nội dung độc hại hoặc duy trì những thiên kiến vốn có trong dữ liệu huấn luyện của chúng. Điều này đặt ra những thách thức mới: phải bảo đảm đầu ra của LLM không chỉ chính xác và liên quan, mà còn đúng đắn về mặt đạo đức và không mang tính phân biệt đối xử.

Chỉ tiền huấn luyện LLM thôi có thể dẫn tới nhiều loại vấn đề căn chỉnh khác nhau. Mục tiêu cuối cùng của căn chỉnh là giải quyết hoặc giảm nhẹ các vấn đề này để bảo đảm LLM vừa hữu ích vừa an toàn. Vì LLM được huấn luyện trên một lượng dữ liệu khổng lồ, người ta có thể cho rằng với đủ dữ liệu chất lượng cao, tiền huấn luyện có thể làm LLM đủ chính xác và an toàn, và có lẽ không cần một giai đoạn căn chỉnh riêng nữa. Tuy nhiên, thực tế là gần như không thể thu thập được dữ liệu bao quát mọi nhiệm vụ hoặc thể hiện đầy đủ sở thích của con người. Điều này khiến việc đạt được căn chỉnh chỉ bằng tiền huấn luyện là rất khó. Hiện

nay, căn chỉnh vẫn là một bước then chốt trong quá trình phát triển LLM.

Trong chương này, chúng ta tập trung vào các phương pháp căn chỉnh cho LLM. Ta bắt đầu bằng việc bàn về các khái niệm chung của căn chỉnh. Sau đó, ta xem xét hai cách tiếp cận được dùng rộng rãi: **căn chỉnh theo chỉ dẫn** (*instruction alignment*) và **căn chỉnh theo sở thích của con người** (*human preference alignment*). Cách thứ nhất dùng các kỹ thuật tinh chỉnh có giám sát (*supervised fine-tuning*) để hướng dẫn LLM sinh ra đầu ra bám sát chỉ dẫn của người dùng. Cách thứ hai thường dựa vào các kỹ thuật học tăng cường (*reinforcement learning*), trong đó mô hình được huấn luyện dựa trên phản hồi của con người. Dù hai phương pháp này xuất phát từ những mục tiêu khác nhau, chúng thường được dùng kết hợp để phát triển các LLM được căn chỉnh tốt.

Chú giải của người dịch: "Căn chỉnh" ở đây nghĩa là gì?

Hãy hình dung một LLM vừa tiền huấn luyện xong như một người đã đọc gần hết sách báo trên đời nhưng chưa từng đi làm: biết rất nhiều, nhưng chưa biết *làm theo yêu cầu* của người khác, và có thể nói ra cả những điều thiếu lịch sự hay sai lệch mà nó đã đọc được. "Căn chỉnh" là quá trình "uốn nắn" mô hình sao cho hành vi của nó *khớp* với điều con người muốn: hiểu và làm đúng yêu cầu, trả lời hữu ích, trung thực và không gây hại. Chữ *alignment* vốn có nghĩa là "thẳng hàng" — ở đây là làm cho mục tiêu của mô hình "thẳng hàng" với mục tiêu của con người.

10.1 Tổng quan về căn chỉnh LLM

Căn chỉnh có thể được thực hiện theo nhiều cách khác nhau. Ta cần nhiều phương pháp khác nhau cho việc căn chỉnh LLM vì bản thân bài toán này phức tạp và nhiều mặt, đòi hỏi kết hợp nhiều chiến lược kỹ thuật. Ở đây ta xét ba cách tiếp cận được dùng rộng rãi để căn chỉnh LLM.

Cách tiếp cận thứ nhất là tinh chỉnh LLM bằng dữ liệu có gán nhãn. Cách này đơn giản vì nó chỉ mở rộng quá trình tiền huấn luyện để thích nghi LLM với các nhiệm vụ cụ thể. Một ví dụ là **tinh chỉnh có giám sát (SFT)** (*supervised fine-tuning*), trong đó LLM được huấn luyện thêm trên một tập dữ liệu gồm các chỉ dẫn dành riêng cho từng nhiệm vụ, ghép cặp với đầu ra mong đợi của chúng. Tập dữ liệu SFT thường nhỏ hơn nhiều so với tập huấn luyện ban đầu, nhưng dữ liệu này có tính chuyên biệt cao.

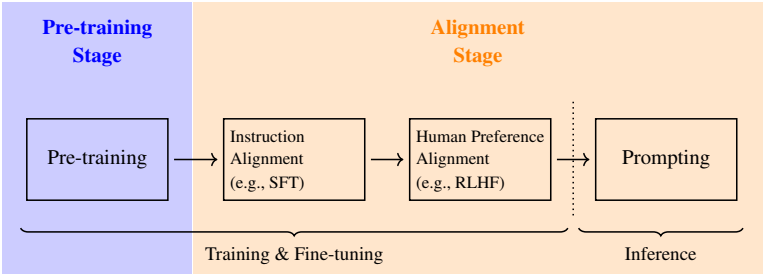
Kết quả của SFT là LLM có thể học cách thực hiện nhiệm vụ dựa trên chỉ dẫn của người dùng. Đó có thể là những nhiệm vụ đã gặp trong SFT, hoặc những nhiệm vụ mới nhưng tương tự. Ví dụ, tinh chỉnh LLM trên một tập các cặp câu hỏi–câu trả lời giúp mô hình trả lời được các câu hỏi cụ thể, kể cả những câu hỏi không có trực tiếp trong tập dữ liệu SFT. Phương pháp này đặc biệt hữu ích khi quan hệ giữa đầu vào và đầu ra tương đối dễ mô tả và dữ liệu dễ gán nhãn.

Cách tiếp cận thứ hai là tinh chỉnh LLM bằng các mô hình phần thưởng (*reward model*). Một khó khăn trong căn chỉnh là các giá trị và kỳ vọng của con người rất phức tạp và khó mô tả. Trong nhiều trường hợp, chính con người cũng khó diễn đạt chính xác điều gì là đúng đắn về mặt đạo đức hay phù hợp về mặt văn hóa. Do đó, việc thu thập và gán nhãn dữ liệu tinh chỉnh không đơn giản như với SFT. Hơn nữa, căn chỉnh không chỉ đơn thuần là một nhiệm vụ khớp dữ liệu. Một số ít mẫu do con người gán nhãn thường không đủ để định nghĩa đầy đủ những hành vi phức tạp này. Thay vào đó, mô hình phải được dạy cách đánh giá xem đầu ra nào phản ánh tốt nhất sở thích của con người. Ví dụ, ta không chỉ muốn đầu ra chính xác về mặt kỹ thuật mà còn muốn nó phù hợp với kỳ vọng và giá trị của con người. Một ý tưởng là xây dựng một mô hình phần thưởng mô phỏng sự đánh giá của con người. Mô hình phần thưởng này sẽ hoạt động bằng cách thưởng cho LLM mỗi khi nó sinh ra câu trả lời sát hơn với sở thích của con người, giống như cách thầy giáo nhận xét bài làm của học sinh. Để có được mô hình phần thưởng như vậy, ta có thể huấn luyện một hàm chấm điểm từ dữ liệu sở thích của con người. Mô hình phần thưởng đã huấn luyện sau đó được dùng làm "người dẫn đường" để điều chỉnh và cải thiện LLM. Cách nhìn này biến nhiệm vụ căn chỉnh LLM thành một nhiệm vụ học tăng cường. Các phương pháp thu được, chẳng hạn **học tăng cường từ phản hồi của con người (RLHF)** (*reinforcement learning from human feedback*), đã được chứng minh là đặc biệt thành công trong việc thích nghi LLM để tuân theo những điều tinh tế trong hành vi và chuẩn mực xã hội của con người.

Cách tiếp cận thứ ba là thực hiện căn chỉnh trong lúc suy luận (*inference*) thay vì trong lúc huấn luyện hay tinh chỉnh. Từ góc nhìn này, prompting đối với LLM cũng có thể được xem là một hình thức căn chỉnh không cần huấn luyện. Nhờ đó, ta có thể thích nghi LLM một cách linh hoạt với nhiều nhiệm vụ khác nhau với chi phí tối thiểu. Một cách khác

để căn chỉnh lúc suy luận là chấm điểm lại các đầu ra của LLM. Ví dụ, ta có thể xây dựng một hệ thống chấm điểm mô phỏng phản hồi của con người đối với các đầu ra của LLM (giống như một mô hình phần thưởng) và ưu tiên những đầu ra nhận được phản hồi tích cực hơn.

Ba phương pháp nêu trên thường được dùng nối tiếp nhau sau khi tiền huấn luyện hoàn tất: trước tiên ta thực hiện SFT, sau đó là RLHF, rồi đến prompting LLM trong lúc suy luận. Điều này chia quá trình phát triển LLM thành hai giai đoạn một cách đại thể — giai đoạn tiền huấn luyện và giai đoạn căn chỉnh. Hình 10.1 minh họa điều này. Vì các kỹ thuật prompting đã được bàn kỹ trong chương trước, phần còn lại của chương này sẽ chủ yếu tập trung vào các phương pháp căn chỉnh dựa trên tinh chỉnh.



Hình 10.1: Sơ đồ minh họa quy trình "tiền huấn luyện rồi căn chỉnh" để phát triển LLM. Trong giai đoạn tiền huấn luyện, ta huấn luyện một LLM trên lượng dữ liệu khổng lồ bằng cách dự đoán token tiếp theo. Sau đó, trong giai đoạn căn chỉnh, ta căn chỉnh LLM theo chỉ dẫn, ý định và sở thích của người dùng. Giai đoạn này bao gồm căn chỉnh theo chỉ dẫn, căn chỉnh theo sở thích của con người, và prompting.

Chú giải của người dịch: Đọc Hình 10.1

Các nhãn trong hình: *Pre-training Stage* = giai đoạn tiền huấn luyện; *Alignment Stage* = giai đoạn căn chỉnh; *Instruction Alignment (e.g., SFT)* = căn chỉnh theo chỉ dẫn (ví dụ SFT); *Human Preference Alignment (e.g., RLHF)* = căn chỉnh theo sở thích của con người (ví dụ RLHF); *Training & Fine-tuning* = huấn luyện và tinh chỉnh (làm thay đổi tham số mô hình); *Inference* = suy luận (chỉ dùng mô hình, không đổi tham số). Có thể so sánh với việc đào tạo một nhân viên: đi học đại học (tiền huấn luyện) → được

hướng dẫn làm đúng các loại việc ở công ty (SFT) → được sếp và khách hàng nhận xét để làm vừa ý hơn (RLHF) → mỗi lần giao việc thì dặn dò cụ thể (prompting).

10.2 Căn chỉnh theo chỉ dẫn

Một đặc điểm của LLM là chúng có thể làm theo các prompt do người dùng cung cấp để thực hiện nhiều nhiệm vụ khác nhau. Trong nhiều ứng dụng, một prompt gồm một chỉ dẫn đơn giản và đầu vào của người dùng, và mục tiêu là để LLM thực hiện nhiệm vụ một cách chính xác theo chỉ dẫn đó. Năng lực này thường được gọi là khả năng làm theo chỉ dẫn (*instruction following*). Ví dụ, dưới đây là một prompt trong đó LLM được yêu cầu rút ra các ý chính và viết một bản tóm tắt ngắn gọn cho một bài báo dài.

Instruction	Tóm tắt văn bản này trong ba câu.
Input	Giờ mùa hè (Daylight Savings Time – DST) — việc vặn đồng hồ nhanh thêm một giờ vào mùa hè — được bắt đầu ở Đức vào năm 1916. Trong Chiến tranh thế giới thứ nhất, đó là một cách để tiết kiệm …
Output	___

Nhiệm vụ này đòi hỏi LLM hiểu chỉ dẫn "Tóm tắt văn bản này trong ba câu" và tóm tắt văn bản theo đúng như vậy. Tuy nhiên, các LLM nền đã tiền huấn luyện thường được huấn luyện để dự đoán token tiếp theo chứ không phải để sinh ra đầu ra làm theo chỉ dẫn. Áp dụng một LLM đã tiền huấn luyện vào ví dụ trên nhiều khả năng sẽ khiến mô hình viết tiếp bài báo thay vì tóm tắt nó. Mục tiêu của căn chỉnh theo chỉ dẫn (hay **tinh chỉnh theo chỉ dẫn** (*instruction fine-tuning*)) là điều chỉnh LLM để nó đáp ứng chính xác chỉ dẫn và ý định của người dùng. Phần còn lại của mục này sẽ bàn về một số chủ đề liên quan đến căn chỉnh theo chỉ dẫn, bao gồm tinh chỉnh LLM để làm theo chỉ dẫn, sinh ra hoặc thu thập dữ liệu chỉ dẫn, và tổng quát hóa căn chỉnh theo chỉ dẫn.

Chú giải của người dịch: Vì sao LLM nền lại "viết tiếp" thay vì tóm tắt?

Trong tiền huấn luyện, mô hình chỉ học một việc: đọc một đoạn văn và đoán token (mẩu từ) kế tiếp. Trên Internet, sau câu "…đó là một cách để tiết kiệm" thường là phần tiếp theo của chính bài báo ("…than đá và nhiên liệu"), chứ hiếm khi là một bản tóm tắt. Vì vậy, mô hình nền có xu hướng viết tiếp cho "trơn tru" thay vì hiểu rằng dòng đầu là một *yêu cầu*. Tinh chỉnh theo chỉ dẫn dạy mô hình nhận ra cấu trúc "chỉ dẫn + đầu vào → đầu ra".

10.2.1 Tinh chỉnh có giám sát

Một cách tiếp cận đơn giản để thích nghi LLM cho việc làm theo chỉ dẫn là tinh chỉnh các mô hình này bằng các cặp đầu vào–đầu ra đã được gán nhãn [Ouyang và cộng sự, 2022; Wei và cộng sự, 2022a]. Trong bối cảnh căn chỉnh LLM, SFT thường được triển khai dưới dạng tinh chỉnh theo chỉ dẫn. Khác với huấn luyện mô hình ngôn ngữ thông thường, trong đó hàm mất mát được tính trên tất cả các token của chuỗi, SFT chỉ tính hàm mất mát trên các token đầu ra, với điều kiện là đầu vào. Gọi $\mathbf{x} = x_0 \dots x_m$ là một chuỗi đầu vào (ví dụ: chỉ dẫn + đầu vào của người dùng) và $\mathbf{y} = y_1 \dots y_n$ là chuỗi đầu ra tương ứng. Dữ liệu SFT gồm các cặp đầu vào–đầu ra như vậy (ký hiệu là S), trong đó mỗi đầu ra là một câu trả lời mong muốn ứng với đầu vào. Ví dụ, bảng sau đây cho thấy một số mẫu trong một tập dữ liệu SFT.

x (chỉ dẫn + đầu vào của người dùng)	**y** (đầu ra)
Tóm tắt bài báo sau. Bài báo: Trong những năm gần đây, năng lượng mặt trời đã tăng trưởng chưa từng có, trở thành nguồn năng lượng phát triển nhanh nhất …	{∗bản tóm tắt∗}
Phân tích cảm xúc của bài nhận xét sau. Nhận xét: Tôi cực kỳ thích trải nghiệm ăn uống mới này. Đồ ăn tuyệt hảo và phục vụ thì không chê vào đâu được.	Tích cực
Dịch câu sau sang tiếng Pháp. Câu: practice indeed helps. ("luyện tập quả thật có ích")	La pratique aide effectivement.

x (chỉ dẫn + đầu vào của người dùng)	**y** (đầu ra)
Trích xuất các số liệu tài chính chính từ báo cáo kết quả kinh doanh sau. Báo cáo: Công ty báo cáo doanh thu 10 triệu đô la trong quý đầu tiên với biên lợi nhuận 15% …	Doanh thu: 10 triệu đô la, Biên lợi nhuận: 15%
Phân loại email sau là thư rác hay không phải thư rác. Văn bản: Chúc mừng! Bạn đã trúng một thẻ quà tặng trị giá 500 đô la. Nhấn vào đây để nhận ngay.	Thư rác
Đưa ra giải pháp cho sự cố kỹ thuật sau. Sự cố: máy tính của tôi chạy chậm và thường xuyên bị treo.	Trước tiên, hãy kiểm tra …

trong đó các chỉ dẫn được tô sáng. Tập dữ liệu này chứa các chỉ dẫn và đầu ra tương ứng cho nhiều bài toán NLP khác nhau, vì vậy ta có thể tinh chỉnh một LLM để xử lý đồng thời nhiều nhiệm vụ.

Trong SFT, ta muốn cực đại hóa xác suất của đầu ra đúng khi biết đầu vào. Xét một LLM với các tham số đã tiền huấn luyện $\hat{\theta}$. Khi đó mục tiêu tinh chỉnh có thể được viết như sau:

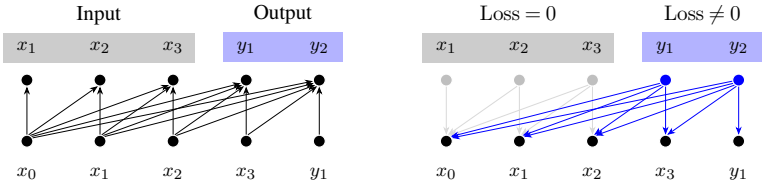
$$\tilde{\theta} \;=\; \arg\max_{\hat{\theta}^+} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_{\hat{\theta}^+}(\mathbf{y}|\mathbf{x}), \tag{10.1}$$

trong đó $\tilde{\theta}$ ký hiệu các tham số được tối ưu qua tinh chỉnh, và $\hat{\theta}^+$ biểu diễn một sự điều chỉnh của $\hat{\theta}$. Từ đây ta sẽ bỏ chỉ số trên $+$ và dùng θ để biểu diễn $\hat{\theta}^+$ cho ký hiệu gọn gàng. Nhưng bạn đọc nên nhớ rằng quá trình tinh chỉnh bắt đầu từ các tham số đã tiền huấn luyện chứ không phải từ các tham số khởi tạo ngẫu nhiên.

Hàm mục tiêu $\log \Pr_\theta(\mathbf{y}|\mathbf{x})$ được tính bằng cách cộng log-xác suất của các token trong $\mathbf{y}$, với điều kiện là đầu vào $\mathbf{x}$ và tất cả các token đứng trước $\mathbf{y}_{<i}$:

$$\log \Pr_\theta(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr_\theta(y_i|\mathbf{x}, \mathbf{y}_{<i}). \tag{10.2}$$

Cách viết này tương đương với việc cực tiểu hóa hàm mất mát entropy chéo (*cross-entropy*).



(a) Forward Pass (b) Backward Pass

Hình 10.2: Minh họa tinh chỉnh có giám sát cho LLM. Ta nối đầu vào và đầu ra thành một chuỗi duy nhất. Trong lượt truyền xuôi, ta chạy LLM như bình thường. Trong lượt truyền ngược, ta chỉ tính hàm mất mát cho phần đầu ra và đơn giản đặt hàm mất mát cho phần đầu vào bằng 0. (Nhãn trong hình: *Input* = đầu vào, *Output* = đầu ra, *Loss* = mất mát; (a) lượt truyền xuôi, (b) lượt truyền ngược.)

Chú giải của người dịch: Ý nghĩa của Phương trình (10.1) và (10.2)

$\Pr_\theta(\mathbf{y}|\mathbf{x})$ là xác suất mô hình (với bộ tham số θ) gán cho câu trả lời đúng $\mathbf{y}$ khi đọc đầu vào $\mathbf{x}$. Ta muốn xác suất này càng lớn càng tốt cho mọi cặp mẫu trong tập dữ liệu, nên lấy tổng log của chúng rồi tìm θ làm tổng này lớn nhất (arg max). Mô hình sinh câu trả lời từng token một, nên theo quy tắc nhân xác suất, xác suất cả câu bằng *tích* xác suất từng token (mỗi token tính khi đã biết đầu vào và các token trước nó). Lấy logarit biến tích thành tổng: $\log(ab) = \log a + \log b$ — đó chính là Phương trình (10.2). Ví dụ: nếu $\mathbf{y}$ có 2 token với xác suất lần lượt 0.5 và 0.8 thì $\Pr(\mathbf{y}|\mathbf{x}) = 0.5 \times 0.8 = 0.4$ và $\log 0.4 = \log 0.5 + \log 0.8$. Cực đại hóa tổng log-xác suất cũng chính là cực tiểu hóa *số đối* của nó, tức là hàm mất mát entropy chéo.

Nếu ta nối $\mathbf{x}$ và $\mathbf{y}$ thành một chuỗi duy nhất, một dạng tổng quát hơn của mô hình hóa ngôn ngữ dựa trên log-xác suất đồng thời $\log \Pr_\theta(\mathbf{x}, \mathbf{y})$, tức là ta cực tiểu hóa hàm mất mát (hay cực đại hóa log-hợp lý) trên tất cả các token của chuỗi $\text{seq}_{\mathbf{x},\mathbf{y}} = [\mathbf{x}, \mathbf{y}]$. Ta có thể viết xác suất của chuỗi này bằng quy tắc chuỗi

$$\begin{aligned} \log \Pr_\theta(\text{seq}_{\mathbf{x},\mathbf{y}}) &= \log \Pr_\theta(\mathbf{x}, \mathbf{y}) \\ &= \underbrace{\log \Pr_\theta(\mathbf{x})}_{\text{đặt bằng 0}} + \underbrace{\log \Pr_\theta(\mathbf{y}|\mathbf{x})}_{\text{tính mất mát}}. \end{aligned} \tag{10.3}$$

Vế phải của phương trình có hai số hạng. Ta có thể đơn giản bỏ qua số

hạng thứ nhất $\log \Pr_\theta(\mathbf{x})$ và chỉ tập trung vào số hạng thứ hai $\log \Pr_\theta(\mathbf{y}|\mathbf{x})$ để tính mất mát. Nhờ vậy, việc huấn luyện có thể được thực hiện bằng các LLM tiêu chuẩn. Với chuỗi $\text{seq}_{\mathbf{x},\mathbf{y}}$, trước tiên ta chạy lượt truyền xuôi như bình thường. Sau đó, trong lượt truyền ngược, ta buộc phần mất mát tương ứng với $\mathbf{x}$ bằng không. Hình 10.2 minh họa quá trình này.

Chú giải của người dịch: "Che" phần đầu vào khi tính mất mát

Phương trình (10.3) dựa trên công thức xác suất quen thuộc $P(A \cap B) = P(A) \cdot P(B|A)$, lấy log hai vế thành tổng. Trong thực tế, ta đưa cả chuỗi "chỉ dẫn + đầu vào + câu trả lời" vào mô hình, mô hình đoán từng token như lúc tiền huấn luyện, nhưng chỉ "chấm điểm" (tính mất mát) ở những vị trí thuộc câu trả lời. Ta không muốn mô hình học cách *viết ra câu hỏi*, mà chỉ muốn nó học cách *trả lời*. Giống như chấm bài: giáo viên chỉ chấm phần bài làm, không chấm phần đề bài được chép lại.

Lấy $\log \Pr_\theta(\text{seq}_{\mathbf{x},\mathbf{y}})$ làm hàm mục tiêu, ta có thể mô tả SFT bằng một dạng huấn luyện mô hình ngôn ngữ thông thường:

$$\tilde{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\text{seq}_{\mathbf{x},\mathbf{y}}). \tag{10.4}$$

Bài toán ta xét ở trên về bản chất là một bài toán **dự đoán một lượt** (*single-round prediction*), trong đó LLM sinh ra câu trả lời dựa trên một đầu vào duy nhất mà không có thêm tương tác hay phản hồi nào từ người dùng. Đầu vào được xử lý, và đầu ra được sinh ra trong một lần. Đây là tình huống điển hình khi chỉ có một câu hỏi được đặt ra và một câu trả lời được đưa ra, không có câu hỏi tiếp theo hay yêu cầu làm rõ nào. Tuy nhiên, trong thực tế, đôi khi ta phải xử lý các bài toán dự đoán nhiều lượt (*multi-round prediction*), chẳng hạn khi một LLM tham gia một cuộc hội thoại qua nhiều lượt. Trong bối cảnh này, LLM không chỉ sinh câu trả lời dựa trên đầu vào ban đầu mà còn phải tiếp nhận các đầu vào tiếp theo, vốn có thể làm rõ hoặc mở rộng những trao đổi trước đó. Ví dụ, ta có thể dùng LLM làm một chatbot trợ lý chăm sóc sức khỏe và trò chuyện với người dùng như sau

User Dạo này tôi thấy rất mệt mỏi.

Chatbot Tôi rất tiếc khi nghe điều đó. Ngoài cảm giác mệt mỏi, bạn có nhận thấy triệu chứng nào khác không?

User Có, tôi cũng hay bị đau đầu.

Chatbot Các triệu chứng này đã kéo dài bao lâu rồi?

User Khoảng một tuần nay.

Chatbot Có lẽ bạn nên đi khám với nhân viên y tế. Bạn có muốn tôi giúp đặt lịch hẹn không?

User Vâng, làm ơn. Có thể hẹn sau giờ làm việc được không?

Chatbot Chắc chắn rồi, tôi có thể sắp xếp. Còn chỗ trống vào thứ Tư và thứ Năm tuần sau, sau 5 giờ chiều. Ngày nào thuận tiện hơn cho bạn?

…

Trong nhiệm vụ này có nhiều lượt hội thoại, mỗi lượt đòi hỏi sinh ra một câu trả lời dựa trên yêu cầu hoặc câu hỏi của người dùng và lịch sử hội thoại. Giả sử ta có K lượt hội thoại, ký hiệu là $\{\mathbf{x}^1, \mathbf{y}^1, \mathbf{x}^2, \mathbf{y}^2, \ldots, \mathbf{x}^K, \mathbf{y}^K\}$. Ở đây $\mathbf{x}^k$ và $\mathbf{y}^k$ lần lượt ký hiệu yêu cầu của người dùng và câu trả lời ở lượt thứ k. Log-xác suất sinh ra câu trả lời có thể được viết là $\log \Pr_\theta(\mathbf{y}^k | \mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^k)$. Khi đó, mục tiêu của ta là cực đại hóa tổng các log-xác suất này

$$\tilde{\theta} \;=\; \arg\max_\theta \sum_{k=1}^{K} \log \Pr_\theta(\mathbf{y}^k | \mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^k). \tag{10.5}$$

Một cách cài đặt đơn giản là tính xác suất có điều kiện cho từng k. Tuy nhiên, cách này đòi hỏi chạy LLM K lần, mỗi lần với lịch sử hội thoại dài thêm để đưa ra dự đoán. Một phương pháp hiệu quả hơn là tính mất mát cho tất cả các câu trả lời chỉ trong một lần chạy LLM. Để làm vậy, ta biểu diễn cuộc hội thoại dưới dạng một chuỗi $\text{seq}_{\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K, \mathbf{y}^K} = [\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K, \mathbf{y}^K]$ (gọi tắt là seq). Log-xác suất của chuỗi này là

$$\begin{aligned}
\log \Pr_\theta(\text{seq}) &= \log \Pr_\theta(\mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K, \mathbf{y}^K) \\
&= \underbrace{\log \Pr_\theta(\mathbf{x}^1)}_{\text{đặt bằng 0}} + \underbrace{\log \Pr_\theta(\mathbf{y}^1 | \mathbf{x}^1)}_{\text{tính mất mát}} + \cdots +
\end{aligned}$$

$$\underbrace{\log \Pr_\theta(\mathbf{x}^K | \mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{y}^{K-1})}_{\text{đặt bằng 0}} + \underbrace{\log \Pr_\theta(\mathbf{y}^K | \mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{x}^K)}_{\text{tính mất mát}}. \tag{10.6}$$

Mẹo ở đây là ta bỏ qua mất mát của việc sinh ra các đầu vào của người dùng (tức là $\log \Pr_\theta(\mathbf{x}^1)$, …, $\log \Pr_\theta(\mathbf{x}^K | \mathbf{x}^1, \mathbf{y}^1, \ldots, \mathbf{y}^{K-1})$), như minh họa ở Hình 10.3. Do đó ta chỉ tính xác suất sinh ra các câu trả lời khi biết lịch sử hội thoại của chúng; nói cách khác, Phương trình (10.6) tương đương với Phương trình (10.5). Giống như với Phương trình (10.4), việc huấn luyện mô hình dự đoán nhiều lượt này có thể được thực hiện bằng cách cực đại hóa log-hợp lý trên một tập dữ liệu huấn luyện $\mathcal{D}$:

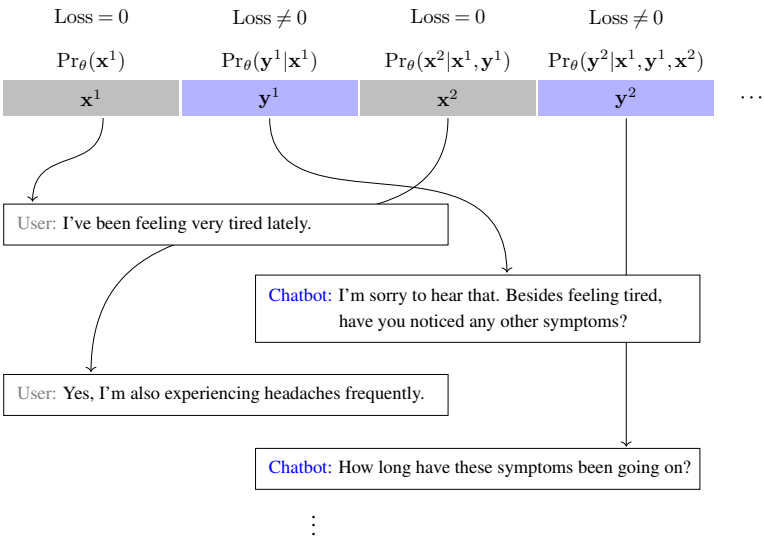
$$\tilde{\theta} \;=\; \arg\max_\theta \sum_{\text{seq} \in \mathcal{D}} \log \Pr_\theta(\text{seq}). \tag{10.7}$$

Chú giải của người dịch: Hội thoại nhiều lượt: một lần chạy cho cả cuộc trò chuyện

Phương trình (10.6) là quy tắc nhân xác suất áp dụng liên tiếp: xác suất của cả cuộc hội thoại bằng tích xác suất của từng "lời nói", mỗi lời tính khi đã biết mọi lời trước nó; lấy log thì tích thành tổng. Trong tổng đó, các số hạng ứng với lời của *người dùng* bị bỏ đi (đặt mất mát bằng 0), chỉ giữ các số hạng ứng với lời của *chatbot* — chính là các số hạng trong Phương trình (10.5). Lợi ích: với một hội thoại $K = 4$ lượt, thay vì chạy mô hình 4 lần (mỗi lần với lịch sử dài hơn), ta chỉ cần đưa cả cuộc hội thoại vào một lần và "che" các phần của người dùng khi tính mất mát.

Mặc dù việc cài đặt các phương pháp SFT nêu trên có vẻ đơn giản vì về cơ bản chúng giống huấn luyện mô hình ngôn ngữ thông thường, vẫn còn những vấn đề cần cân nhắc trong thực tế. Chẳng hạn,

- SFT đòi hỏi dữ liệu có gán nhãn. Điều này khiến SFT khá khác với tiền huấn luyện, nơi văn bản thô được dùng làm dữ liệu huấn luyện và luôn sẵn có. Cũng như trong các bài toán học máy có giám sát khác, việc gán nhãn và chọn lọc dữ liệu trong SFT không phải là việc đơn giản. Nói chung, ta muốn xây dựng dữ liệu SFT vừa nhiều về số lượng vừa cao về chất lượng, và dữ liệu này phải liên quan chặt chẽ đến những nhiệm vụ mà LLM sẽ thực hiện. Mặt khác, lại



Hình 10.3: Minh họa tinh chỉnh có giám sát cho các mô hình hội thoại. Ở đây LLM đóng vai một chatbot trả lời từng yêu cầu dựa trên lịch sử hội thoại. Cuộc hội thoại diễn ra bằng cách luân phiên giữa người dùng (*User*) và chatbot. Trong SFT, ta coi toàn bộ cuộc hội thoại là một chuỗi, giống như trong các LLM tiêu chuẩn, nhưng chỉ tính mất mát cho các câu trả lời của LLM. (Nội dung các ô hội thoại là đoạn đầu của ví dụ hội thoại ở trên.)

cần tinh chỉnh LLM với ít dữ liệu hơn để giảm thiểu chi phí tính toán và chi phí xây dựng dữ liệu. Chất lượng của LLM thường phụ thuộc rất nhiều vào dữ liệu dùng trong SFT. Vì vậy, dữ liệu này phải được xây dựng và kiểm tra cẩn thận. Như ta sẽ thấy ở các tiểu mục sau, SFT có thể hiệu quả và hiệu nghiệm hơn nhờ các kỹ thuật xây dựng dữ liệu tiên tiến hơn.

- SFT vẫn tốn kém về mặt tính toán đối với LLM do kích thước lớn của chúng. Kết quả là việc duy trì và cập nhật các mô hình như vậy tiêu tốn nhiều tài nguyên. Ví dụ, việc cập nhật gradient cho hàng tỉ tham số trong một LLM đòi hỏi năng lực tính toán và bộ nhớ đáng kể. Điều này thường cần các môi trường tính toán hiệu năng cao vốn tốn kém để vận hành. Để giải quyết những thách thức này, nhiều chiến lược tối ưu đã được khám phá, chẳng hạn cắt tỉa (*pruning*), lượng tử hóa (*quantization*), và sử dụng các thuật toán huấn luyện hiệu quả hơn. Đặc biệt, đã có nhiều quan tâm đến các phương pháp tinh chỉnh hiệu quả tham số (*parameter-efficient fine-tuning*), được thiết kế để duy trì hiệu năng tiên tiến nhất mà không cần nhiều tài nguyên tính toán. Ta đã thấy ở Chương 9 rằng áp dụng các kỹ thuật như prompt mềm (*soft prompts*) có thể làm quá trình tinh chỉnh hiệu quả hơn. Để tìm hiểu thêm về các phương pháp hiệu quả tham số, bạn đọc có thể tham khảo các bài báo liên quan [Houlsby và cộng sự, 2019; Hu và cộng sự, 2022; Han và cộng sự, 2024].

- SFT có thể được xem là một bước hậu huấn luyện (*post-training*) tiếp sau tiền huấn luyện. Đó là một giai đoạn huấn luyện riêng, được thiết kế để giữ lại những ưu điểm của quá trình tiền huấn luyện ban đầu đồng thời bổ sung những điều chỉnh mới. Điều này có vẻ nghịch lý, vì cập nhật một LLM đã tiền huấn luyện bằng thêm dữ liệu có thể khiến mô hình quên đi một phần tri thức trước đó. Hãy hình dung tình huống ta có một lượng lớn dữ liệu SFT và tinh chỉnh LLM rất nhiều. Khi đó, LLM có thể quá khớp (*overfit*) dữ liệu, điều này lại có thể làm giảm khả năng tổng quát hóa hoặc gây ra hiện tượng quên thảm khốc (*catastrophic forgetting*). Một chiến lược phổ biến để giảm nhẹ vấn đề này là dùng các kỹ thuật chính quy hóa (*regularization*) và dừng sớm (*early stopping*). Một cách tiếp cận thực tế khác là dùng tốc độ học (*learning rate*) nhỏ hơn để điều chỉnh nhẹ nhàng các trọng số của LLM. Ngoài ra, tinh chỉnh bằng

dữ liệu từ nhiều nguồn và nhiều lĩnh vực bài toán khác nhau cũng có thể có lợi. Tuy vậy, trong thực tế, bước SFT thường được xem xét cẩn thận và đòi hỏi nhiều công sức kỹ thuật cũng như thực nghiệm để tối ưu.

Chú giải của người dịch: Quên thảm khốc và cách phòng tránh

Quên thảm khốc là hiện tượng mô hình học việc mới thì "quên" mất nhiều điều đã học trước đó, vì cùng một bộ tham số bị kéo sang phục vụ mục tiêu mới. Hình dung một học sinh ôn thi cấp tốc chỉ một dạng bài suốt nhiều tuần, đến lúc gặp dạng khác lại lúng túng. Các biện pháp nêu trên đều nhằm "thay đổi vừa phải": *tốc độ học nhỏ* nghĩa là mỗi bước cập nhật chỉ xê dịch tham số một chút; *dừng sớm* là ngừng huấn luyện khi kết quả trên tập kiểm định bắt đầu xấu đi; *chính quy hóa* là thêm ràng buộc để tham số không đi quá xa giá trị ban đầu. *Lượng tử hóa* là lưu mỗi tham số bằng ít bit hơn (ví dụ 8 bit thay vì 32 bit) để tiết kiệm bộ nhớ; *cắt tỉa* là bỏ bớt những tham số ít quan trọng.

10.2.2 Xây dựng dữ liệu tinh chỉnh

Dữ liệu tinh chỉnh quan trọng đến mức rất nhiều nghiên cứu gần đây về LLM đã tập trung vào việc xây dựng các tập dữ liệu cho tinh chỉnh theo chỉ dẫn. Giống như với hầu hết các nhiệm vụ học máy, nhìn chung có hai cách tiếp cận để xây dựng tập dữ liệu tinh chỉnh: dữ liệu do con người tạo ra và dữ liệu được sinh tự động.

1. Dữ liệu do con người tạo ra

Một phương pháp đơn giản là thuê người gán nhãn tạo ra các cặp đầu vào–đầu ra cho những nhiệm vụ ta quan tâm. Khác với việc gán nhãn dữ liệu trong NLP truyền thống, chẳng hạn phân loại văn bản, nơi người gán nhãn chỉ việc gán nhãn cho các văn bản đã thu thập theo hướng dẫn, việc tạo dữ liệu tinh chỉnh cho LLM đòi hỏi nhiều bước và công sức hơn, do đó khó khăn hơn. Giả sử ta muốn có dữ liệu tinh chỉnh cho nhiệm vụ dịch máy từ tiếng Anh sang tiếng Trung. Bước đầu tiên là viết một khuôn mẫu prompt để mô tả nhiệm vụ và định dạng bài toán một cách rõ ràng. Ví dụ,

Instruction	Dịch văn bản từ tiếng Anh sang tiếng Trung.
User Input	{∗text∗}
Output	{∗translation∗}

Sau đó, ta thu thập các cặp văn bản nguồn và đích (tức là văn bản tiếng Anh và bản dịch tiếng Trung tương ứng), rồi thay các biến {∗text∗} (văn bản) và {∗translation∗} (bản dịch) để sinh ra các mẫu tinh chỉnh. Ví dụ, với một cặp câu tiếng Anh và tiếng Trung

$$\underset{\{*\text{text}*\}}{\text{How's the weather today?}} \quad \rightarrow \quad \underset{\{*\text{translation}*\}}{\text{今天天气怎么样?}}$$

(cả hai câu đều có nghĩa là "Hôm nay thời tiết thế nào?"), ta có thể sinh ra một mẫu tinh chỉnh bằng khuôn mẫu prompt như sau

Instruction	Dịch văn bản từ tiếng Anh sang tiếng Trung.
User Input	How's the weather today?
Output	今天天气怎么样?

Tức là,

x: Dịch văn bản từ tiếng Anh sang tiếng Trung.\n How's the weather today?
y: 今天天气怎么样?

[1] Ta có thể dùng cặp $(\mathbf{x}, \mathbf{y})$ này để tinh chỉnh LLM như đã mô tả ở tiểu mục trước.

Một khó khăn ở đây là có rất nhiều cách khác nhau để viết khuôn mẫu prompt cho cùng một nhiệm vụ, và những người khác nhau có thể tạo ra các khuôn mẫu với chất lượng và độ phức tạp khác nhau. Đôi khi ta viết khuôn mẫu prompt với các chỉ dẫn quá phức tạp hoặc dài dòng. Đôi khi ta thậm chí không biết chính xác nhiệm vụ đích là gì và nên mô tả nó thế nào. Một chiến lược được áp dụng rộng rãi là tạo khuôn mẫu prompt cho các nhiệm vụ NLP sẵn có, vì đã có rất nhiều bài toán và bộ đánh giá chuẩn NLP được xây dựng hoàn chỉnh [Bach và cộng sự, 2022; Wang và cộng sự, 2022e; Mishra và cộng sự, 2022]. Trong trường hợp này, người gán

[1] **[ND]** \n là ký hiệu xuống dòng: chỉ dẫn và đầu vào của người dùng được nối với nhau thành một chuỗi văn bản, ngăn cách bởi một dấu xuống dòng.

nhãn có thể được cung cấp mô tả nhiệm vụ gốc cùng nhiều ví dụ. Sau đó, họ có thể dùng cách riêng của mình để diễn đạt việc prompt LLM thực hiện các nhiệm vụ đó. Lưu ý rằng, mặc dù phương pháp này có thể giúp việc tạo prompt dễ dàng hơn, ta vẫn cần các khung gán nhãn và hệ thống huy động cộng đồng (*crowdsourcing*) để quản lý công việc và kiểm soát chất lượng. Ví dụ, nhìn chung ta cần thiết kế hướng dẫn gán nhãn và một định dạng thống nhất để viết khuôn mẫu prompt, đặc biệt khi có nhiều người gán nhãn cùng đóng góp cho một nhiệm vụ. Một ưu điểm của việc xây dựng prompt từ các nhiệm vụ NLP sẵn có là, một khi các khuôn mẫu prompt đã được xây dựng, ta dễ dàng sinh ra các ví dụ tinh chỉnh bằng các mẫu đã được gán nhãn trong các tập dữ liệu gốc. Ví dụ, với một tập dữ liệu song ngữ cho dịch Anh–Trung, ta có thể dễ dàng tạo ra nhiều ví dụ tinh chỉnh bằng cách điền các cặp câu trong tập dữ liệu này vào các chỗ trống của khuôn mẫu trên.

Một cách tiếp cận khác là trực tiếp dùng dữ liệu có sẵn tự nhiên trên Internet. Một ví dụ phổ biến là thu thập các cặp hỏi–đáp từ các trang web hỏi đáp để tinh chỉnh LLM cho các nhiệm vụ hỏi đáp miền mở (*open-domain QA*) [Joshi và cộng sự, 2017]. Nhiều bộ đánh giá chuẩn về hỏi đáp được xây dựng theo cách này, vì có quá nhiều loại câu hỏi đến mức một nhóm nhỏ người gán nhãn khó lường trước được mọi loại câu hỏi có thể có. Thay vào đó, dùng dữ liệu từ các trang web ấy có thể bảo đảm dữ liệu tinh chỉnh LLM đủ lớn và đủ đa dạng.

Ngoài việc sử dụng các nguồn tài nguyên sẵn có, một cách đơn giản khác để xây dựng tập dữ liệu tinh chỉnh là huy động cộng đồng tạo dữ liệu. Một cách đơn giản là cho phép người dùng nhập bất kỳ câu hỏi nào, sau đó câu trả lời hoặc được viết thủ công, hoặc được LLM sinh tự động rồi được người gán nhãn xem xét và sửa chữa. Nhờ vậy, ta có thể nắm bắt hành vi thực của người dùng và từ đó thu thập đầu vào và đầu ra cho rất nhiều bài toán mới mà các nhiệm vụ NLP truyền thống không bao quát.

Một vấn đề liên quan đến việc xây dựng tập dữ liệu tinh chỉnh là ta thường muốn dữ liệu càng đa dạng càng tốt. Nhiều nghiên cứu đã phát hiện rằng tăng tính đa dạng của dữ liệu tinh chỉnh có thể cải thiện độ bền vững (*robustness*) và khả năng tổng quát hóa của LLM. Vì lý do này, đã có nhiều quan tâm đến việc đưa thêm các prompt và nhiệm vụ đa dạng hơn vào các tập dữ liệu tinh chỉnh LLM. Ta sẽ bàn thêm về tổng quát hóa của tinh chỉnh ở Mục 10.2.4.

2. Dữ liệu được sinh tự động

Một hạn chế của việc tạo dữ liệu thủ công là chất lượng và tính đa dạng phụ thuộc rất nhiều vào kinh nghiệm và sự sáng tạo của con người. Do đó, nếu ta muốn LLM tổng quát hóa được cho các chỉ dẫn đa dạng của người dùng, việc dựa vào dữ liệu do con người gán nhãn để tinh chỉnh LLM thường kém hiệu quả. Hơn nữa, độ bao phủ của dữ liệu như vậy có thể hạn chế, và dữ liệu thậm chí có thể chứa những thiên kiến do chính người gán nhãn đưa vào. Một cách tiếp cận thay thế là sinh dữ liệu tự động. Ví dụ, ta có thể thu thập một số câu hỏi bằng cách huy động cộng đồng, rồi dùng một LLM đã được tinh chỉnh tốt để sinh câu trả lời cho các câu hỏi đó. Các cặp câu hỏi–câu trả lời này sau đó được dùng làm mẫu tinh chỉnh như thường lệ. Phương pháp này, dù rất đơn giản, đã được áp dụng rộng rãi để sinh dữ liệu tinh chỉnh quy mô lớn cho LLM.

Cách tạo dữ liệu tinh chỉnh tổng hợp nêu trên tương tự các phương pháp dùng trong tăng cường dữ liệu (*data augmentation*) của NLP. Nếu có một LLM, ta có thể tạo ra một đầu ra đáp lại bất kỳ đầu vào nào. Lặp lại quá trình này cho nhiều đầu vào khác nhau cho phép ta tạo ra lượng lớn dữ liệu tinh chỉnh. Phương pháp như vậy đặc biệt hữu ích khi tinh chỉnh các LLM mới bằng một LLM đã được tinh chỉnh tốt. Tuy nhiên, một nhược điểm của cách này là nó dựa vào các đầu vào do con người soạn hoặc thu thập để sinh dữ liệu, và những đầu vào đó có thể hóa ra không phù hợp cho việc tổng quát hóa LLM. Trong nhiều ứng dụng LLM, một thách thức lớn nảy sinh từ phạm vi rất rộng của các câu hỏi và yêu cầu của người dùng, nhiều trong số đó không được bao quát trong các nhiệm vụ và tập dữ liệu NLP hiện có. Trong những trường hợp này, cần phải sinh ra không chỉ các dự đoán mà cả chính các đầu vào.

Ở đây ta xét phương pháp **tự tạo chỉ dẫn** (*self-instruct*) làm ví dụ để minh họa cách sinh mẫu tinh chỉnh LLM [Wang và cộng sự, 2023e; Honovich và cộng sự, 2023]. Ý tưởng là ta có thể prompt một LLM để tạo ra một chỉ dẫn mới bằng cách học từ các chỉ dẫn khác. Với chỉ dẫn này, LLM sau đó có thể điền các trường khác (chẳng hạn đầu vào của người dùng) và sinh ra đầu ra. Hình 10.4 là sơ đồ minh họa phương pháp tự tạo chỉ dẫn. Dưới đây ta phác thảo ngắn gọn các bước chính.

- Thuật toán tự tạo chỉ dẫn duy trì một *kho nhiệm vụ*. Ban đầu, kho chứa một số nhiệm vụ hạt giống do con người soạn, mỗi nhiệm vụ

gồm một chỉ dẫn và một mẫu đầu vào–đầu ra. Khi thuật toán chạy, các chỉ dẫn và mẫu do LLM sinh ra sẽ được thêm vào kho này.

- Ở mỗi bước, một số ít chỉ dẫn được rút ra từ kho chỉ dẫn. Ví dụ, ta có thể chọn ngẫu nhiên vài chỉ dẫn do con người viết và vài chỉ dẫn do LLM sinh ra để bảo đảm tính đa dạng.

- Các chỉ dẫn được chọn được dùng làm ví dụ minh họa trong ngữ cảnh để sinh ra một chỉ dẫn mới. Dưới đây là một khuôn mẫu ví dụ để prompt LLM.

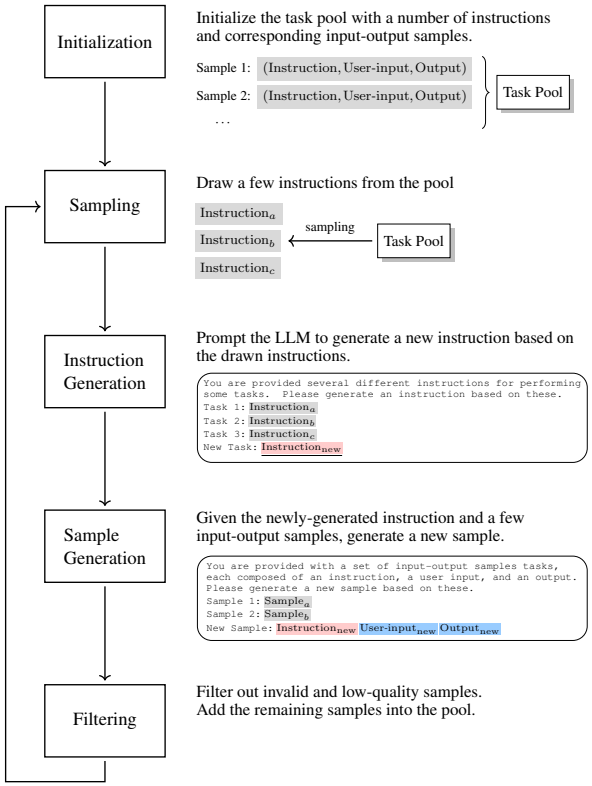
> Bạn được cung cấp một vài chỉ dẫn khác nhau để thực hiện một số nhiệm vụ. Hãy sinh ra một chỉ dẫn dựa trên những chỉ dẫn này.
> Nhiệm vụ 1: {∗instruction$_1$∗}
> Nhiệm vụ 2: {∗instruction$_2$∗}
> Nhiệm vụ 3: {∗instruction$_3$∗}
> Nhiệm vụ 4: {∗instruction$_4$∗}
> Nhiệm vụ mới: ___

- Với chỉ dẫn vừa sinh ra, LLM tiếp tục được prompt để hoàn thiện mẫu bằng cách điền các trường đầu vào còn lại và sinh ra đầu ra tương ứng. Dưới đây là một khuôn mẫu prompt.

> Bạn được cung cấp một tập các mẫu đầu vào–đầu ra, mỗi mẫu gồm một chỉ dẫn, một đầu vào của người dùng và một đầu ra. Hãy sinh ra một mẫu mới dựa trên các mẫu này.
> Mẫu 1: {∗instruction$_1$∗}
> Đầu vào: {∗user-input$_1$∗}
> Đầu ra: {∗output$_1$∗}
> Mẫu 2: {∗instruction$_2$∗}
> Đầu vào: {∗user-input$_2$∗}
> Đầu ra: {∗output$_2$∗}
> Mẫu mới: {∗new-instruction∗}
> ___

- Mẫu mới sinh ra này được đánh giá hoặc lọc bằng các quy tắc kinh nghiệm (chẳng hạn loại bỏ các mẫu hoặc chỉ dẫn giống với những mẫu đã có trong kho). Nếu vượt qua, mẫu và chỉ dẫn được thêm vào kho.

2



Hình 10.4: Minh họa phương pháp tự tạo chỉ dẫn [Wang và cộng sự, 2023c]. Phương pháp này duy trì một kho chỉ dẫn và các mẫu đầu vào–đầu ra tương ứng. Ban đầu, kho chứa một số chỉ dẫn và mẫu do con người soạn. Mỗi lần, ta rút một vài chỉ dẫn từ kho. Sau đó, một LLM được prompt để sinh ra các chỉ dẫn và mẫu mới dựa trên những chỉ dẫn đã rút. Cuối cùng, các chỉ dẫn và mẫu mới sinh được lọc rồi thêm vào kho.

[2] **[ND]** Trong các khuôn mẫu trên, *instruction* = chỉ dẫn, *user-input* = đầu vào của người dùng, *output* = đầu ra, *new-instruction* = chỉ dẫn mới; mỗi {∗…∗} là một biến sẽ được thay bằng nội dung cụ thể.

Chú giải của người dịch: Đọc Hình 10.4 và ý tưởng "tự tạo chỉ dẫn"

Năm bước trong hình (từ trên xuống): *Initialization* — khởi tạo kho nhiệm vụ (*Task Pool*) bằng một số chỉ dẫn và mẫu (*Instruction, User-input, Output*); *Sampling* — rút vài chỉ dẫn từ kho; *Instruction Generation* — prompt LLM sinh chỉ dẫn mới từ các chỉ dẫn đã rút; *Sample Generation* — với chỉ dẫn mới và vài mẫu, sinh ra một mẫu mới; *Filtering* — loại bỏ mẫu không hợp lệ hoặc kém chất lượng, thêm phần còn lại vào kho. Các hộp chữ nhỏ trong hình chính là hai khuôn mẫu prompt đã dịch ở trên.

Hình dung một giáo viên có sẵn 10 đề bài mẫu, nhờ một trợ giảng giỏi "nhìn mấy đề này rồi ra thêm một đề tương tự nhưng khác", sau đó tự giải luôn đề mới. Đề nào trùng lặp hay sai thì bỏ, đề tốt thì đưa vào ngân hàng đề. Lặp lại nhiều lần, ngân hàng đề lớn dần từ vài chục lên hàng chục nghìn đề — đó là cách LLM "tự làm" ra dữ liệu để huấn luyện.

Quá trình sinh này có thể được lặp lại nhiều lần để thu được đủ số lượng mẫu tinh chỉnh. Lưu ý rằng ở trên ta chỉ trình bày các khuôn mẫu prompt đơn giản để sinh chỉ dẫn và mẫu tinh chỉnh. Tất nhiên, ta có thể xây dựng các khuôn mẫu tốt hơn để sinh ra chỉ dẫn và mẫu tinh chỉnh đa dạng và chính xác hơn. Ví dụ, với một số nhiệm vụ như phân loại văn bản, LLM có thể có xu hướng đưa ra các dự đoán thiên lệch, chẳng hạn hầu hết các mẫu sinh ra đều thuộc về một lớp duy nhất. Trong những trường hợp như vậy, ta có thể điều chỉnh thứ tự sinh các trường khác nhau. Cụ thể hơn, ta có thể chỉ định trước đầu ra (tức là lớp) theo một phân phối tiên nghiệm nào đó, rồi prompt LLM sinh ra đầu vào của người dùng khi biết cả chỉ dẫn lẫn đầu ra. Phương pháp này giống với đảo ngược đầu vào (*input inversion*), trong đó LLM sinh ra đầu vào dựa trên đầu ra đã được chỉ định [Longpre và cộng sự, 2023].

Dùng các chỉ dẫn và mẫu tinh chỉnh do LLM sinh ra đã trở thành một phương pháp phổ biến để phát triển LLM, nhất là khi việc xây dựng dữ liệu như vậy bằng tay tốn kém đến mức hầu hết các nhóm nghiên cứu không kham nổi. Ở một số LLM được tinh chỉnh tốt, tập dữ liệu tinh chỉnh của chúng bao gồm một lượng dữ liệu tổng hợp nhất định, và điều này đã được chứng tỏ là hiệu quả [Ouyang và cộng sự, 2022; Taori và cộng sự, 2023; Chiang và cộng sự, 2023b]. Đã có thêm nhiều nghiên cứu về sinh dữ liệu tổng hợp cho tinh chỉnh LLM. Ví dụ, ta có thể sinh ra các chỉ dẫn đa dạng hơn bằng cách đưa vào các thuật toán tiến hóa (*evolutionary algorithms*)

[Xu và cộng sự, 2024], hoặc dùng dữ liệu tổng hợp làm tín hiệu giám sát trong một quá trình tinh chỉnh tiên tiến hơn [Chen và cộng sự, 2024b]. Gần đây hơn, cũng có nhiều quan tâm đến việc dùng dữ liệu tổng hợp ngay trong giai đoạn tiền huấn luyện [Gunasekar và cộng sự, 2023; Allal và cộng sự, 2024].

Trong nhiều ứng dụng, một tình huống thực tế là: với một nhiệm vụ cho trước, ta có thể thu thập hoặc gán nhãn một lượng tương đối nhỏ dữ liệu tinh chỉnh, chẳng hạn thuê chuyên gia soạn câu hỏi cho các nhiệm vụ hỏi đáp trong một lĩnh vực cụ thể. Nhưng số lượng và độ đa dạng của dữ liệu này nhìn chung là không đủ. Trong trường hợp này, ta có thể dùng các kỹ thuật tự tạo chỉ dẫn để sinh thêm các cặp câu hỏi–câu trả lời đa dạng hơn, nhờ đó tăng cường dữ liệu tinh chỉnh. Điều này cung cấp một cách "tự nâng" (*bootstrapping*) LLM bắt đầu từ một tập mẫu tinh chỉnh hạt giống. Lưu ý rằng dùng dữ liệu tự sinh là một thực hành phổ biến và đã được áp dụng từ lâu trong NLP. Ví dụ, cách tiếp cận này đã được dùng thành công trong phân tích cú pháp và dịch máy [Charniak, 1997; Sennrich và cộng sự, 2016a].

10.2.3 Tinh chỉnh với ít dữ liệu hơn

Với vai trò ngày càng nổi bật của tinh chỉnh theo chỉ dẫn, nhu cầu về dữ liệu tinh chỉnh quy mô lớn, chất lượng cao đã tăng vọt. Ví dụ, tập dữ liệu tinh chỉnh FLAN, được tổng hợp từ 1,836 nhiệm vụ, chứa 15 triệu mẫu [Longpre và cộng sự, 2023].[3] Tinh chỉnh LLM với các tập dữ liệu lớn như vậy thường là một nhiệm vụ tốn kém về tính toán, đặc biệt vì việc cập nhật số lượng lớn tham số của LLM tiêu tốn nhiều tài nguyên. Một cách để giảm nhẹ vấn đề này là tìm các phương pháp huấn luyện mô hình hiệu quả, chẳng hạn dùng các phương pháp hiệu quả tham số để chỉ cập nhật một phần nhỏ của mô hình.

Một cách tiếp cận khác là tinh chỉnh LLM chỉ bằng những mẫu huấn luyện giàu thông tin nhất. Nhờ đó ta có thể giảm lượng dữ liệu cần xử lý mà vẫn duy trì hiệu năng tinh chỉnh. Có một số phương pháp để làm điều này. Ví dụ, Zhou và cộng sự [2023a] đã xây dựng một tập dữ liệu làm theo chỉ dẫn chỉ gồm 1,000 mẫu bằng cách soạn prompt cẩn thận và thu thập mẫu từ nhiều nhiệm vụ NLP khác nhau. Họ cho thấy mô hình

[3] **[ND]** Giữ cách viết số như bản gốc: 1,836 = 1 836 nhiệm vụ.

LLaMa 65B được tinh chỉnh trên tập dữ liệu này có thể cạnh tranh ngang ngửa, thậm chí tốt hơn các mô hình được tinh chỉnh công phu hơn nhiều. Điều này gợi ý rằng LLM có thể được thích nghi để đáp ứng các nhiệm vụ đa dạng mà không nhất thiết phải tinh chỉnh trên mọi loại dữ liệu làm theo chỉ dẫn. Chen và cộng sự [2024a] đã phát triển một hệ thống dựa trên mô hình GPT-3.5 để đánh giá chất lượng của từng mẫu làm theo chỉ dẫn. Nhờ vậy, họ có thể chọn ra các mẫu chất lượng cao từ các tập dữ liệu sẵn có, cho hiệu năng tinh chỉnh tốt hơn với ít mẫu tinh chỉnh hơn. Các nhà nghiên cứu cũng đã phát triển những phương pháp chọn lọc hoặc loại bỏ dữ liệu bằng các quy tắc kinh nghiệm [Zhao và cộng sự, 2024; Ge và cộng sự, 2024], hoặc ưu tiên những dữ liệu có ảnh hưởng lớn hơn đến quá trình tinh chỉnh [Xia và cộng sự, 2024]. Thực ra, hầu hết các phương pháp này có thể được xem là trường hợp riêng của những họ phương pháp lựa chọn và lọc dữ liệu rộng hơn. Và thường thì dùng dữ liệu chất lượng cao hơn (dù có thể ít hơn) sẽ có lợi cho việc huấn luyện các mô hình NLP.

Những phát hiện trong tinh chỉnh theo chỉ dẫn có phần khác với quan điểm truyền thống trong NLP: khả năng xử lý các bài toán phức tạp của mô hình có thể được "kích hoạt" bằng một lượng nhỏ dữ liệu có gán nhãn, thay vì cần lượng dữ liệu có giám sát khổng lồ để huấn luyện kỹ lưỡng. Một cách giải thích khả dĩ là khả năng sinh câu trả lời đúng theo chỉ dẫn đã được học trong quá trình tiền huấn luyện, nhưng LLM có thể không gán xác suất cao cho những câu trả lời như vậy lúc suy luận. Tinh chỉnh có thể điều chỉnh nhẹ mô hình để nó làm theo chỉ dẫn, và do đó cần ít công sức huấn luyện hơn đáng kể so với tiền huấn luyện. Điều này liên quan chặt chẽ đến cái gọi là **giả thuyết căn chỉnh bề mặt** (*superficial alignment hypothesis*), cho rằng việc học chủ yếu diễn ra trong tiền huấn luyện, còn giai đoạn tinh chỉnh hay căn chỉnh sau đó không đóng góp đáng kể vào nền tảng tri thức của một LLM [Zhou và cộng sự, 2023a]. Vì năng lực cốt lõi và tri thức của mô hình đã được hình thành từ tiền huấn luyện, việc tinh chỉnh hiệu quả để căn chỉnh với nhu cầu người dùng có thể đạt được với công sức tinh chỉnh tương đối nhỏ. Điều này hàm ý khả năng tinh chỉnh LLM với rất ít dữ liệu. Theo một hướng khác, có thể không cần giới hạn việc tinh chỉnh vào dữ liệu theo cặp chỉ dẫn–câu trả lời. Ví dụ, Hewitt và cộng sự [2024] phát hiện rằng khả năng làm theo chỉ dẫn có thể đạt được một cách ngầm định bằng cách tinh chỉnh LLM chỉ trên các câu trả lời, không cần các chỉ dẫn tương ứng.

Chú giải của người dịch: Giả thuyết căn chỉnh bề mặt

Theo giả thuyết này, hầu như mọi "kiến thức" và "kỹ năng" của LLM đến từ tiền huấn luyện; tinh chỉnh chỉ dạy mô hình *cách trình bày* — ví dụ: khi thấy một câu hỏi thì trả lời thẳng vào câu hỏi, theo văn phong của một trợ lý. Giống như một học sinh đã học giỏi kiến thức, chỉ cần được hướng dẫn vài buổi về cách trình bày bài thi là làm bài tốt, không cần học lại từ đầu. Vì vậy, chỉ khoảng 1 000 mẫu được chọn kỹ đã có thể đủ.

Một khái niệm liên quan đến thảo luận ở đây là **hiệu quả mẫu** (*sample efficiency*). Một phương pháp học máy được gọi là hiệu quả mẫu nếu nó có thể học tốt từ một số ít mẫu huấn luyện. Theo nghĩa này, tinh chỉnh theo chỉ dẫn là hiệu quả mẫu so với tiền huấn luyện. Từ góc nhìn học máy, các phương pháp hiệu quả mẫu có thể được xem là những cách hiệu quả để lấy mẫu không gian dữ liệu, và có lợi thế vì chúng tận dụng tối ưu dữ liệu khan hiếm. Những ý tưởng này cũng liên quan đến các bối cảnh học tăng cường, nơi hiệu quả mẫu đặc biệt quan trọng. Do đó, các kỹ thuật học dựa trên lấy mẫu, chẳng hạn nhiều thuật toán học tăng cường, có thể được hưởng lợi từ những cách tiếp cận hiệu quả mẫu này. Ví dụ, trong căn chỉnh theo sở thích của con người, ta có thể lấy mẫu dữ liệu sở thích một cách hiệu quả thông qua mô hình phần thưởng [Liu và cộng sự, 2024b] hoặc cải thiện hiệu quả lấy mẫu trong việc học chính sách (*policy*) [Wang và cộng sự, 2024a].

10.2.4 Tổng quát hóa chỉ dẫn

Trong nhiều bài toán học máy và NLP, huấn luyện một mô hình có khả năng tổng quát hóa (*generalization*) là một mục tiêu cơ bản. Ví dụ, trong phân loại văn bản, ta muốn mô hình phân loại đúng những văn bản mới chưa từng thấy trong huấn luyện. Tuy nhiên, tổng quát hóa đặt ra thêm những thách thức trong tinh chỉnh theo chỉ dẫn. Ta muốn các LLM đã tinh chỉnh theo chỉ dẫn không chỉ sinh ra câu trả lời phù hợp cho các đầu vào khác nhau trong cùng một nhiệm vụ, mà còn thực hiện chính xác nhiều nhiệm vụ khác nhau được mô tả bằng những chỉ dẫn khác nhau. Để minh họa vấn đề này, xét một LLM $\Pr(\mathbf{y}|\mathbf{c}, \mathbf{z})$, trong đó $\mathbf{c}$ là một chỉ dẫn, $\mathbf{z}$ là một đầu vào của người dùng, và $\mathbf{y}$ là đầu ra tương ứng của mô hình (tức là câu trả lời). Giả sử hiệu năng của mô hình này được đánh giá theo một độ

đo, viết là Performance($\Pr(\mathbf{y}|\mathbf{c},\mathbf{z})$), gọi tắt là $\mathrm{P}(\mathbf{c},\mathbf{z},\mathbf{y})$. Một cách không hình thức, khi ta nói mô hình này có thể tổng quát hóa trong một nhiệm vụ cho trước (được chỉ định bởi chỉ dẫn $\mathbf{c}^*$), tức là tồn tại một ngưỡng ϵ nào đó sao cho hiệu năng trung bình trên các đầu vào mới lớn hơn giá trị này:

$$\frac{1}{|\mathcal{Z}|}\sum_{\mathbf{z}'\in\mathcal{Z}}\mathrm{P}(\mathbf{c}^*,\mathbf{z}',\mathbf{y}') > \epsilon, \tag{10.8}$$

trong đó $\mathcal{Z}$ là tập các đầu vào mới, $\mathbf{z}'$ là một đầu vào, và $\mathbf{y}'$ là đầu ra của mô hình tương ứng với đầu vào $\mathbf{z}'$.

Tương tự, ta có thể nói mô hình này tổng quát hóa được giữa các nhiệm vụ nếu hiệu năng trung bình trên tất cả các cặp chỉ dẫn–đầu vào lớn hơn một ngưỡng ϵ nào đó:

$$\frac{1}{|\mathcal{D}|}\sum_{(\mathbf{c}',\mathbf{z}')\in\mathcal{D}}\mathrm{P}(\mathbf{c}',\mathbf{z}',\mathbf{y}') > \epsilon, \tag{10.9}$$

trong đó $\mathcal{D}$ là tập các cặp chỉ dẫn–đầu vào mới.

Chú giải của người dịch: Đọc Phương trình (10.8) và (10.9)

Cả hai công thức đều chỉ là "điểm trung bình phải vượt ngưỡng". $|\mathcal{Z}|$ là số phần tử của tập $\mathcal{Z}$; tổng chia cho số phần tử chính là trung bình cộng quen thuộc trong thống kê. Ở (10.8), chỉ dẫn cố định ($\mathbf{c}^*$, ví dụ "Dịch sang tiếng Pháp"), chỉ thay đổi đầu vào: mô hình dịch tốt cả những câu chưa gặp. Ở (10.9), cả chỉ dẫn lẫn đầu vào đều mới: mô hình làm tốt cả những *loại việc* chưa gặp. Ví dụ: kiểm tra trên 4 cặp mới, điểm lần lượt 0.9, 0.7, 0.8, 0.6 thì trung bình là 0.75; nếu chọn ngưỡng $\epsilon = 0.7$ thì ta nói mô hình tổng quát hóa được.

Ở đây, ta phải xử lý sự biến thiên theo hai chiều: chỉ dẫn và đầu vào của người dùng. Điều này làm bài toán tổng quát hóa trở nên rất phức tạp, vì theo trực giác, một mô hình cần học từ rất nhiều nhiệm vụ và nhiều cặp đầu vào–đầu ra khác nhau ứng với mỗi nhiệm vụ để đạt được khả năng tổng quát hóa tốt. Như ta đã bàn nhiều lần trong sách này, đạt được khả năng tổng quát hóa như vậy tốn ít chi phí hơn nhiều so với tiền huấn luyện. Nhìn chung, tinh chỉnh LLM bằng dữ liệu chỉ dẫn–câu trả lời ở một mức độ nào đó có thể giúp mô hình bộc lộ khả năng làm theo chỉ dẫn trên các nhiệm vụ mới. Tuy vậy, người ta thường cho rằng vẫn cần một số nỗ lực

nhất định để thích nghi LLM sao cho chúng hiểu và thực hiện được chỉ dẫn một cách rộng rãi.

Một cách để tổng quát hóa tinh chỉnh theo chỉ dẫn là tăng tính đa dạng của dữ liệu tinh chỉnh. Trong các nghiên cứu ban đầu về tinh chỉnh theo chỉ dẫn, các nhà nghiên cứu đã xây dựng nhiều tập dữ liệu bao quát rất nhiều nhiệm vụ NLP và các chỉ dẫn khác nhau cho mỗi nhiệm vụ [Wang và cộng sự, 2022e; Sanh và cộng sự, 2022; Longpre và cộng sự, 2023]. Bằng cách chuyển các nhiệm vụ này về một định dạng thống nhất, ta có thể tinh chỉnh LLM với một số lượng mẫu đủ lớn. Ví dụ, một số tập dữ liệu tinh chỉnh theo chỉ dẫn chứa hơn 100 nhiệm vụ NLP và hơn một triệu mẫu. Tuy nhiên, các tập dữ liệu ban đầu này chủ yếu tập trung vào các bài toán học thuật sẵn có, chứ không phải những nhiệm vụ gặp trong ứng dụng thực tế. Nhiều nghiên cứu gần đây đã chuyển trọng tâm sang giải quyết các bài toán mới và thực tế hơn. Ví dụ, đã có nhiều quan tâm đến việc xây dựng các tập dữ liệu chứa những ví dụ minh họa lớn, phức tạp và các câu trả lời của những mô hình tiên tiến nhất (SOTA) cho truy vấn thực của người dùng [Wang và cộng sự, 2023d; Teknium, 2023].

Có lẽ việc dùng các tập dữ liệu tinh chỉnh lớn và đa dạng bắt nguồn từ những nỗ lực mở rộng quy mô LLM theo nhiều chiều khác nhau. Thật vậy, các định luật co giãn (*scaling laws*) đã được dùng rộng rãi làm động lực để phát triển rất nhiều LLM tinh chỉnh theo chỉ dẫn khác nhau. Và việc mở rộng quy mô tinh chỉnh theo chỉ dẫn để LLM làm theo được nhiều loại chỉ dẫn là hợp lý. Tuy nhiên, từ góc nhìn căn chỉnh LLM, mở rộng quy mô tinh chỉnh theo chỉ dẫn có thể không phải là cách hiệu quả nhất để đạt được khả năng tổng quát hóa.

Một vấn đề là tinh chỉnh theo chỉ dẫn dựa vào học có giám sát, vốn mô hình hóa sự tổng quát hóa giữa các nhiệm vụ dựa trên các ánh xạ chỉ dẫn–câu trả lời tường minh. Tuy nhiên, cách tiếp cận như vậy không nắm bắt được những sở thích tinh tế hay phức tạp của con người (ví dụ: giọng điệu, văn phong, hay chất lượng mang tính chủ quan), vì những điều này khó mã hóa thành dữ liệu chỉ dẫn–câu trả lời tường minh. Hơn nữa, hiệu năng tổng quát hóa bị giới hạn bởi tính đa dạng và chất lượng của tập dữ liệu chỉ dẫn–câu trả lời. Với những hạn chế này, ta muốn dùng thêm các mô hình sở thích như một bước tinh chỉnh bổ sung sau tinh chỉnh theo chỉ dẫn, để LLM có thể tổng quát hóa xa hơn (xem Mục 10.3).

Một quan điểm khác cho rằng một số ánh xạ chỉ dẫn–câu trả lời có

thể đã được học trong tiền huấn luyện. Tuy nhiên, vì ta thường không biết chính xác dữ liệu nào được dùng trong tiền huấn luyện, khó mà đánh giá liệu các ánh xạ đó có còn cần được học trong tinh chỉnh hay không. Một câu hỏi liên quan là: khả năng tổng quát hóa ngoài phân phối (*out-of-distribution generalization*) chủ yếu đạt được trong tiền huấn luyện hay trong tinh chỉnh? Trả lời trực tiếp câu hỏi này nằm ngoài phạm vi chương này, nhưng người ta đã chỉ ra rằng tiền huấn luyện trên các tập dữ liệu lớn và đa dạng giúp cải thiện hiệu năng ngoài phân phối một cách hiệu quả [Hendrycks và cộng sự, 2020; Radford và cộng sự, 2021; Gunasekar và cộng sự, 2023]. Điều này đặt ra một vấn đề thú vị: nếu một LLM đã được tiền huấn luyện tốt ở quy mô lớn, tinh chỉnh có thể không còn thiết yếu đến thế cho việc tổng quát hóa ngoài phân phối, vì mô hình có thể đã gặp đủ sự biến thiên về phân phối. Điều này thúc đẩy các nhà nghiên cứu tinh chỉnh LLM với công sức vừa phải, hoặc tìm các phương pháp mới để đạt được khả năng làm theo chỉ dẫn. Ví dụ, như đã bàn ở tiểu mục trước, khả năng làm theo chỉ dẫn có thể đạt được bằng cách tinh chỉnh trên một số ít cặp chỉ dẫn–câu trả lời được chọn lọc kỹ [Zhou và cộng sự, 2023a], hoặc thậm chí bằng những phương pháp vốn không được thiết kế tường minh cho mục đích đó [Kung và Peng, 2023].

Chú giải của người dịch: "Ngoài phân phối" là gì?

Dữ liệu *trong phân phối* là dữ liệu "giống" với dữ liệu huấn luyện (cùng loại, cùng phong cách); dữ liệu *ngoài phân phối* là dữ liệu khác hẳn. Ví dụ: một mô hình chỉ được luyện trên các đề toán lớp 10, nay gặp một bài toán đố dạng truyện tranh — đó là tình huống ngoài phân phối. Mô hình tổng quát hóa ngoài phân phối tốt là mô hình vẫn làm tốt trong những tình huống "lạ" như vậy.

Thảo luận trên đưa ra hai chiến lược khác nhau: một là mở rộng quy mô tập dữ liệu tinh chỉnh để tăng tính đa dạng, hai là dùng các tập dữ liệu tinh chỉnh nhỏ nhưng thiết yếu để thích nghi LLM một cách hiệu quả. Tuy nhiên, trong thực tế, đưa vào các chỉ dẫn đa dạng thường có ích. Trong nhiều trường hợp, ta cần thích nghi LLM cho những mục đích cụ thể. Nhưng LLM, vốn có thể đã mã hóa các ánh xạ làm theo chỉ dẫn rộng rãi trong tiền huấn luyện, có thể có xu hướng hành xử như một "người thực thi chỉ dẫn" đa năng ngay cả khi chỉ được tinh chỉnh vừa phải. Một hiện tượng thú vị là khi tinh chỉnh trên dữ liệu toán học, LLM thu được có thể

không chuyên biệt hóa vào các đầu ra toán học. Thay vào đó, mô hình này vẫn có thể đáp lại bình thường các chỉ dẫn tổng quát, chẳng hạn nó có thể làm thơ nếu được yêu cầu [Hewitt, 2024]. Đây không phải là điều xấu, nhưng nó cho thấy LLM có thể không dễ thay đổi bản chất làm theo các chỉ dẫn tổng quát của mình. Trong trường hợp này, các bước thích nghi bổ sung với dữ liệu đa dạng hơn có thể giúp điều chỉnh cách LLM làm theo chỉ dẫn, đặc biệt là cho những nhiệm vụ mà ta nhắm tới.

10.2.5 Tổng quát hóa từ yếu đến mạnh

Cho đến giờ, ta đã khám phá nhiều phương pháp tinh chỉnh theo chỉ dẫn dựa trên dữ liệu có gán nhãn. Một hạn chế của nhiều phương pháp như vậy là chúng đòi hỏi dữ liệu được con người gán nhãn hoặc được sinh ra bởi các LLM mạnh, cả hai đều có thể cung cấp tín hiệu giám sát chính xác. Tuy nhiên, trong nhiều trường hợp, LLM ta đang có đã mạnh sẵn (hoặc ít nhất là xuất sắc ở một số khía cạnh giải quyết vấn đề), nên không dễ tìm được một mô hình giỏi hơn để giám sát nó. Ngay cả với chuyên gia con người, khi bài toán trở nên phức tạp, việc đưa ra câu trả lời đúng và chi tiết có thể rất khó, đôi khi bất khả thi. Ví dụ, khi đối mặt với một văn bản cực kỳ dài, các chuyên gia sẽ thấy khó phát hiện bất kỳ điểm thiếu nhất quán, thiên kiến tinh vi hay ý chính bị bỏ sót nào nếu không tiến hành một đợt rà soát toàn diện và tốn thời gian.

Đến đây có thể đặt câu hỏi: liệu ta có thể dùng các LLM yếu để giám sát các LLM mạnh? Đây có vẻ là một thách thức lớn, nhưng nó có thể phản ánh một viễn cảnh tương lai, khi ta cần giám sát những hệ thống AI thông minh hơn con người hay các hệ thống AI hiện có [Burns và cộng sự, 2023b]. Bài toán dùng các mô hình nhỏ hơn, kém phức tạp hơn để cải thiện việc huấn luyện các mô hình lớn hơn, phức tạp hơn còn được gọi là bài toán **tổng quát hóa từ yếu đến mạnh** (*weak-to-strong generalization*). Dù hiện chưa có các cách tiếp cận hoàn thiện cho tổng quát hóa từ yếu đến mạnh, việc dùng các mô hình nhỏ hơn để hỗ trợ các mô hình mạnh hơn quả thực đã tỏ ra hữu ích trong một số lĩnh vực của LLM.

Đối với tinh chỉnh theo chỉ dẫn, một trong những cách đơn giản nhất để tận dụng LLM yếu là dùng các mô hình này để sinh dữ liệu tinh chỉnh tổng hợp. Giả sử ta có một tập các đầu vào X, trong đó mỗi đầu vào gồm một chỉ dẫn và, nếu cần, một đầu vào của người dùng. Với mỗi $\mathbf{x} \in X$, ta

dùng một LLM yếu $\Pr^w(\cdot)$ để sinh ra một dự đoán $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr^w(\mathbf{y}|\mathbf{x})$. Sau đó, LLM mạnh $\Pr^s_\theta(\cdot)$ có thể được huấn luyện trên các dự đoán này (xem Phương trình (10.1)):

$$\tilde{\theta} \;=\; \arg\max_{\theta} \sum_{\mathbf{x}\in X} \log \Pr^s_\theta(\hat{\mathbf{y}}|\mathbf{x}), \tag{10.10}$$

trong đó θ biểu diễn các tham số của mô hình.

Dạng trên biến bài toán tinh chỉnh thành một bài toán chưng cất tri thức (*knowledge distillation*); nói cách khác, ta chưng cất tri thức từ mô hình yếu sang mô hình mạnh. Do đó, ta có thể dùng nhiều phương pháp chưng cất tri thức khác nhau để đạt mục tiêu này. Tuy nhiên, giải thích tinh chỉnh từ yếu đến mạnh từ góc nhìn chưng cất tri thức không hề đơn giản. Một mối lo ngại lớn là mô hình mạnh có thể chỉ bắt chước hoặc quá khớp các lỗi của mô hình yếu và không tổng quát hóa được. Ví dụ, mô hình mạnh sau tinh chỉnh vẫn không giải được những bài toán khó mà mô hình yếu không dự đoán chính xác được. May mắn là các thí nghiệm bước đầu theo hướng nghiên cứu này đã cho kết quả tích cực và đầy hứa hẹn. Ví dụ, Burns và cộng sự [2023a] phát hiện rằng tinh chỉnh các mô hình cỡ GPT-4 với sự giám sát từ mô hình cỡ GPT-2 có thể cải thiện khả năng tổng quát hóa trên một số nhiệm vụ NLP. Để đo mức độ mô hình yếu cải thiện khả năng tổng quát hóa của mô hình mạnh, ta định nghĩa các đại lượng sau:

- **Hiệu năng yếu** (*Weak Performance*, $\mathrm{P}_{\mathrm{weak}}$). Đây là hiệu năng trên tập kiểm tra của mô hình yếu, có thể xem là hiệu năng cơ sở.

- **Hiệu năng từ yếu đến mạnh** (*Weak-to-strong Performance*, $\mathrm{P}_{\mathrm{weak}\to\mathrm{strong}}$). Đây là hiệu năng trên tập kiểm tra của mô hình mạnh được tinh chỉnh bằng mô hình yếu.

- **Hiệu năng trần của mô hình mạnh** (*Strong Ceiling Performance*, $\mathrm{P}_{\mathrm{ceiling}}$). Đây là hiệu năng trên tập kiểm tra của mô hình mạnh được tinh chỉnh bằng dữ liệu nhãn đúng (*ground truth*). Ví dụ, ta tinh chỉnh mô hình mạnh bằng dữ liệu do con người gán nhãn và lấy mô hình thu được làm mức trần.

Khi đó, **tỉ lệ khoảng cách hiệu năng được thu hồi (PGR)** (*performance*

gap recovered) có thể được định nghĩa là

$$\text{PGR} \;=\; \max\left\{0,\; \frac{\text{P}_{\text{weak}\to\text{strong}} - \text{P}_{\text{weak}}}{\text{P}_{\text{ceiling}} - \text{P}_{\text{weak}}}\right\}. \tag{10.11}$$

Độ đo này cho biết mô hình từ yếu đến mạnh có thể thu hồi được bao nhiêu phần của khoảng cách giữa hiệu năng trần và hiệu năng của mô hình yếu. PGR bằng 1 cho thấy tinh chỉnh từ yếu đến mạnh có thể lấp hoàn toàn khoảng cách hiệu năng, còn PGR bằng 0 cho thấy không có cải thiện nào. Trong nghiên cứu của Burns và cộng sự [2023a], PGR được chỉ ra là có thể đạt khoảng 0.8 trên 22 nhiệm vụ phân loại NLP. Cần lưu ý rằng, dù tiềm năng của tinh chỉnh từ yếu đến mạnh là đầy hứa hẹn, việc đạt được khả năng tổng quát hóa từ yếu đến mạnh đáng kể vẫn là một mục tiêu đầy thách thức cần được nghiên cứu thêm [Aschenbrenner, 2024].

Chú giải của người dịch: Tính thử PGR

Giả sử trên một bài kiểm tra: mô hình yếu đúng 60% ($\text{P}_{\text{weak}} = 0.6$); mô hình mạnh học từ nhãn đúng của con người đạt 90% ($\text{P}_{\text{ceiling}} = 0.9$); mô hình mạnh học từ "bài giải" của mô hình yếu đạt 84% ($\text{P}_{\text{weak}\to\text{strong}} = 0.84$). Khoảng cách cần lấp là $0.9 - 0.6 = 0.3$; phần đã lấp được là $0.84 - 0.6 = 0.24$. Vậy $\text{PGR} = 0.24/0.3 = 0.8$: học trò giỏi học từ thầy kém hơn mình mà vẫn lấy lại được 80% khoảng cách so với khi học từ đáp án chuẩn. Phép $\max\{0, \cdot\}$ chỉ để PGR không âm khi mô hình mạnh còn tệ hơn cả mô hình yếu.

Tinh chỉnh LLM bằng giám sát yếu chỉ là một lựa chọn trong việc dùng mô hình nhỏ để cải thiện mô hình lớn. Mặc dù mục này chủ yếu tập trung vào tinh chỉnh LLM, ta cũng đề cập thêm các phương pháp khác ở đây để phần thảo luận đầy đủ hơn (xem minh họa các phương pháp này ở Hình 10.5).

- Thay vì dùng mô hình nhỏ để sinh dữ liệu tổng hợp, ta cũng có thể đơn giản đưa vào một hàm mất mát chưng cất tri thức dựa trên các mô hình này. Ví dụ, một hàm mất mát đơn giản đo sự khác biệt giữa mô hình nhỏ và mô hình lớn có thể được định nghĩa là:

$$\mathcal{L}_{\text{kd}} \;=\; \text{D}_{\text{KL}}\big(\,\Pr^{w}(\cdot|\mathbf{x}) \,\|\, \Pr^{s}_{\theta}(\cdot|\mathbf{x})\big), \tag{10.12}$$

  hàm này khuyến khích phân phối của mô hình mạnh khớp với phân phối của mô hình yếu. Sau đó, ta có thể cộng hàm mất mát này vào

hàm mất mát mô hình hóa ngôn ngữ ban đầu và thu được mục tiêu huấn luyện sau

$$\tilde{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_{\theta}^{s}(\mathbf{y}|\mathbf{x}) - \lambda \cdot \mathcal{L}_{\text{kd}}, \tag{10.13}$$

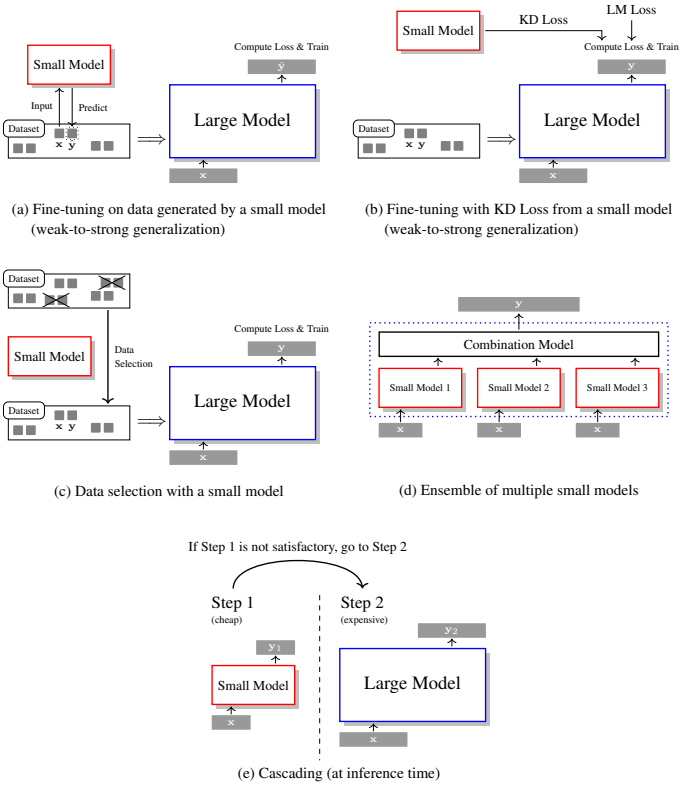
trong đó $\mathcal{D}$ là tập các cặp đầu vào và đầu ra, và λ là hệ số nội suy (*interpolation coefficient*). Phương pháp này có thể được dùng trong giai đoạn tiền huấn luyện hoặc tinh chỉnh. Ta có thể điều chỉnh λ để kiểm soát mức độ ảnh hưởng của mô hình nhỏ lên quá trình huấn luyện. Ví dụ, ta có thể giảm dần λ để việc huấn luyện dựa nhiều hơn vào hàm mất mát mô hình hóa ngôn ngữ ban đầu khi mô hình lớn trở nên giỏi hơn.

- Một cách khác để đưa mô hình nhỏ vào tiền huấn luyện và tinh chỉnh LLM là dùng chúng để chọn lọc hoặc lọc dữ liệu. Với một chuỗi cho trước, ta có thể tính độ hợp lý (likelihood) hoặc entropy chéo bằng một mô hình nhỏ. Các đại lượng này sau đó có thể được dùng làm tiêu chí để chọn lọc hoặc lọc dữ liệu. Ví dụ, các chuỗi có độ hợp lý thấp hoặc entropy chéo cao có thể bị loại khỏi tập huấn luyện, vì chúng ít khớp với phân phối mà mô hình nhỏ đã học. Ngược lại, các chuỗi có độ hợp lý cao hoặc entropy chéo thấp có thể được ưu tiên, bảo đảm việc huấn luyện tập trung vào dữ liệu liên quan hơn hoặc chất lượng cao hơn.

- Học kết hợp (*ensemble learning*) là một cách đơn giản và hiệu quả để xây dựng một mô hình mạnh bằng cách kết hợp nhiều mô hình yếu. Áp dụng kỹ thuật này cho LLM rất đơn giản. Ta có thể gộp các phân phối được dự đoán bởi nhiều mô hình nhỏ hoặc các mô hình con chuyên biệt, rồi rút ra dự đoán cuối cùng từ kết quả đã gộp. Việc gộp này có thể được thực hiện bằng các phương pháp như bỏ phiếu đa số (*majority voting*), trung bình có trọng số (*weighted averaging*), hoặc xếp chồng (*stacking*).

- Mô hình nhỏ cũng có thể được dùng lúc suy luận để cải thiện hiệu quả tổng thể. Giả sử ta có một mô hình lớn chậm nhưng chính xác hơn, và một mô hình nhỏ nhanh nhưng kém chính xác hơn. Trong phân tầng mô hình (*model cascading*), mô hình nhỏ xử lý dữ liệu đầu vào trước, nhanh chóng sinh ra kết quả sơ bộ. Nếu các kết quả

này thỏa mãn một số tiêu chí định trước, chúng có thể được dùng trực tiếp. Tuy nhiên, nếu kết quả ban đầu chưa đủ tốt, đầu vào sẽ được chuyển cho mô hình lớn hơn, chính xác hơn để tạo ra kết quả tốt hơn. Cách tiếp cận này giảm đáng kể chi phí tính toán và độ trễ, vì mô hình nhỏ có thể tự xử lý hiệu quả nhiều đầu vào mà không cần đến mô hình lớn.



(a) Fine-tuning on data generated by a small model (weak-to-strong generalization)

(b) Fine-tuning with KD Loss from a small model (weak-to-strong generalization)

(c) Data selection with a small model

(d) Ensemble of multiple small models

(e) Cascading (at inference time)

Hình 10.5: Minh họa các cách dùng mô hình nhỏ để cải thiện mô hình lớn trong LLM. Một cách tiếp cận là dùng các mô hình nhỏ hơn cho việc tinh chỉnh hoặc tiền huấn luyện các mô hình lớn hơn. Cách này bao gồm sinh dữ liệu tổng hợp (a), đưa vào hàm mất mát phụ trợ (b), và chọn lọc dữ liệu phù hợp (c). Một cách tiếp cận khác là kết hợp mô hình nhỏ và mô hình lớn. Cách này bao gồm học một mô hình mạnh bằng cách gộp nhiều mô hình nhỏ (d), và phân tầng mô hình nhỏ với mô hình lớn (e).

Chú giải của người dịch: Nhãn trong Hình 10.5 và ý nghĩa của Phương trình (10.12)

Các nhãn: *Small/Large Model* = mô hình nhỏ/lớn; *Dataset* = tập dữ liệu; *Predict* = dự đoán; *Compute Loss & Train* = tính mất mát và huấn luyện; *KD Loss* = mất mát chưng cất tri thức; *LM Loss* = mất mát mô hình hóa ngôn ngữ; *Data Selection* = chọn lọc dữ liệu; *Combination Model* = mô hình kết hợp; *Step 1 (cheap)/Step 2 (expensive)* = bước 1 (rẻ)/bước 2 (đắt); dòng chữ trên cùng của (e): "Nếu bước 1 chưa đạt yêu cầu, chuyển sang bước 2"; *Cascading (at inference time)* = phân tầng (lúc suy luận).
$\mathrm{D}_{\mathrm{KL}}(P \,\|\, Q)$ là *độ phân kỳ Kullback–Leibler*, một con số không âm đo hai phân phối xác suất P và Q "khác nhau" bao nhiêu; nó bằng 0 khi hai phân phối trùng nhau. Trong (10.13), ta vừa muốn mô hình mạnh đoán đúng dữ liệu (số hạng log-xác suất), vừa muốn nó không lệch quá xa "lời khuyên" của mô hình nhỏ (trừ đi $\lambda \cdot \mathcal{L}_{\mathrm{kd}}$). λ lớn thì nghe theo mô hình nhỏ nhiều hơn; $\lambda = 0$ thì bỏ qua mô hình nhỏ hoàn toàn.

10.3 Căn chỉnh theo sở thích của con người: RLHF

Cho đến giờ trong chương này, chúng ta tập trung vào việc tinh chỉnh LLM bằng các cặp đầu vào–đầu ra có gán nhãn. Cách tiếp cận này cho phép ta điều chỉnh LLM để làm theo chỉ dẫn thông qua học có giám sát. Tuy nhiên, trong nhiều ứng dụng, LLM không chỉ cần làm theo chỉ dẫn mà còn cần hành xử theo những cách phù hợp hơn với các giá trị và sở thích của con người. Hãy xét tình huống một người dùng hỏi LLM cách xâm nhập trái phép vào một hệ thống máy tính. Nếu LLM không được căn chỉnh thích đáng, nó có thể trả lời bằng cách cung cấp chi tiết về cách thực hiện hành vi phạm pháp này. Thay vào đó, một câu trả lời đáng mong muốn hơn có thể là khuyên người dùng không tham gia vào các hoạt động phạm pháp và đưa ra cái nhìn tổng quát về hậu quả của những hành vi như vậy. Khó khăn trong việc đạt được điều này là những sắc thái đạo đức và các cân nhắc theo ngữ cảnh mà LLM cần có để phản hồi phù hợp trong các tình huống như thế thường rất khó mã hóa một cách tường minh vào dữ liệu tinh chỉnh. Khó khăn hơn nữa là chính con người nhiều khi cũng không diễn đạt được chính xác sở thích của bản thân.

Trong mục này, chúng ta bàn về một phương pháp tinh chỉnh LLM khác, gọi là học tăng cường từ phản hồi của con người (*reinforcement learning from human feedback*), gọi tắt là RLHF [Christiano và cộng sự, 2017; Stiennon và cộng sự, 2020]. Ý tưởng cơ bản của RLHF là LLM có thể học từ việc so sánh các đầu ra của mô hình nhờ mô hình phần thưởng (*reward model*) (xem Hình 10.6). Để làm điều này, ta có thể thuê các chuyên gia là con người chỉ ra họ thích đầu ra nào hơn trong một cặp đầu ra của mô hình. Dữ liệu sở thích này được dùng để huấn luyện một mô hình phần thưởng có khả năng dự đoán chất lượng (theo cảm nhận của con người) của các đầu ra của LLM. Sau khi được huấn luyện, mô hình phần thưởng cung cấp phản hồi bằng cách chấm điểm cho các đầu ra mới mà LLM sinh ra đáp lại các đầu vào. LLM dùng các điểm số này để cập nhật tham số của mình thông qua các thuật toán học tăng cường (*reinforcement learning*). Trong phần còn lại của mục này, trước tiên chúng tôi giới thiệu các khái niệm cơ bản của học tăng cường để thuận tiện cho việc thảo luận, sau đó bàn về các phương pháp huấn luyện mô hình phần thưởng và căn chỉnh LLM bằng các mô hình này.

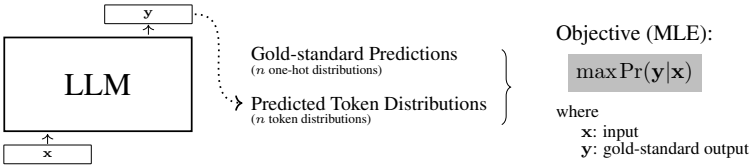
Chú giải của người dịch: Vì sao không chỉ dùng tinh chỉnh có giám sát?

Trong tinh chỉnh có giám sát (SFT), ta phải *viết sẵn* câu trả lời "chuẩn" cho từng câu hỏi. Nhưng với nhiều câu hỏi, rất khó viết ra một đáp án duy nhất đúng — trong khi việc *so sánh* hai câu trả lời và nói "cái này tốt hơn" lại dễ hơn nhiều. Giống như một giám khảo cuộc thi nấu ăn: ông ấy có thể không tự nấu được món hoàn hảo, nhưng nếm hai món thì dễ dàng nói món nào ngon hơn. RLHF tận dụng đúng khả năng "chấm so sánh" này của con người: (1) thu thập các phán đoán "A tốt hơn B"; (2) huấn luyện một mô hình phần thưởng bắt chước giám khảo; (3) cho LLM "luyện tập" để đạt điểm cao từ vị giám khảo máy đó.

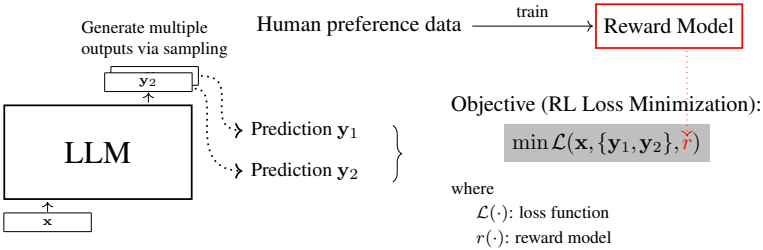
10.3.1 Cơ bản về học tăng cường

Chúng ta bắt đầu bằng việc xem xét một số khái niệm cơ bản của học tăng cường. Lưu ý rằng ký hiệu dùng ở đây hơi khác so với ký hiệu ở các mục và chương trước, vì chúng tôi muốn cách trình bày nhất quán hơn với tài liệu về học tăng cường. Dù vậy, chúng tôi sẽ chỉ ra các ký hiệu này tương ứng với ký hiệu của mô hình hóa ngôn ngữ như thế nào. Bạn đọc đã quen

với các kỹ thuật học tăng cường có thể bỏ qua hoặc đọc lướt tiểu mục này.



(a) Supervised fine-tuning (maximizing the prediction probability given the input)



(b) Reinforcement Learning from Human Feedback

Hình 10.6: Tinh chỉnh có giám sát so với học tăng cường từ phản hồi của con người. Trong tinh chỉnh có giám sát, ta tối ưu LLM bằng cách cực đại hóa xác suất của dự đoán khi biết đầu vào. Trong học tăng cường từ phản hồi của con người, trước tiên ta huấn luyện một mô hình phần thưởng trên dữ liệu sở thích của con người (với mỗi cặp dự đoán, người đánh giá được yêu cầu chọn cái mà họ thích hơn). Sau đó, ta dùng mô hình phần thưởng này để giám sát LLM trong quá trình tinh chỉnh.

Chú giải của người dịch: Đọc các nhãn trong Hình 10.6

(a) *Supervised fine-tuning*: tinh chỉnh có giám sát; *Gold-standard Predictions (n one-hot distributions)*: dự đoán chuẩn (n phân phối one-hot); *Predicted Token Distributions*: các phân phối token mà mô hình dự đoán; *Objective (MLE):* $\max \Pr(\mathbf{y}|\mathbf{x})$: mục tiêu ước lượng hợp lý cực đại, với $\mathbf{x}$ là đầu vào (*input*), $\mathbf{y}$ là đầu ra chuẩn (*gold-standard output*). (b) *Generate multiple outputs via sampling*: sinh nhiều đầu ra bằng lấy mẫu; *Prediction* $\mathbf{y}_1$, $\mathbf{y}_2$: hai dự đoán; *Human preference data* $\xrightarrow{\text{train}}$ *Reward Model*: dữ liệu sở

thích của con người dùng để huấn luyện mô hình phần thưởng; *Objective (RL Loss Minimization)*: mục tiêu là cực tiểu hóa mất mát học tăng cường $\mathcal{L}(\cdot)$, trong đó $r(\cdot)$ là mô hình phần thưởng.

Một khung học tăng cường tổng quát mô tả cách một tác tử (*agent*) tương tác với một môi trường (*environment*) động. Sự tương tác này được mô hình hóa thành một chuỗi các hành động (*action*) mà tác tử thực hiện để đáp lại trạng thái (*state*) của môi trường. Tại mỗi bước thời gian, tác tử quan sát trạng thái hiện tại, chọn một hành động dựa trên chính sách của nó, thực hiện hành động đó, rồi nhận phản hồi từ môi trường dưới dạng một phần thưởng và một trạng thái mới. Quá trình quan sát–hành động–phần thưởng này được lặp lại cho đến khi tác tử đạt được mục tiêu của mình.

Một hệ thống học tăng cường gồm một số thành phần:

- **Tác tử**. Đây là người học hay người ra quyết định trong học tăng cường. Trong bối cảnh LLM, có thể xem tác tử chính là LLM.

- **Môi trường**. Môi trường bao gồm mọi thứ bên ngoài tác tử mà tác tử tương tác cùng. Tuy nhiên, môi trường trong LLM không hẳn là một không gian vật lý hay ảo, mà đúng hơn là cái khung trong đó tác tử (ví dụ một LLM) nhận phản hồi và học.

- **Trạng thái** (s). Một trạng thái biểu diễn tình huống hiện tại của môi trường. Với một chuỗi token trong mô hình hóa ngôn ngữ, trạng thái tại một bước thời gian có thể được xem là các token ta đã quan sát được cho đến lúc đó, tức là các token ngữ cảnh mà ta dùng để dự đoán token tiếp theo. Ví dụ, ta có thể định nghĩa $(\mathbf{x}, \mathbf{y}_{<t})$ là trạng thái khi dự đoán token tiếp theo ở bước thời gian t.

- **Hành động** (a). Hành động biểu diễn các quyết định khả dĩ mà tác tử có thể đưa ra. Trong bối cảnh LLM, các hành động tương ứng với các token ứng viên trong bộ từ vựng.

- **Phần thưởng** (R). Phần thưởng là phản hồi từ môi trường, đánh giá mức độ thành công của một hành động. Ví dụ, $r(s, a, s')$ ký hiệu phần thưởng mà tác tử nhận được khi thực hiện hành động a ở trạng thái s và chuyển sang trạng thái tiếp theo s'. Nếu đã cho chuỗi trạng thái–hành động, ta có thể ký hiệu phần thưởng tại bước thời gian t

là $r_t = r(s_t, a_t, s_{t+1})$. Cũng lưu ý rằng nếu quá trình ra quyết định là tất định, ta có thể bỏ s_{t+1} vì nó được xác định bởi s_t và a_t. Trong những trường hợp như vậy, ta có thể dùng $r(s_t, a_t)$ làm cách viết tắt cho $r(s_t, a_t, s_{t+1})$.

- **Chính sách** (π). Với một LLM, chính sách được định nghĩa là phân phối xác suất trên các token mà LLM dự đoán, khi biết các token ngữ cảnh đứng trước. Một cách hình thức, điều này có thể được biểu diễn là

$$\pi(a|s) \;=\; \Pr(y_t|\mathbf{x}, \mathbf{y}_{<t}), \tag{10.14}$$

trong đó a tương ứng với token y_t, và s tương ứng với ngữ cảnh $(\mathbf{x}, \mathbf{y}_{<t})$. Hình 10.7 minh họa cách một LLM có thể được xem như một chính sách trong khung học tăng cường.

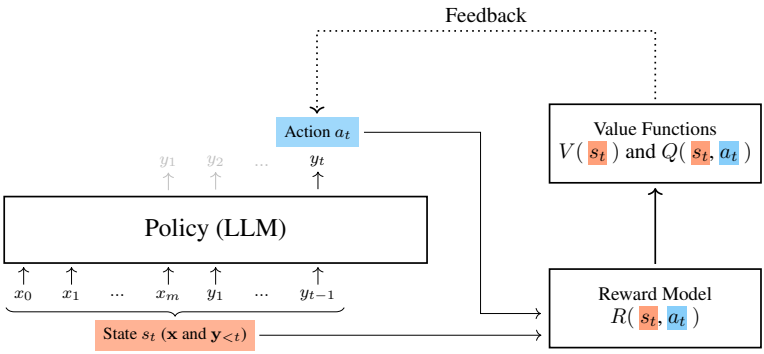
- **Hàm giá trị** (V và Q). Một **hàm giá trị trạng thái** (*state-value function*) (gọi tắt là hàm giá trị (*value function*)) đánh giá lợi tức chiết khấu (*discounted return*) kỳ vọng (tức tổng phần thưởng tích lũy) của một tác tử bắt đầu từ một trạng thái s cụ thể và đi theo một chính sách π cụ thể. Nó được định nghĩa là:

$$\begin{aligned} V(s) &= \mathbb{E}\Big[r(s_0, a_0, s_1) + \gamma r(s_1, a_1, s_2) + \\ &\qquad \gamma^2 r(s_2, a_2, s_3) + \cdots \;\Big|\; s_0 = s, \pi\Big] \\ &= \mathbb{E}\Big[r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots \;\Big|\; s_0 = s, \pi\Big] \\ &= \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t r_t \;\Big|\; s_0 = s, \pi\Big], \end{aligned} \tag{10.15}$$

trong đó $\gamma \in [0, 1]$ là hệ số chiết khấu (*discount factor*), điều chỉnh tầm quan trọng của các phần thưởng tương lai; $s_0 = s$ chỉ ra rằng tác tử bắt đầu từ trạng thái s; và kỳ vọng $\mathbb{E}$ được lấy trên mọi quỹ đạo (*trajectory*) khả dĩ (tức các chuỗi trạng thái–hành động). Tương tự, một **hàm giá trị hành động** (*action-value function*) (hay **hàm giá trị Q** (*Q-value function*)) đo lợi tức kỳ vọng khi bắt đầu từ trạng thái s, thực hiện hành động a, và sau đó đi theo chính sách π, được cho bởi

$$Q(s, a) \;=\; \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t r_t \;\Big|\; s_0 = s, a_0 = a, \pi\Big], \tag{10.16}$$

trong đó $a_0 = a$ chỉ ra rằng hành động được thực hiện ở trạng thái ban đầu là a.



Hình 10.7: LLM đóng vai trò chính sách trong học tăng cường. Tại mỗi bước t, LLM dự đoán một token y_t khi biết đầu vào mô hình $\mathbf{x}$ và các token đã sinh trước đó $\mathbf{y}_{<t}$. Quá trình này có thể được phát biểu thành một bài toán học tăng cường, trong đó y_t đóng vai trò hành động, $(\mathbf{x}, \mathbf{y}_{<t})$ là trạng thái, và phân phối dự đoán $\Pr(y_t|\mathbf{x}, \mathbf{y}_{<t})$ là chính sách. Mô hình phần thưởng đánh giá câu trả lời được sinh ra và gán điểm phần thưởng, rồi các điểm này được dùng để ước lượng các hàm giá trị như $V(s_t)$ và $Q(s_t, a_t)$ phục vụ việc tối ưu chính sách. Trong thực tế, mô hình phần thưởng trong RLHF thường được định nghĩa trên toàn bộ câu trả lời, dù mục tiêu tối ưu vẫn có thể được phân rã theo từng token.

Chú giải của người dịch: Học tăng cường qua ví dụ chú chó học lệnh

Hãy hình dung bạn dạy một chú chó ngồi theo lệnh.

- **Tác tử** là chú chó (người học); **môi trường** là mọi thứ xung quanh, kể cả bạn.
- **Trạng thái** là tình huống hiện tại: "chủ vừa hô *Ngồi!*, tay đang cầm bánh".
- **Hành động** là việc chó chọn làm: ngồi, sủa, chạy đi…
- **Phần thưởng** là phản hồi: được bánh ($+1$) hoặc không được gì (0).
- **Chính sách** là "thói quen" của chó: ở mỗi trạng thái, nó có xác suất bao nhiêu để chọn từng hành động, ví dụ $\pi(\text{ngồi}|\text{nghe lệnh}) = 0.7$, $\pi(\text{sủa}|\text{nghe lệnh}) = 0.3$. Học tăng cường là dần điều chỉnh các xác suất này để được nhiều bánh hơn.

Với LLM: *trạng thái* là câu hỏi cùng phần câu trả lời đã viết; *hành động* là chọn token tiếp theo; *chính sách* chính là phân phối xác suất trên bộ từ vựng mà LLM đưa ra; *phần thưởng* là điểm do mô hình phần thưởng chấm.

Chú giải của người dịch: Hàm giá trị, hệ số chiết khấu và kỳ vọng

Kỳ vọng $\mathbb{E}[\cdot]$ là "giá trị trung bình theo xác suất": nếu một trò chơi cho 10 điểm với xác suất 0.3 và 0 điểm với xác suất 0.7, thì kỳ vọng là $10 \cdot 0.3 + 0 \cdot 0.7 = 3$ điểm. Vì chính sách chọn hành động ngẫu nhiên, ta không biết trước tổng phần thưởng sẽ là bao nhiêu, nên lấy trung bình trên mọi khả năng.
Hệ số chiết khấu γ giống "lãi suất ngược": phần thưởng nhận ngay có giá trị hơn phần thưởng ở xa. Với $\gamma = 0.9$ và chuỗi phần thưởng $r_0 = 1, r_1 = 1, r_2 = 1$, lợi tức chiết khấu là $1 + 0.9 \cdot 1 + 0.81 \cdot 1 = 2.71$ (thay vì 3). Với $\gamma = 1$, mọi bước được coi trọng như nhau.
$V(s)$ trả lời câu hỏi: "Đang ở tình huống s, nếu cứ hành xử theo thói quen π thì trung bình về sau ta thu được bao nhiêu điểm?". $Q(s, a)$ hỏi chi tiết hơn: "Đang ở s, nếu *bước này* ta chọn a, rồi sau đó mới theo π, thì trung bình được bao nhiêu điểm?". Ví dụ khi chơi cờ: V là "thế cờ này tốt đến mức nào", còn Q là "nước đi này từ thế cờ này tốt đến mức nào".

Mục tiêu của học tăng cường là học một chính sách cực đại hóa **phần thưởng tích lũy** (*cumulative reward*) (hay **lợi tức** (*return*)) mà tác tử nhận được về lâu dài. Để đơn giản, trong các suy dẫn sau đây chúng tôi bỏ hệ số chiết khấu (tức $\gamma = 1$), trừ khi có nói khác. Cho một chuỗi trạng thái–hành động $\tau = \{(s_1, a_1), \ldots, (s_T, a_T)\}$[4], phần thưởng tích lũy trên chuỗi này có thể được viết là

$$R(\tau) \; = \; \sum_{t=1}^{T} r_t. \tag{10.17}$$

Kỳ vọng của phần thưởng tích lũy này trên một không gian các chuỗi trạng thái–hành động có dạng

$$J(\theta) = \mathbb{E}_{\tau \sim \mathcal{T}} \Big[R(\tau) \,\Big|\, \pi_\theta \Big]$$

[4]Chúng tôi giả sử chuỗi trạng thái–hành động bắt đầu bằng s_1 và a_1 thay vì s_0 và a_0, để thống nhất với ký hiệu thường dùng trong chương này, nơi dự đoán **y** thường bắt đầu từ y_1. Dĩ nhiên, trong tài liệu cũng thường ký hiệu chuỗi trạng thái–hành động là $\{(s_0, a_0), \ldots, (s_T, a_T)\}$ hoặc $\{(s_0, a_0), \ldots, (s_{T-1}, a_{T-1})\}$. Nhưng sự khác biệt về ký hiệu này không ảnh hưởng đến việc thảo luận các mô hình được trình bày ở đây.

$$= \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) R(\tau)$$

$$= \sum_{\tau \in \mathcal{T}} \Pr_\theta(\tau) \sum_{t=1}^{T} r_t, \tag{10.18}$$

trong đó $\tau \sim \mathcal{T}$ chỉ ra rằng τ được lấy ra từ không gian các chuỗi trạng thái–hành động $\mathcal{T}$, và chỉ số dưới θ chỉ các tham số của chính sách. $J(\theta)$ còn được gọi là **hàm hiệu năng** (*performance function*). Khi đó, mục tiêu huấn luyện là cực đại hóa $J(\theta)$:

$$\tilde{\theta} \;=\; \arg\max_{\theta} J(\theta). \tag{10.19}$$

Chú giải của người dịch: Hàm hiệu năng $J(\theta)$ bằng một ví dụ số

Giả sử chính sách chỉ có thể sinh ra hai quỹ đạo: τ_1 với xác suất $\Pr_\theta(\tau_1) = 0.6$ và tổng phần thưởng $R(\tau_1) = 5$; τ_2 với xác suất 0.4 và $R(\tau_2) = -2$. Khi đó $J(\theta) = 0.6 \cdot 5 + 0.4 \cdot (-2) = 2.2$. Nếu thay đổi θ để xác suất của τ_1 tăng lên 0.9 thì $J = 0.9 \cdot 5 + 0.1 \cdot (-2) = 4.3$. Học tăng cường chính là tìm cách "dịch chuyển xác suất" về phía các quỹ đạo được thưởng cao.

Đến đây, ta có một cách tiếp cận học tăng cường đơn giản: 1) lấy mẫu một số chuỗi trạng thái–hành động; rồi 2) đánh giá từng chuỗi bằng hàm hiệu năng; rồi 3) cập nhật mô hình để cực đại hóa hàm hiệu năng này. Nếu ta lấy Phương trình (10.18) và dùng hạ gradient để tối ưu chính sách, điều này tạo thành một phương pháp gradient chính sách (*policy gradient*) [Williams, 1992].

Lưu ý rằng trong nhiều bài toán NLP, chẳng hạn dịch máy, phần thưởng thường thưa (*sparse*). Ví dụ, phần thưởng chỉ được nhận ở cuối một câu hoàn chỉnh. Điều này có nghĩa là $r_t = 0$ với mọi $t < T$, và r_t chỉ khác không khi $t = T$. Lý tưởng nhất, người ta có thể muốn phản hồi đến ngay lập tức và thường xuyên (dày đặc), nhờ đó việc huấn luyện chính sách có thể dễ dàng và hiệu quả hơn. Dù đã có một số phương pháp được đề xuất để xử lý phần thưởng thưa, chẳng hạn định hình phần thưởng (*reward shaping*), trong phần thảo luận tiếp theo chúng tôi vẫn giả định thiết lập phần thưởng thưa, trong đó phần thưởng chỉ có được khi hoàn tất dự đoán.

Mô hình mô tả trong các Phương trình (10.17–10.19) thiết lập một dạng học tăng cường cơ bản, và nhiều biến thể cũng như cải tiến của mô

hình này đã được phát triển. Trước khi trình bày những mô hình tinh vi hơn đó, hãy dành chút thời gian diễn giải hàm mục tiêu $J(\theta)$ từ góc nhìn gradient chính sách. Trong hạ gradient, ta cần tính gradient của $J(\theta)$ theo θ:

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta} &= \frac{\partial \sum_{\tau\in\mathcal{T}} \Pr_\theta(\tau) R(\tau)}{\partial\theta} \\ &= \sum_{\tau\in\mathcal{T}} \frac{\partial \Pr_\theta(\tau)}{\partial\theta} R(\tau) \\ &= \sum_{\tau\in\mathcal{T}} \Pr_\theta(\tau) \frac{\partial \Pr_\theta(\tau)/\partial\theta}{\Pr_\theta(\tau)} R(\tau) \\ &= \sum_{\tau\in\mathcal{T}} \Pr_\theta(\tau) \frac{\partial \log \Pr_\theta(\tau)}{\partial\theta} R(\tau). \end{aligned} \tag{10.20}$$

Chú giải của người dịch: Mẹo đạo hàm logarit

Dòng cuối của Phương trình (10.20) dùng một quy tắc quen thuộc ở THPT: $(\ln u)' = \dfrac{u'}{u}$, nên $u' = u \cdot (\ln u)'$. Ở đây $u = \Pr_\theta(\tau)$ và đạo hàm lấy theo tham số θ (ký hiệu ∂ chỉ "đạo hàm riêng": đạo hàm theo một biến trong khi coi các biến khác là hằng số). Lợi ích của mẹo này: tổng $\sum_\tau \Pr_\theta(\tau)[\ldots]$ lại có dạng một *kỳ vọng*, nên ta có thể ước lượng nó bằng cách lấy mẫu vài quỹ đạo rồi tính trung bình, thay vì phải duyệt qua mọi quỹ đạo (là con số khổng lồ).

Trong thực tế, kỳ vọng thường được xấp xỉ bằng các quỹ đạo được lấy mẫu đều từ một lô (tức $\Pr_\theta(\tau) = 1/|\mathcal{T}|$). Khi đó ta có thể đơn giản hóa Phương trình (10.20) và chỉ cần xét các số hạng $\frac{\partial \log \Pr_\theta(\tau)}{\partial\theta}$ và $R(\tau)$:

$$\frac{\partial J(\theta)}{\partial\theta} = \frac{1}{m} \sum_{\tau\in\mathcal{T}} \frac{\partial \log \Pr_\theta(\tau)}{\partial\theta} R(\tau). \tag{10.21}$$

Một ưu điểm của kết quả này là $R(\tau)$ không cần khả vi, nghĩa là ta có thể dùng bất kỳ loại hàm phần thưởng nào trong học tăng cường.

Chú giải của người dịch: Ý nghĩa của Phương trình (10.21)

Ở đây $\mathcal{T}$ là tập các quỹ đạo lấy mẫu trong lô và m là số quỹ đạo đó (tức $m = |\mathcal{T}|$). Công thức nói: với mỗi quỹ đạo, hãy đẩy tham số theo hướng

làm tăng log-xác suất của quỹ đạo đó, với "độ mạnh" tỷ lệ với phần thưởng $R(\tau)$. Quỹ đạo được thưởng lớn được "khuyến khích" mạnh; quỹ đạo có phần thưởng âm bị "kìm hãm" (đẩy theo hướng ngược lại). Vì $R(\tau)$ chỉ đóng vai trò một con số nhân vào, nó có thể đến từ bất cứ đâu — một người chấm điểm, một chương trình kiểm tra đáp án, hay một mô hình phần thưởng — mà không cần lấy đạo hàm của nó.

Bằng cách xem việc sinh ra chuỗi τ như một quá trình quyết định Markov (*Markov decision process*), ta có thể suy dẫn tiếp $\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta}$ và thu được:

$$\begin{aligned}\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta} &= \frac{\partial}{\partial \theta} \log \prod_{t=1}^{T} \pi_\theta(a_t|s_t) \Pr(s_{t+1}|s_t, a_t) \\ &= \frac{\partial}{\partial \theta} \sum_{t=1}^{T} \underbrace{\log \pi_\theta(a_t|s_t)}_{\text{chính sách}} + \frac{\partial}{\partial \theta} \sum_{t=1}^{T} \underbrace{\log \Pr(s_{t+1}|s_t, a_t)}_{\text{động lực}},\end{aligned} \tag{10.22}$$

trong đó gradient được phân rã thành hai phần: gradient chính sách và gradient động lực (*dynamics*). Thành phần chính sách, $\log \pi_\theta(a_t|s_t)$, xác định log-xác suất của việc thực hiện hành động a_t khi ở trạng thái s_t, và được tham số hóa bởi θ. Thành phần động lực, $\log \Pr(s_{t+1}|s_t, a_t)$, biểu diễn log-xác suất chuyển sang trạng thái s_{t+1} từ trạng thái s_t sau khi thực hiện hành động a_t. Trong các thiết lập học tăng cường điển hình, động lực không bị ảnh hưởng trực tiếp bởi tham số chính sách θ, do đó đạo hàm của chúng thường bằng không. Vì vậy, trong trường hợp này, Phương trình (10.22) có thể được đơn giản thành:

$$\frac{\partial \log \Pr_\theta(\tau)}{\partial \theta} = \frac{\partial}{\partial \theta} \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t). \tag{10.23}$$

Nói cách khác, ta chỉ tập trung tối ưu chính sách mà không cần bận tâm đến động lực bên dưới.

Chú giải của người dịch: Quá trình quyết định Markov và "động lực"

"Markov" nghĩa là bước tiếp theo chỉ phụ thuộc vào hiện tại (s_t và a_t), không cần nhớ cả quá khứ. Xác suất của cả quỹ đạo là tích xác suất từng bước (như quy tắc nhân trong xác suất), và log của tích bằng tổng các log. Mỗi bước gồm hai việc: *tác tử chọn* hành động (phần chính sách π_θ, phụ thuộc θ) và *thế giới phản ứng* (phần động lực, không phụ thuộc θ). Như chơi cờ: bạn chỉ điều khiển nước đi của mình, còn đối thủ đi thế nào không thay đổi khi bạn "chỉnh" cách nghĩ của mình. Với LLM thì còn đơn giản hơn: trạng thái mới chỉ là trạng thái cũ nối thêm token vừa chọn, nên động lực là tất định.

Thay Phương trình (10.23) vào Phương trình (10.21), và khai triển $R(\tau)$, ta thu được

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{\partial}{\partial \theta} \Big(\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) \sum_{t=1}^{T} r_t \Big). \tag{10.24}$$

Dù cách tiếp cận gradient chính sách này đơn giản, nó gặp vấn đề là phương sai (*variance*) của các gradient ước lượng có thể rất cao, khiến quá trình học bị nhiễu và kém hiệu quả. Một lý do của vấn đề phương sai cao này là phần thưởng có thể biến động rất lớn giữa các bước hoặc các tình huống khác nhau. Hãy hình dung trong một chuỗi các quyết định hành động, mô hình phần thưởng có xu hướng gán phần thưởng nhỏ cho các hành động tốt (ví dụ $R_t = 2$) và mức phạt lớn cho các hành động kém (ví dụ $R_t = -50$). Những thang phần thưởng chênh lệch như vậy cho hành động tốt và kém có thể dẫn đến tổng phần thưởng rất thấp cho cả chuỗi, ngay cả khi chuỗi đó có chứa các hành động tốt.

Một phương pháp đơn giản để giảm phương sai của gradient là đặt một đường cơ sở (*baseline*) b và trừ nó khỏi $\sum_{t=1}^{T} r_t$, được $\sum_{t=1}^{T} r_t - b$.[5] Ở đây, đường cơ sở có thể được hiểu như một điểm tham chiếu. Bằng cách đưa phần thưởng về quanh đường cơ sở này, ta loại bỏ các thiên lệch có hệ thống trong tín hiệu phần thưởng, khiến các bước cập nhật ổn định hơn

[5] Thực ra, việc dùng đường cơ sở b không làm thay đổi phương sai của tổng phần thưởng $\sum_{t=1}^{T} r_t$. Tuy nhiên, cần lưu ý rằng mặc dù việc đưa vào đường cơ sở không làm thay đổi phương sai tổng thể của phần thưởng, nó giúp giảm phương sai của các gradient ước lượng. Nói chung, phép toán $\sum_{t=1}^{T} r_t - b$ đưa phần thưởng về quanh giá trị không (ví dụ b được định nghĩa là giá trị kỳ vọng của $\sum_{t=1}^{T} r_t$), điều này có thể làm giảm phương sai của tích $\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t)(\sum_{t=1}^{T} r_t - b)$.

và ít nhạy cảm hơn với những dao động cực đoan của từng phần thưởng riêng lẻ.

Chú giải của người dịch: Vì sao trừ đi một "đường cơ sở"?

Giả sử mọi bài làm của bạn đều được chấm từ 80 đến 90 điểm. Nếu ta dùng điểm thô làm tín hiệu, mọi bài đều "được khen" (vì điểm dương), chỉ khác nhau mức độ — rất khó phân biệt bài nào thực sự tốt. Nếu trừ đi điểm trung bình $b = 85$, bài 90 điểm thành $+5$ (khuyến khích), bài 80 điểm thành -5 (kìm hãm): tín hiệu rõ ràng hơn nhiều. *Phương sai* (đã gặp trong thống kê mô tả) đo mức độ phân tán; khi tín hiệu dao động ít quanh 0, các bước cập nhật tham số đỡ "giật cục".

Mô hình gradient chính sách có đường cơ sở này có thể được cho bởi

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta} &= \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big(\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big)\Big(\sum_{t=1}^{T}r_t - b\Big)\\ &= \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big[\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big(\sum_{k=1}^{T}r_k - b\Big)\Big]\\ &= \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big[\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big(\sum_{k=1}^{t-1}r_k + \sum_{k=t}^{T}r_k - b\Big)\Big].\end{aligned} \tag{10.25}$$

Ở đây ta viết $\sum_{k=1}^{T} r_k$ thành tổng của hai số hạng $\sum_{k=1}^{t-1} r_k$ và $\sum_{k=t}^{T} r_k$ để phân biệt giữa các phần thưởng tích lũy trước và sau hành động tại bước thời gian t. Lưu ý rằng các phần thưởng nhận được trước bước thời gian t không phụ thuộc vào hành động thực hiện tại bước t. Do đó, số hạng $\sum_{k=1}^{t-1} r_k$ không đóng góp vào gradient và có thể bỏ đi. Điều này dẫn tới một phiên bản đơn giản hơn của Phương trình (10.25)

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}}\frac{\partial}{\partial\theta}\Big[\sum_{t=1}^{T}\log\pi_\theta(a_t|s_t)\Big(\sum_{k=t}^{T}r_k - b\Big)\Big]. \tag{10.26}$$

Cũng lưu ý rằng việc bỏ $\sum_{k=1}^{t-1} r_k$ còn có thể giảm thêm phương sai của gradient.

Chú giải của người dịch: "Quá khứ không thể thay đổi"

Hành động ở bước t chỉ có thể ảnh hưởng đến những gì xảy ra *từ bước t trở đi*. Ví dụ, khi đánh giá một quyết định đổi chiến thuật ở phút 60 của trận bóng, ta chỉ nên tính các bàn thắng/thua từ phút 60 về sau; các bàn đã ghi trước đó không do quyết định này gây ra. Tổng $\sum_{k=t}^{T} r_k$ vì thế thường được gọi là "phần thưởng còn lại tính từ bước t" (*reward-to-go*).

Có nhiều cách để định nghĩa đường cơ sở b. Ở đây ta xét hàm giá trị của trạng thái s_t, tức là giá trị ước lượng của việc ở trạng thái s_t: $V(s_t) = \mathbb{E}(r_t + r_{t+1} + \cdots + r_T)$. Do đó ta có

$$\begin{aligned} A(s_t, a_t) &= \sum_{k=t}^{T} r_k - b \\ &= \sum_{k=t}^{T} r_k - V(s_t), \end{aligned} \tag{10.27}$$

trong đó $\sum_{k=t}^{T} r_k$ biểu diễn lợi tức thực tế nhận được, còn $V(s_t)$ biểu diễn lợi tức kỳ vọng. $A(s_t, a_t)$ (viết tắt là A_t) được gọi là **lợi thế** (*advantage*) tại bước thời gian t; nó định lượng lợi ích tương đối của hành động a_t so với giá trị kỳ vọng của việc đi theo chính sách từ trạng thái s_t trở đi.

Bằng cách dùng hàm lợi thế $A(s_t, a_t)$, gradient của $J(\theta)$ có thể được viết dưới dạng

$$\frac{\partial J(\theta)}{\partial \theta} \;=\; \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{\partial}{\partial \theta} \Big(\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A(s_t, a_t) \Big). \tag{10.28}$$

Chú giải của người dịch: Lợi thế: "tốt hơn mong đợi bao nhiêu?"

Giả sử một học sinh thường đạt trung bình 7 điểm môn Toán (đây là $V(s_t)$ — mức kỳ vọng). Lần này, sau khi áp dụng một cách ôn mới (hành động a_t), bạn ấy được 9 điểm (lợi tức thực tế). Lợi thế là $A = 9 - 7 = +2$: cách ôn mới tốt hơn mong đợi, nên "tăng xác suất" dùng lại nó. Nếu chỉ được 6 điểm thì $A = -1$: nên giảm. Lưu ý: điều quan trọng không phải là điểm cao hay thấp tuyệt đối, mà là *cao hơn hay thấp hơn mức thường lệ*.

Mục tiêu này tạo nền tảng cho các phương pháp **tác nhân–phê bình có lợi thế** (*advantage actor-critic*) (**A2C**) trong học tăng cường [Mnih và

cộng sự, 2016]. Trong phương pháp này, tác nhân (*actor*) có nhiệm vụ học một chính sách. Nó cập nhật các tham số chính sách bằng Phương trình (10.28) để tập trung nhiều hơn vào các hành động có khả năng cải thiện hiệu năng. Mặt khác, bộ phê bình (*critic*) cập nhật ước lượng của nó về hàm giá trị, thứ được dùng để tính hàm lợi thế $A(s_t, a_t)$, nhờ đó đóng vai trò người đánh giá chính sách mà tác nhân đang học.

Trong phương pháp A2C, $A(s_t, a_t)$ thường được biểu diễn là hiệu của hàm giá trị hành động $Q(s_t, a_t)$ và hàm giá trị trạng thái $V(s_t)$:

$$A(s_t, a_t) \;=\; Q(s_t, a_t) - V(s_t). \tag{10.29}$$

Thoạt nhìn, mô hình này có vẻ khó xây dựng vì nó đòi hỏi hai mô hình con riêng biệt để lần lượt tính $Q(s_t, a_t)$ và $V(s_t)$. May mắn thay, xét rằng $Q(s_t, a_t)$ có thể được định nghĩa là lợi tức $r_t + V(s_{t+1})$, ta có thể viết lại Phương trình (10.29) thành

$$A(s_t, a_t) \;=\; r_t + V(s_{t+1}) - V(s_t), \tag{10.30}$$

hoặc đưa thêm hệ số chiết khấu γ vào để có dạng tổng quát hơn

$$A(s_t, a_t) \;=\; r_t + \gamma V(s_{t+1}) - V(s_t). \tag{10.31}$$

$A(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t)$ còn được gọi là sai số **chênh lệch thời gian** (*temporal difference*) (**TD**). Điều ta cần làm là huấn luyện một mạng phê bình cho hàm giá trị $V(s_t)$, rồi dùng nó để tính hàm lợi thế[6].

Chú giải của người dịch: Tác nhân–phê bình và sai số chênh lệch thời gian

Hãy nghĩ tới một cầu thủ (*tác nhân*) và huấn luyện viên (*bộ phê bình*). Cầu thủ quyết định đá thế nào; huấn luyện viên không đá, nhưng đánh giá "tình thế này đáng giá bao nhiêu" (V). Sau mỗi pha bóng, ta so sánh: *dự đoán trước đó* $V(s_t)$ với *dự đoán mới sau khi đã thấy một bước thực tế* $r_t + \gamma V(s_{t+1})$.

[6]Hàm mất mát huấn luyện cho mạng giá trị (hay mạng phê bình) trong A2C thường được phát biểu là sai số bình phương trung bình giữa lợi tức tính được $r_t + \gamma V(s_{t+1})$ và giá trị trạng thái dự đoán $V(s_t)$. Giả sử mạng giá trị được tham số hóa bởi ω. Hàm mất mát được cho bởi

$$\mathcal{L}_v(\omega) \;=\; \frac{1}{M} \sum \big(r_t + \gamma V_\omega(s_{t+1}) - V_\omega(s_t)\big)^2, \tag{10.32}$$

trong đó M là số mẫu huấn luyện; ví dụ, với một chuỗi gồm T token, ta có thể đặt $M = T$.

Hiệu số giữa hai giá trị đó chính là sai số TD. Ví dụ: huấn luyện viên dự đoán tình thế hiện tại đáng $V(s_t) = 3$; cầu thủ chuyền bóng, nhận ngay $r_t = 0$, nhưng tình thế mới đáng $V(s_{t+1}) = 5$; với $\gamma = 1$, $A = 0+5-3 = +2 > 0$: đường chuyền tốt hơn mong đợi. Hay ở chỗ: ta không cần đợi đến cuối trận mới biết hành động có tốt không, chỉ cần nhìn một bước rồi "tin" vào ước lượng của bộ phê bình. Đồng thời, bộ phê bình tự cải thiện bằng cách làm cho sai số này nhỏ dần (Phương trình (10.32)).

Cho đến đây, chúng ta đã dành khá nhiều dung lượng để bàn về những điều cơ bản của học tăng cường, đặc biệt là cách suy dẫn mục tiêu tối ưu cho phương pháp A2C. Tuy nhiên, học tăng cường là một lĩnh vực rộng lớn, và nhiều chi tiết kỹ thuật không thể đề cập ở đây. Bạn đọc quan tâm có thể tham khảo các sách về học tăng cường để biết thêm chi tiết [Sutton và Barto, 2018; Szepesvári, 2010]. Dù vậy, giờ đây ta đã có đủ kiến thức cần thiết để thảo luận tiếp về RLHF. Trong các tiểu mục tiếp theo, chúng ta sẽ quay lại thảo luận về căn chỉnh LLM, cho thấy cách dùng phương pháp A2C để căn chỉnh theo sở thích của con người.

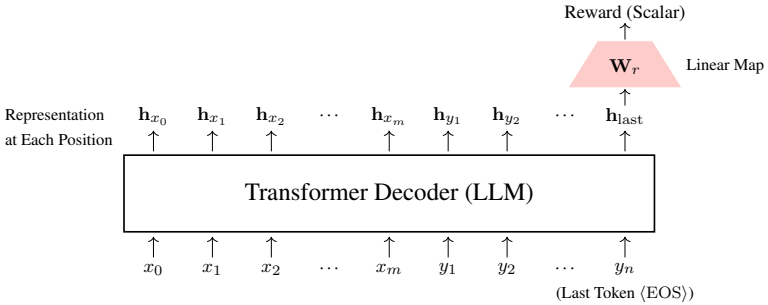
10.3.2 Huấn luyện mô hình phần thưởng

Chúng ta đã thấy rằng mô hình phần thưởng đóng vai trò rất quan trọng trong khung học tăng cường tổng quát và là cơ sở để tính các hàm giá trị. Giờ ta xét bài toán huấn luyện các mô hình phần thưởng này.

Trong RLHF, mô hình phần thưởng là một mạng nơ-ron ánh xạ một cặp chuỗi token đầu vào và đầu ra thành một điểm phần thưởng vô hướng. Cho một đầu vào $\mathbf{x}$ và một đầu ra $\mathbf{y}$, phần thưởng có thể được biểu diễn là

$$r \;=\; \text{Reward}(\mathbf{x}, \mathbf{y}), \tag{10.33}$$

trong đó $\text{Reward}(\cdot)$ là mô hình phần thưởng. r có thể được hiểu là thước đo mức độ đầu ra $\mathbf{y}$ phù hợp với hành vi mong muốn khi cho trước đầu vào $\mathbf{x}$. Như đã bàn ở tiểu mục trước, cả $\mathbf{x}$ và $\mathbf{y}$ đều được giả định là các chuỗi token hoàn chỉnh. Do đó, mô hình phần thưởng đánh giá chất lượng của một đầu ra đã được sinh trọn vẹn, có điều kiện theo đầu vào, chứ không gán phần thưởng cho các bước sinh trung gian. Ví dụ, trong quá trình sinh chuỗi, phần thưởng thường chỉ được cung cấp tại vị trí cuối cùng của chuỗi đầu ra $\mathbf{y} = y_1 \dots y_n$. Để ký hiệu đỡ rườm rà, từ đây trở đi ta sẽ dùng $r(\mathbf{x}, \mathbf{y})$ để ký hiệu mô hình phần thưởng $\text{Reward}(\mathbf{x}, \mathbf{y})$.



Hình 10.8: Kiến trúc của một mô hình phần thưởng dựa trên Transformer. Thành phần chính của mô hình này vẫn là một LLM. Ta dùng bộ giải mã Transformer làm mô hình biểu diễn chuỗi. Ta lấy biểu diễn tại vị trí cuối cùng của bộ giải mã làm biểu diễn của cả chuỗi $[\mathbf{x}, \mathbf{y}]$. Sau đó ta ánh xạ biểu diễn này thành một số vô hướng thông qua một phép biến đổi tuyến tính; số này đóng vai trò điểm phần thưởng của $[\mathbf{x}, \mathbf{y}]$.

Có nhiều cách để hiện thực mô hình phần thưởng. Một cách đơn giản là xây dựng mô hình phần thưởng dựa trên một LLM đã tiền huấn luyện. Cụ thể hơn, ta có thể nối $\mathbf{x}$ và $\mathbf{y}$ để tạo thành một chuỗi token duy nhất $\text{seq}_{\mathbf{x},\mathbf{y}} = [\mathbf{x}, \mathbf{y}]$. Ta chạy một LLM đã tiền huấn luyện trên chuỗi này như thường lệ, và tại mỗi vị trí, ta thu được một biểu diễn từ tầng Transformer cuối cùng. Sau đó, ta lấy biểu diễn tại vị trí cuối cùng (ký hiệu là $\mathbf{h}_{\text{last}}$) và ánh xạ nó thành một số vô hướng thông qua một phép biến đổi tuyến tính:

$$r(\mathbf{x}, \mathbf{y}) \;=\; \mathbf{h}_{\text{last}} \mathbf{W}_r, \tag{10.34}$$

trong đó $\mathbf{h}_{\text{last}}$ là một vectơ d chiều, và $\mathbf{W}_r$ là một ma trận ánh xạ tuyến tính kích thước $d \times 1$. Kiến trúc này của mô hình phần thưởng được minh họa trong Hình 10.8.

Chú giải của người dịch: Đọc Hình 10.8 và Phương trình (10.34)

Các nhãn trong hình: *Representation at Each Position* — biểu diễn tại mỗi vị trí; *Transformer Decoder (LLM)* — bộ giải mã Transformer (chính là LLM); *Last Token ⟨EOS⟩* — token cuối cùng (ký hiệu kết thúc chuỗi); *Linear Map* — ánh xạ tuyến tính; *Reward (Scalar)* — phần thưởng (một số vô hướng). Nhân vectơ hàng $\mathbf{h}_{\text{last}}$ (có d số) với ma trận cột $\mathbf{W}_r$ (d số) chính là *tích vô hướng*: $r = h_1 w_1 + h_2 w_2 + \cdots + h_d w_d$. Ví dụ $d = 3$: $\mathbf{h}_{\text{last}} = [1,\ 2,\ -1]$,

$\mathbf{W}_r = [0.5,\ 0.1,\ 0.3]^\top$ thì $r = 0.5 + 0.2 - 0.3 = 0.4$. Vì LLM đọc từ trái sang phải, token cuối "đã nhìn thấy" toàn bộ câu hỏi và câu trả lời, nên biểu diễn của nó là bản tóm tắt tốt của cả chuỗi.

Để huấn luyện mô hình phần thưởng, bước đầu tiên là thu thập phản hồi của con người trên một tập các đầu ra đã sinh. Cho một đầu vào $\mathbf{x}$, ta dùng LLM để tạo ra nhiều đầu ra ứng viên $\{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$. Phản hồi của con người có thể được thu thập theo một số cách:

- **So sánh theo cặp** (**xếp hạng theo cặp**, *Pairwise Comparison / Pairwise Ranking*). Cho hai đầu ra khác nhau, các chuyên gia là con người chọn ra cái nào tốt hơn.

- **Chấm điểm** (*Rating*). Các chuyên gia là con người cho mỗi đầu ra một điểm số hay mức đánh giá. Điểm này thường là một giá trị số liên tục hoặc rời rạc, chẳng hạn điểm trên một thang (ví dụ 1–5 sao, hoặc 1–10 điểm). Trong một số trường hợp, đánh giá có thể là nhị phân, thể hiện sở thích kiểu "có/không" hoặc "tích cực/tiêu cực".

- **Xếp hạng theo danh sách** (*Listwise Ranking*). Các chuyên gia là con người được yêu cầu xếp hạng hoặc sắp thứ tự tập các đầu ra khả dĩ đã cho.

Ở đây ta xét phản hồi dạng so sánh theo cặp vì đó là một trong những dạng phản hồi của con người đơn giản và phổ biến nhất được dùng trong RLHF. Trong thiết lập này, với mỗi lần so sánh, hai đầu ra $(\mathbf{y}_a, \mathbf{y}_b)$ được lấy ngẫu nhiên từ tập ứng viên $\{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$. Sau đó các cặp này được đưa cho các chuyên gia là con người, và họ được yêu cầu quyết định mình thích đầu ra nào hơn dựa trên các tiêu chí như độ rõ ràng, mức độ liên quan và độ chính xác. Phản hồi của con người có thể được mã hóa thành một nhãn nhị phân. Ví dụ, $\mathbf{y}_a \succ \mathbf{y}_b$ biểu thị sự ưa thích dành cho $\mathbf{y}_a$, và $\mathbf{y}_b \succ \mathbf{y}_a$ biểu thị sự ưa thích dành cho $\mathbf{y}_b$.

Một mô hình đơn giản và được dùng rộng rãi để mô tả các so sánh theo cặp như vậy là **mô hình Bradley–Terry** (*Bradley-Terry model*) [Bradley và Terry, 1952]. Đây là một mô hình xác suất ước lượng xác suất một đối tượng được ưa thích hơn một đối tượng khác. Điều chỉnh mô hình này theo ký hiệu dùng ở đây, ta có thể viết xác suất $\mathbf{y}_a$ được ưa thích hơn $\mathbf{y}_b$ dưới

dạng

$$\begin{aligned}\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) &= \frac{e^{r(\mathbf{x},\mathbf{y}_a)}}{e^{r(\mathbf{x},\mathbf{y}_a)} + e^{r(\mathbf{x},\mathbf{y}_b)}} \\ &= \frac{e^{r(\mathbf{x},\mathbf{y}_a)-r(\mathbf{x},\mathbf{y}_b)}}{e^{r(\mathbf{x},\mathbf{y}_a)-r(\mathbf{x},\mathbf{y}_b)} + 1} \\ &= \text{Sigmoid}\big(r(\mathbf{x},\mathbf{y}_a) - r(\mathbf{x},\mathbf{y}_b)\big). \end{aligned} \tag{10.35}$$

Chú giải của người dịch: Mô hình Bradley–Terry qua ví dụ thi đấu

Mô hình này ban đầu dùng để dự đoán kết quả đối đầu: mỗi đội có một "chỉ số sức mạnh", và xác suất đội A thắng đội B chỉ phụ thuộc vào *hiệu* hai chỉ số. Ở đây "chỉ số sức mạnh" của một câu trả lời là điểm phần thưởng r. Hàm $\text{Sigmoid}(z) = \dfrac{1}{1+e^{-z}}$ biến một số thực bất kỳ thành xác suất trong $(0,1)$. Ví dụ: $r(\mathbf{x},\mathbf{y}_a) = 2$, $r(\mathbf{x},\mathbf{y}_b) = 1$ thì $\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) = \text{Sigmoid}(1) = \dfrac{1}{1+e^{-1}} \approx 0.73$. Nếu hai điểm bằng nhau thì xác suất là 0.5 ("ngang tài ngang sức"). Để ý rằng cộng cùng một hằng số vào mọi điểm thì xác suất không đổi — chỉ hiệu số là quan trọng.

Khi huấn luyện mô hình phần thưởng, mục tiêu của ta là cực đại hóa xác suất ưa thích này. Một hàm mất mát dựa trên mô hình Bradley–Terry được cho bởi

$$\mathcal{L}_r(\phi) \;=\; -\mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\big[\log \Pr_\phi(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\big], \tag{10.36}$$

trong đó $(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)$ được lấy từ một tập dữ liệu do con người gán nhãn $\mathcal{D}_r$, gồm các cặp đầu ra có thứ tự ưa thích cùng với đầu vào tương ứng của chúng. ϕ biểu diễn các tham số của mô hình phần thưởng, bao gồm cả tham số của bộ giải mã Transformer lẫn ma trận ánh xạ tuyến tính $\mathbf{W}_r$. Trong thực tế, giả sử $(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)$ được lấy mẫu đều từ $\mathcal{D}_r$, kỳ vọng có thể được xấp xỉ bằng trung bình thực nghiệm

$$\mathcal{L}_r(\phi) \;=\; -\frac{1}{|\mathcal{D}_r|}\sum_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\in\mathcal{D}_r} \log \Pr_\phi(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}). \tag{10.37}$$

Mục tiêu của việc huấn luyện mô hình phần thưởng là tìm các tham số tối ưu $\hat{\phi}$ cực tiểu hóa hàm mất mát này, được cho bởi

$$\hat{\phi} \;=\; \arg\min_{\phi} \mathcal{L}_r(\phi). \tag{10.38}$$

Vì bản thân mô hình phần thưởng cũng là một LLM, ta có thể đơn giản dùng quy trình huấn luyện Transformer để tối ưu mô hình phần thưởng. Điểm khác biệt so với việc huấn luyện một LLM thông thường là ta chỉ cần thay mất mát entropy chéo bằng mất mát so sánh theo cặp như mô tả trong Phương trình (10.37). Sau khi huấn luyện mô hình phần thưởng, ta có thể áp dụng mô hình phần thưởng đã huấn luyện $r_{\hat{\phi}}(\cdot)$ để giám sát LLM đích nhằm căn chỉnh nó.

Chú giải của người dịch: Hàm mất mát của mô hình phần thưởng bằng số

Với mỗi cặp mà con người nói $\mathbf{y}_a$ tốt hơn $\mathbf{y}_b$, mất mát là $-\log \text{Sigmoid}(r_a - r_b)$, với r_a, r_b là điểm mô hình chấm. Nếu mô hình chấm đúng chiều và cách xa, ví dụ $r_a - r_b = 3$, thì $\text{Sigmoid}(3) \approx 0.95$ và mất mát $\approx -\ln 0.95 \approx 0.05$ (rất nhỏ). Nếu chấm ngược, ví dụ $r_a - r_b = -2$, thì $\text{Sigmoid}(-2) \approx 0.12$ và mất mát $\approx -\ln 0.12 \approx 2.1$ (lớn). Cực tiểu hóa trung bình các mất mát này buộc mô hình chấm câu được ưa thích cao hơn câu bị chê. Lưu ý mô hình không cần biết "điểm tuyệt đối" đúng là bao nhiêu — nó chỉ học *thứ tự*.

Đáng lưu ý là mặc dù ta huấn luyện mô hình phần thưởng để thực hiện xếp hạng theo cặp, trong quá trình căn chỉnh ta lại dùng nó để chấm điểm độc lập từng cặp đầu vào–đầu ra. Mục tiêu xếp hạng theo cặp đảm bảo mô hình phần thưởng nhạy với những khác biệt tinh tế giữa các đầu ra, nhưng ta dựa vào các điểm số liên tục do mô hình phần thưởng tạo ra để định hướng việc tối ưu LLM. Một ưu điểm của cách tiếp cận này là ta có thể chọn hoặc kết hợp nhiều hàm mất mát xếp hạng (*ranking loss*) khác nhau, mà vẫn áp dụng các mô hình phần thưởng thu được theo cùng cách như ta đã làm trong tiểu mục này. Sự linh hoạt này khiến khung mô hình hóa phần thưởng áp dụng được rộng rãi cho nhiều dạng giám sát bằng sở thích của con người.

10.3.3 Huấn luyện LLM

Khi đã có mô hình phần thưởng, ta tiếp tục huấn luyện chính sách (tức LLM) bằng phương pháp A2C. Nhớ lại từ Mục 10.3.1 rằng một chuỗi trạng thái–hành động hay quỹ đạo τ có thể được đánh giá bằng hàm tiện

ích (*utility function*)

$$U(\tau;\theta) \;=\; \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A(s_t,a_t), \tag{10.39}$$

trong đó $A(s_t,a_t)$ là lợi thế của việc thực hiện hành động a_t khi ở trạng thái s_t. Một ước lượng của $A(s_t,a_t)$ được định nghĩa là sai số TD $r_t + \gamma V(s_{t+1}) - V(s_t)$, trong đó hàm giá trị $V(s_t)$ được huấn luyện với mô hình phần thưởng.

Với hàm tiện ích này, hàm mất mát dựa trên A2C có thể được viết dưới dạng

$$\begin{aligned}\mathcal{L}(\theta) &= -\mathbb{E}_{\tau\sim\mathcal{T}}\big[U(\tau;\theta)\big] \\ &= -\mathbb{E}_{\tau\sim\mathcal{T}}\Big[\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A(s_t,a_t)\Big],\end{aligned} \tag{10.40}$$

trong đó $\mathcal{T}$ là một không gian các chuỗi trạng thái–hành động. Như thường lệ, mục tiêu của việc huấn luyện chính sách là cực tiểu hóa hàm mất mát này

$$\tilde{\theta} \;=\; \arg\min_\theta \mathcal{L}(\theta). \tag{10.41}$$

Nếu ánh xạ bài toán trở lại bài toán mô hình hóa ngôn ngữ và dùng ký hiệu của LLM, hàm mất mát có thể được viết là:

$$\mathcal{L}(\theta) \;=\; -\mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}}\big[U(\mathbf{x},\mathbf{y};\theta)\big], \tag{10.42}$$

trong đó

$$U(\mathbf{x},\mathbf{y};\theta) \;=\; \sum_{t=1}^{T} \log \pi_\theta(y_t|\mathbf{x},\mathbf{y}_{<t}) A(\mathbf{x},\mathbf{y}_{<t},y_t). \tag{10.43}$$

Ở đây $\mathcal{D}$ là một tập các cặp đầu vào–đầu ra, và $\pi_\theta(y_t|\mathbf{x},\mathbf{y}_{<t}) = \Pr_\theta(y_t|\mathbf{x},\mathbf{y}_{<t})$ là LLM được tham số hóa bởi θ.

Nói chung, trong RLHF ta không có một tập dữ liệu đầu vào–đầu ra $\mathcal{D}$ do con người gán nhãn, mà chỉ có một tập dữ liệu chứa các đầu vào. Trong trường hợp này, các đầu ra thường là các dự đoán do LLM tạo ra. Khi đó hàm mất mát được định nghĩa là

$$\mathcal{L}(\theta) \;=\; -\mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\big[U(\mathbf{x},\mathbf{y};\theta)\big], \tag{10.44}$$

trong đó $\mathcal{D}$ ký hiệu tập dữ liệu chỉ có đầu vào, và $\mathbf{y} \sim \pi_\theta(\cdot|\mathbf{x})$ chỉ ra rằng đầu ra $\mathbf{y}$ được lấy mẫu bởi chính sách $\pi_\theta(\cdot|\mathbf{x})$.

Chú giải của người dịch: So sánh với tinh chỉnh có giám sát

Trong SFT, mất mát là $-\sum_t \log \Pr_\theta(y_t|\mathbf{x}, \mathbf{y}_{<t})$ với $\mathbf{y}$ là đáp án chuẩn: mọi token của đáp án đều được "khuyến khích" như nhau. Phương trình (10.43)–(10.44) khác ở hai điểm: (1) $\mathbf{y}$ do chính LLM tự viết ra (lấy mẫu), không phải do người viết sẵn; (2) mỗi số hạng log-xác suất được nhân với lợi thế A, nên token nào giúp câu trả lời tốt hơn mong đợi ($A > 0$) thì được tăng xác suất, token nào làm câu trả lời tệ đi ($A < 0$) thì bị giảm xác suất. Có thể nói SFT là "chép bài mẫu", còn RLHF là "tự làm bài rồi được chấm điểm".

Công thức trên cung cấp một dạng cơ bản của phương pháp A2C cho LLM. Các phiên bản cải tiến của mô hình này được dùng phổ biến hơn trong RLHF. Trong phần thảo luận tiếp theo, chúng tôi vẫn dùng ký hiệu học tăng cường để đơn giản hóa cách trình bày và sẽ quay lại ký hiệu mô hình hóa ngôn ngữ sau.

Một cải tiến phổ biến của các phương pháp gradient chính sách là xây dựng một mục tiêu thay thế bằng **lấy mẫu theo trọng số quan trọng** (*importance sampling*). Mục tiêu thu được có thể viết là

$$U(\tau;\theta) \;=\; \sum_{t=1}^{T} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)} A(s_t, a_t). \tag{10.45}$$

Ở đây ta thay log-xác suất $\log \pi_\theta(a_t|s_t)$ bằng tỉ số $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}$. θ_{ref} ký hiệu các tham số của chính sách trước đó (chẳng hạn mô hình ban đầu mà từ đó ta bắt đầu huấn luyện). Do vậy, $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}$, còn được gọi là **hàm tỉ số** (*ratio function*), có thể được hiểu là tỉ số xác suất giữa chính sách hiện tại π_θ và chính sách tham chiếu $\pi_{\theta_{\text{ref}}}$ (gọi nó là chính sách tham chiếu (*reference policy*)). Bằng cách dùng hàm tỉ số, ta đánh trọng số lại các phần thưởng quan sát được dựa trên khả năng xảy ra của các hành động dưới chính sách hiện tại so với chính sách tham chiếu. Khi $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)} > 1$, hành động a_t được chính sách hiện tại ưa chuộng hơn so với chính sách tham chiếu. Ngược lại, khi $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)} < 1$, hành động a_t ít được chính sách hiện tại ưa chuộng hơn[7].

[7]Xét một trường hợp tổng quát hơn, trong đó ta muốn đánh giá chính sách bằng phần

Chú giải của người dịch: Lấy mẫu theo trọng số quan trọng

Ý tưởng: dữ liệu được thu bằng chính sách *cũ* $\pi_{\theta_{\text{ref}}}$, nhưng ta muốn đánh giá chính sách *mới* π_θ mà không phải sinh lại dữ liệu. Cách làm: đánh trọng số mỗi mẫu bằng tỉ số "chính sách mới thích nó gấp mấy lần chính sách cũ". Ví dụ: chính sách cũ chọn token "Xin" với xác suất 0.2, chính sách mới với xác suất 0.3; tỉ số $= 1.5$, nên phần thưởng/lợi thế của mẫu này được tính "nặng" gấp 1.5 lần. Tương tự như khi điều tra xã hội học: nếu nhóm nào bị lấy mẫu ít hơn tỉ lệ thực tế trong dân số, ta nhân trọng số cho nhóm đó lớn hơn để kết quả trung bình vẫn đúng.

Một vấn đề với mô hình trình bày trong Phương trình (10.47) (cũng như trong Phương trình (10.39)) là phương sai của các ước lượng gradient

thưởng kỳ vọng của nó (xem thêm Phương trình (10.18))

$$J(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_\theta}\Big[R(\tau)\Big]. \tag{10.46}$$

Ở đây $\tau \sim \pi_\theta$ nghĩa là chuỗi τ được sinh ra bởi chính sách π_θ. Ngoài ra, ta có thể viết $J(\theta)$ dưới một dạng khác

$$J(\theta) \;=\; \mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]. \tag{10.47}$$

Dễ thấy vế phải của hai phương trình này về bản chất là như nhau vì $\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big] = \sum_\tau \Pr_{\theta_{\text{ref}}}(\tau)\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau) = \sum_\tau \Pr_\theta(\tau)R(\tau) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[R(\tau)\Big]$.

Lưu ý rằng sự tương đương này chỉ đúng khi kỳ vọng được lấy trên toàn bộ không gian chuỗi. Tuy nhiên, trong thực tế, khi học chính sách ta thường chỉ lấy mẫu một số tương đối nhỏ các chuỗi bằng một chính sách. Do đó, bản thân phương pháp lấy mẫu là quan trọng. Phương trình (10.47) đưa ra một cách thú vị để tách quá trình lấy mẫu và quá trình tính phần thưởng: trước tiên ta dùng một chính sách cơ sở (với θ_{ref}) để lấy mẫu một số chuỗi, rồi dùng chính sách đích (với θ) để tính phần thưởng kỳ vọng. Bằng cách này, ta tách riêng chính sách dùng để thu thập dữ liệu và chính sách dùng để tính gradient. Cách tiếp cận này tránh được việc phải lấy mẫu trực tiếp từ chính sách đang được đánh giá, điều có thể có lợi trong những trường hợp việc sinh chuỗi từ chính sách đích tốn kém hoặc khó khăn. Trong học tăng cường, $\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]$ thường được gọi là **mục tiêu thay thế** (*surrogate objective*).

Phương trình (10.47) cũng có thể được diễn giải từ góc nhìn gradient chính sách. Với $\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]$, gradient tại $\theta = \theta_{\text{ref}}$ được cho bởi

$$\frac{\partial}{\partial\theta}\mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\Pr_\theta(\tau)}{\Pr_{\theta_{\text{ref}}}(\tau)}R(\tau)\Big]\Big|_{\theta=\theta_{\text{ref}}} \;=\; \mathbb{E}_{\tau\sim\pi_{\theta_{\text{ref}}}}\Big[\frac{\partial \Pr_\theta(\tau)|_{\theta=\theta_{\text{ref}}}}{\partial\theta}R(\tau)\Big]. \tag{10.48}$$

Vế phải là một dạng chuẩn dùng trong các phương pháp gradient chính sách, nghĩa là ta tính hướng cập nhật tham số tại điểm $\theta = \theta_{\text{ref}}$ trên bề mặt tối ưu.

thường cao, khiến quá trình học không ổn định. Để giảm nhẹ vấn đề này, các kỹ thuật như cắt ngưỡng (*clipping*) thường được dùng để chặn các trọng số quan trọng và ngăn những bước cập nhật lớn. Một phiên bản cắt ngưỡng của hàm tiện ích (còn gọi là hàm mục tiêu thay thế có cắt ngưỡng) được cho bởi

$$U_{\text{clip}}(\tau;\theta) \;=\; \sum_{t=1}^{T} \text{Clip}\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, A(s_t,a_t)\Big), \tag{10.49}$$

trong đó

$$\begin{aligned}
&\text{Clip}\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, A(s_t,a_t)\Big) \\
&\quad= \min\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}A(s_t,a_t), \\
&\qquad\quad \text{bound}\Big(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\Big)A(s_t,a_t)\Big).
\end{aligned} \tag{10.50}$$

Ở đây hàm $\text{bound}(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon)$ giới hạn hàm tỉ số trong khoảng $[1-\epsilon, 1+\epsilon]$.

Chú giải của người dịch: Cắt ngưỡng hoạt động thế nào?

Hàm $\text{bound}(z, 1-\epsilon, 1+\epsilon)$ "kẹp" z vào đoạn $[1-\epsilon, 1+\epsilon]$: nếu z nhỏ hơn $1-\epsilon$ thì lấy $1-\epsilon$, lớn hơn $1+\epsilon$ thì lấy $1+\epsilon$, còn lại giữ nguyên. Ví dụ $\epsilon = 0.2$, lợi thế $A = +2$. Nếu tỉ số đã là 1.5 (chính sách mới đã tăng xác suất hành động tốt này lên nhiều), thì $\min(1.5 \cdot 2,\ 1.2 \cdot 2) = 2.4$: phần vượt quá 1.2 không được tính thêm, nên mô hình không còn động lực đẩy xác suất lên nữa. Giống như quy định "mỗi lần điều chỉnh giá không quá 20%": cho phép tiến bộ, nhưng từng bước nhỏ, an toàn. Phép min đảm bảo ta luôn chọn ước lượng "bi quan" hơn trong hai giá trị.

Một cải tiến tiếp theo của mô hình trên là xét các **vùng tin cậy** (*trust region*) trong tối ưu hóa [Schulman và cộng sự, 2015]. Trong học tăng cường, một bước cập nhật lớn cho chính sách có thể dẫn đến mất ổn định, khi tác tử có thể bắt đầu hoạt động tệ hơn sau một lần cập nhật. Một ý tưởng hợp lý là tối ưu mô hình trong vùng tin cậy, tức là một vùng quanh ước lượng tham số hiện tại, nơi mô hình hoạt động tốt. Một cách để đưa vùng tin cậy vào là áp đặt một ràng buộc lên độ lớn của bước cập nhật

chính sách, bảo đảm chính sách hiện tại không lệch đáng kể khỏi chính sách tham chiếu. Trong RLHF, PPO thường được kết hợp với một khoản phạt bổ sung kiểu KL, ràng buộc chính sách luôn ở gần chính sách tham chiếu. Điều này có thể đạt được bằng cách thêm một khoản phạt liên quan đến sự khác biệt giữa chính sách hiện tại và chính sách tham chiếu. Một dạng đơn giản của khoản phạt như vậy được cho bởi hiệu giữa log-xác suất của chuỗi τ dưới chính sách hiện tại và dưới chính sách tham chiếu:

$$\text{Penalty} \;=\; \log \pi_\theta(\tau) - \log \pi_{\theta_{\text{ref}}}(\tau). \tag{10.51}$$

Trong thực tế, khoản phạt này có thể được xấp xỉ bằng cách chỉ xét các xác suất của chính sách và bỏ qua động lực. Điều này cho

$$\text{Penalty} \;=\; \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) - \sum_{t=1}^{T} \log \pi_{\theta_{\text{ref}}}(a_t|s_t). \tag{10.52}$$

Bằng cách đưa khoản phạt này vào mục tiêu tối ưu, ta khuyến khích chính sách hiện tại ở gần chính sách tham chiếu, hạn chế những bước cập nhật quá lớn có thể làm quá trình học mất ổn định.

Ta có thể đưa khoản phạt này vào hàm mục tiêu thay thế có cắt ngưỡng, và thu được

$$U_{\text{ppo-clip}}(\tau;\theta) \;=\; U_{\text{clip}}(\tau;\theta) - \beta\,\text{Penalty}, \tag{10.53}$$

trong đó β là trọng số của khoản phạt. Phương pháp huấn luyện này được gọi là **tối ưu chính sách cận kề** (*proximal policy optimization*) (**PPO**), một trong những phương pháp học tăng cường phổ biến nhất được dùng trong LLM và nhiều lĩnh vực khác [Schulman và cộng sự, 2017].

Chú giải của người dịch: Khoản phạt KL: "đừng đi quá xa nguồn gốc"

Khoản phạt (Phương trình (10.51)–(10.52)) đo xem chính sách mới gán xác suất khác chính sách tham chiếu đến mức nào (lấy trung bình trên các mẫu, đại lượng này xấp xỉ *độ phân kỳ KL* — một "khoảng cách" giữa hai phân phối xác suất). Vì sao cần? Mô hình phần thưởng chỉ là giám khảo *máy*, có thể có "điểm mù". Nếu để LLM tự do tối ưu, nó có thể tìm ra những câu trả lời kỳ quặc nhưng được chấm điểm cao (giống học sinh "học tủ" theo sở thích giám khảo thay vì học thật). Khoản phạt với trọng số β giữ LLM gần

mô hình ban đầu — vốn đã nói năng tự nhiên — trong khi vẫn cho phép cải thiện. *Cận kề* (*proximal*) trong tên PPO chính là ý "chỉ đi những bước gần".

Giờ ta có thể viết mục tiêu huấn luyện LLM dưới dạng PPO

$$U_{\text{ppo-clip}}(\mathbf{x}, \mathbf{y}; \theta) \;=\; U_{\text{clip}}(\mathbf{x}, \mathbf{y}; \theta) - \beta\,\text{Penalty}, \tag{10.54}$$

trong đó

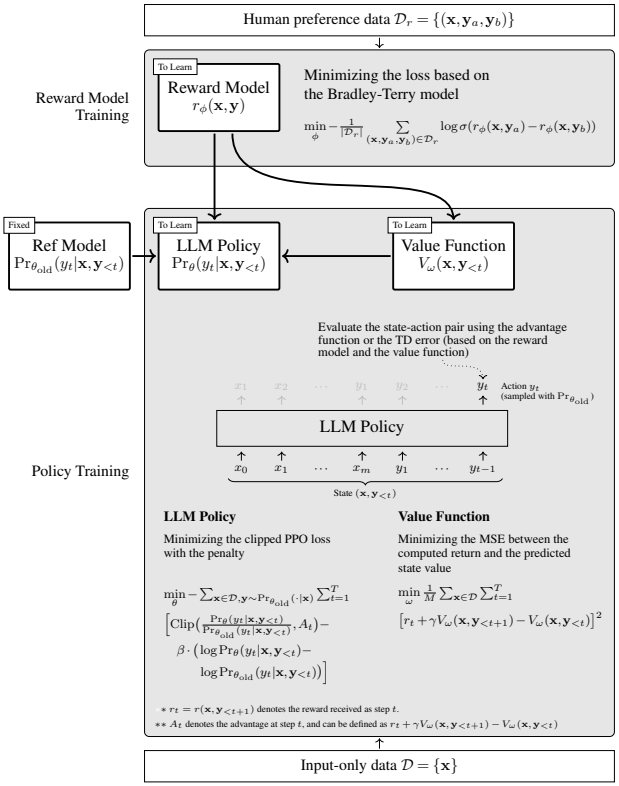
$$U_{\text{clip}}(\mathbf{x}, \mathbf{y}; \theta) = \sum_{t=1}^{T} \text{Clip}\Big(\frac{\pi_\theta(y_t|\mathbf{x}, \mathbf{y}_{<t})}{\pi_{\theta_{\text{ref}}}(y_t|\mathbf{x}, \mathbf{y}_{<t})}, A(\mathbf{x}, \mathbf{y}_{<t}, y_t)\Big), \tag{10.55}$$

$$\begin{aligned}\text{Penalty} &= \log \text{Pr}_\theta(\mathbf{y}|\mathbf{x}) - \log \text{Pr}_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \\ &= \sum_{t=1}^{T} \log \text{Pr}_\theta(y_t|\mathbf{x}, \mathbf{y}_{<t}) - \sum_{t=1}^{T} \log \text{Pr}_{\theta_{\text{ref}}}(y_t|\mathbf{x}, \mathbf{y}_{<t}).\end{aligned} \tag{10.56}$$

Mặc dù ký hiệu ở đây trông hơi rườm rà, ý tưởng của PPO rất đơn giản: ta xây dựng một mục tiêu bằng cách kết hợp tỉ số khả năng (đã cắt ngưỡng) giữa chính sách đích và chính sách tham chiếu với một hàm lợi thế, rồi áp thêm một khoản phạt bảo đảm các bước cập nhật chính sách không quá lớn. RLHF dựa trên PPO được minh họa trong Hình 10.9.

Chú giải của người dịch: Đọc các nhãn trong Hình 10.9

Human preference data — dữ liệu sở thích của con người; *Reward Model Training* — huấn luyện mô hình phần thưởng, bằng cách *cực tiểu hóa mất mát dựa trên mô hình Bradley–Terry*; *To Learn* — cần học (tham số được cập nhật); *Fixed* — cố định; *Ref Model* — mô hình tham chiếu; *LLM Policy* — chính sách LLM; *Value Function* — hàm giá trị; *Policy Training* — huấn luyện chính sách. Dòng chữ giữa hình: *đánh giá cặp trạng thái–hành động bằng hàm lợi thế hoặc sai số TD (dựa trên mô hình phần thưởng và hàm giá trị)*; *Action y_t (sampled with* $\text{Pr}_{\theta_{\text{old}}}$*)* — hành động y_t được lấy mẫu bằng mô hình cũ. Phía dưới: chính sách LLM được huấn luyện bằng *cực tiểu hóa mất mát PPO có cắt ngưỡng kèm khoản phạt*; hàm giá trị được huấn luyện bằng *cực tiểu hóa MSE (sai số bình phương trung bình) giữa lợi tức tính được và giá trị trạng thái dự đoán*. Hai ghi chú cuối: $r_t = r(\mathbf{x}, \mathbf{y}_{<t+1})$ là phần thưởng nhận được tại bước t; A_t là lợi thế tại bước t, có thể định nghĩa là $r_t + \gamma V_\omega(\mathbf{x}, \mathbf{y}_{<t+1}) - V_\omega(\mathbf{x}, \mathbf{y}_{<t})$. *Input-only data* — dữ liệu chỉ có đầu vào.



Hình 10.9: Minh họa RLHF. Bước đầu tiên là thu thập dữ liệu sở thích của con người và huấn luyện mô hình phần thưởng bằng dữ liệu này. Khi mô hình phần thưởng đã được tối ưu, cùng với mô hình tham chiếu, ta tiến hành huấn luyện cả chính sách lẫn hàm giá trị. Tại mỗi bước dự đoán, ta tính tổng mất mát dựa trên PPO và cập nhật tham số của chính sách. Việc này đòi hỏi phải có sẵn mô hình phần thưởng, mô hình tham chiếu và hàm giá trị. Đồng thời, ta cập nhật tham số của hàm giá trị bằng cách cực tiểu hóa mất mát MSE.

(Trong hình, θ_{old} đóng vai trò của θ_{ref} trong văn bản.)

Tóm lại, hiện thực RLHF đòi hỏi xây dựng bốn mô hình, tất cả đều dựa trên kiến trúc bộ giải mã Transformer.

- **Mô hình phần thưởng** ($r_\phi(\cdot)$, trong đó ϕ ký hiệu các tham số). Mô hình phần thưởng học từ dữ liệu sở thích của con người để dự đoán phần thưởng cho mỗi cặp chuỗi token đầu vào và đầu ra. Nó là một bộ giải mã Transformer theo sau bởi một tầng tuyến tính, ánh xạ một chuỗi (phép nối giữa đầu vào và đầu ra) thành một điểm phần thưởng là số thực.

- **Mô hình giá trị** hay **hàm giá trị** ($V_\omega(\cdot)$, trong đó ω ký hiệu các tham số). Hàm giá trị nhận điểm phần thưởng từ mô hình phần thưởng và được huấn luyện để dự đoán tổng phần thưởng kỳ vọng có thể thu được khi bắt đầu từ một trạng thái. Nó thường dựa trên cùng kiến trúc với mô hình phần thưởng.

- **Mô hình tham chiếu** ($\pi_{\theta_{\text{ref}}}(\cdot) = \Pr_{\theta_{\text{ref}}}(\cdot)$, trong đó θ_{ref} ký hiệu các tham số). Mô hình tham chiếu là LLM cơ sở, đóng vai trò điểm xuất phát cho việc huấn luyện chính sách. Trong RLHF, nó biểu diễn phiên bản trước đó của mô hình, hoặc một mô hình được huấn luyện mà không có phản hồi của con người. Nó được dùng để lấy mẫu các chuỗi đầu ra và tham gia vào việc tính mất mát khi huấn luyện chính sách.

- **Mô hình đích** hay **chính sách** ($\pi_\theta(\cdot) = \Pr_\theta(\cdot)$, trong đó θ ký hiệu các tham số). Chính sách này chi phối cách LLM quyết định token tiếp theo thích hợp nhất khi biết ngữ cảnh của nó. Nó được huấn luyện dưới sự giám sát của cả mô hình phần thưởng lẫn mô hình giá trị.

Trong thực tế, các mô hình này cần được huấn luyện theo một thứ tự nhất định. Trước tiên, ta cần khởi tạo chúng bằng các mô hình sẵn có. Ví dụ, mô hình phần thưởng và mô hình giá trị có thể được khởi tạo bằng một LLM đã tiền huấn luyện, còn mô hình tham chiếu và mô hình đích có thể được khởi tạo bằng một mô hình đã được tinh chỉnh theo chỉ dẫn. Lưu ý rằng, tại thời điểm này, mô hình tham chiếu đã sẵn sàng để dùng và sẽ không được cập nhật thêm. Thứ hai, ta cần thu thập dữ liệu sở thích

của con người và huấn luyện mô hình phần thưởng trên dữ liệu này. Thứ ba, cả mô hình giá trị lẫn chính sách được huấn luyện đồng thời bằng mô hình phần thưởng. Tại mỗi vị trí trong một chuỗi token đầu ra, ta cập nhật mô hình giá trị bằng cách cực tiểu hóa MSE của dự đoán giá trị, và chính sách được cập nhật bằng cách cực tiểu hóa mất mát PPO.

Chú giải của người dịch: Toàn cảnh RLHF trong một vòng lặp

Một vòng huấn luyện PPO có thể hình dung như sau: (1) lấy một lô câu hỏi $\mathbf{x}$ (chỉ có câu hỏi, không có đáp án); (2) LLM tự viết câu trả lời $\mathbf{y}$; (3) mô hình phần thưởng chấm điểm câu trả lời; (4) mô hình giá trị ước lượng "mức kỳ vọng" tại từng vị trí token, từ đó tính lợi thế A_t cho từng token; (5) cập nhật LLM bằng mất mát PPO (khuyến khích token có $A_t > 0$, có cắt ngưỡng và phạt nếu đi xa mô hình tham chiếu); (6) cập nhật mô hình giá trị để dự đoán chính xác hơn. Lặp lại nhiều lần. Bốn mô hình cùng lúc nghĩa là RLHF tốn rất nhiều bộ nhớ và tính toán — đây là động lực cho các phương pháp đơn giản hơn ở Mục 10.4.

10.4 Cải tiến việc căn chỉnh theo sở thích của con người

Ở mục trước, chúng ta đã điểm lại các khái niệm cơ bản của học tăng cường (*reinforcement learning*) và khung tổng quát của RLHF. Trong mục này, chúng ta sẽ bàn về một số cải tiến của RLHF và các phương pháp thay thế để đạt được việc căn chỉnh (*alignment*) theo sở thích của con người (*human preference*).

10.4.1 Mô hình hóa phần thưởng tốt hơn

Ở Mục 10.3.2, chúng ta đã nhấn mạnh nhiệm vụ học từ sở thích của con người cũng như việc dùng hàm mất mát xếp hạng theo cặp (*pairwise ranking loss*) để huấn luyện mô hình phần thưởng (*reward model*). Ở đây ta xem xét thêm các phương pháp mô hình hóa phần thưởng (*reward modeling*). Thảo luận của chúng ta sẽ tương đối tổng quát, và vì mô hình phần thưởng được dùng rộng rãi trong nhiều bài toán học tăng cường, ta sẽ dễ dàng áp dụng các phương pháp bàn ở đây cho RLHF và các ứng

dụng liên quan.

1. Tín hiệu giám sát

Việc huấn luyện mô hình phần thưởng nói chung có thể được xem như một bài toán xếp hạng (*ranking*), trong đó mô hình học cách gán điểm cho các đầu ra sao cho thứ hạng thu được phản ánh sở thích mà con người đã chỉ ra. Có một số cách tiếp cận để huấn luyện mô hình phần thưởng từ góc độ xếp hạng.

Một cách tiếp cận là mở rộng xếp hạng theo cặp thành xếp hạng theo danh sách (*listwise ranking*). Với mỗi mẫu trong một tập dữ liệu, ta có thể dùng LLM để sinh ra nhiều đầu ra, rồi nhờ các chuyên gia con người sắp thứ tự các đầu ra này. Chẳng hạn, cho một tập gồm bốn đầu ra $\{\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3, \mathbf{y}_4\}$, một thứ tự có thể có của chúng là $\mathbf{y}_2 \succ \mathbf{y}_3 \succ \mathbf{y}_1 \succ \mathbf{y}_4$. Một cách đơn giản để mô hình hóa những thứ hạng như vậy là cộng các hàm mất mát so sánh theo cặp. Ví dụ, ta có thể định nghĩa hàm mất mát theo danh sách bằng cách cộng dồn mất mát trên mọi cặp đầu ra:

$$\mathcal{L}_{\text{list}} = -\mathbb{E}_{(\mathbf{x},Y)\sim\mathcal{D}_r}\Big[\frac{2}{N(N-1)}\sum_{\substack{\mathbf{y}_a\in Y,\mathbf{y}_b\in Y\\ \mathbf{y}_a\succ\mathbf{y}_b}}\log \Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\Big], \tag{10.57}$$

trong đó Y là một danh sách các đầu ra, và N là số đầu ra trong danh sách. $\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})$ có thể được định nghĩa bằng mô hình Bradley-Terry, tức là $\Pr(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) = \text{Sigmoid}(r(\mathbf{x}, \mathbf{y}_a) - r(\mathbf{x}, \mathbf{y}_b))$. Ở đây ta bỏ chỉ số ϕ trên $\Pr(\cdot)$ để ký hiệu đỡ rườm rà.

Chú giải của người dịch: Vì sao có hệ số $\frac{2}{N(N-1)}$?

Với N đầu ra đã sắp thứ tự, số cặp $(\mathbf{y}_a, \mathbf{y}_b)$ mà $\mathbf{y}_a$ đứng trước $\mathbf{y}_b$ chính là số tổ hợp chập 2 của N: $C_N^2 = \frac{N(N-1)}{2}$. Nhân với $\frac{2}{N(N-1)}$ nghĩa là lấy *trung bình* mất mát trên tất cả các cặp. Ví dụ với 4 đầu ra $\mathbf{y}_2 \succ \mathbf{y}_3 \succ \mathbf{y}_1 \succ \mathbf{y}_4$, có $C_4^2 = 6$ cặp: $(\mathbf{y}_2, \mathbf{y}_3)$, $(\mathbf{y}_2, \mathbf{y}_1)$, $(\mathbf{y}_2, \mathbf{y}_4)$, $(\mathbf{y}_3, \mathbf{y}_1)$, $(\mathbf{y}_3, \mathbf{y}_4)$, $(\mathbf{y}_1, \mathbf{y}_4)$. Ký hiệu $\succ$ đọc là "được ưa thích hơn".

Một mở rộng của mô hình Bradley-Terry cho xếp hạng theo danh sách có thể dùng một cơ chế xếp hạng xét đến toàn bộ danh sách đầu ra thay vì chỉ so sánh từng cặp. Một mô hình như vậy là **mô hình Plackett-Luce** (*Plackett-Luce model*), mô hình này tổng quát hóa mô hình Bradley-Terry

để xử lý nhiều phần tử trong một thứ hạng [Plackett, 1975]. Trong mô hình Plackett-Luce, với mỗi phần tử trong danh sách, ta định nghĩa một "giá trị" (*worth*) cho phần tử đó, phản ánh sức mạnh tương đối của việc nó được chọn so với các phần tử khác. Với bài toán mô hình hóa phần thưởng ở đây, giá trị của $\mathbf{y}$ trong danh sách Y có thể được định nghĩa là

$$\alpha(\mathbf{y}) \;=\; \exp(r(\mathbf{x}, \mathbf{y})). \tag{10.58}$$

Khi đó, xác suất để $\mathbf{y}$ được chọn làm phần tử xếp hạng cao nhất từ Y là

$$\begin{aligned}\Pr(\mathbf{y} \text{ được chọn}|\mathbf{x}, Y) &= \frac{\alpha(\mathbf{y})}{\sum_{\mathbf{y}' \in Y} \alpha(\mathbf{y}')} \\ &= \frac{\exp(r(\mathbf{x}, \mathbf{y}))}{\sum_{\mathbf{y}' \in Y} \exp(r(\mathbf{x}, \mathbf{y}'))}.\end{aligned} \tag{10.59}$$

Giả sử $\mathring{Y}$ là một danh sách có thứ tự $\mathbf{y}_{j_1} \succ \mathbf{y}_{j_2} \succ \cdots \succ \mathbf{y}_{j_N}$. Log-xác suất tổng thể của danh sách có thứ tự này có thể được định nghĩa là tổng các log-xác suất có điều kiện ở mỗi bước lựa chọn:

$$\begin{aligned}\log \Pr(\mathring{Y}|\mathbf{x}) &= \log \Pr(\mathbf{y}_{j_1} \succ \mathbf{y}_{j_2} \succ \cdots \succ \mathbf{y}_{j_N}|\mathbf{x}) \\ &= \log \Pr(\mathbf{y}_{j_1}|\mathbf{x}, \{\mathbf{y}_{j_1}, \mathbf{y}_{j_2}, ..., \mathbf{y}_{j_N}\}) + \\ &\quad \log \Pr(\mathbf{y}_{j_2}|\mathbf{x}, \{\mathbf{y}_{j_2}, ..., \mathbf{y}_{j_N}\}) + \\ &\quad \cdots + \\ &\quad \log \Pr(\mathbf{y}_{j_N}|\mathbf{x}, \{\mathbf{y}_{j_N}\}) \\ &= \sum_{k=1}^{N} \log \Pr(\mathbf{y}_{j_k}|\mathbf{x}, \mathring{Y}_{\geq k}),\end{aligned} \tag{10.60}$$

trong đó $\mathring{Y}_{\geq k}$ biểu diễn tập con các đầu ra trong danh sách còn chưa được chọn ở bước thứ k, tức là $\mathring{Y}_{\geq k} = \{\mathbf{y}_{j_k}, ..., \mathbf{y}_{j_N}\}$. Với log-xác suất $\log \Pr(\mathring{Y}|\mathbf{x})$, ta có thể định nghĩa hàm mất mát dựa trên mô hình Plackett-Luce như sau

$$\mathcal{L}_{\text{pl}} \;=\; -\mathbb{E}_{(\mathbf{x}, \mathring{Y}) \sim \mathcal{D}_r} \big[\log \Pr(\mathring{Y}|\mathbf{x}) \big]. \tag{10.61}$$

Chú giải của người dịch: Mô hình Plackett-Luce: "bốc thăm" lần lượt

Hãy hình dung việc xếp hạng như bốc lần lượt từng phần tử ra khỏi một chiếc hộp: phần tử có điểm thưởng cao thì "nặng ký" hơn và dễ được bốc trước. Giả sử có 3 đầu ra với điểm thưởng $r = \ln 3,\ \ln 2,\ \ln 1$, khi đó các "giá trị" $\alpha = \exp(r)$ lần lượt là $3, 2, 1$. Xác suất của thứ tự $\mathbf{y}_1 \succ \mathbf{y}_2 \succ \mathbf{y}_3$ là: bước 1 chọn $\mathbf{y}_1$ trong cả 3 phần tử với xác suất $\frac{3}{3+2+1} = \frac{1}{2}$; bước 2 chọn $\mathbf{y}_2$ trong 2 phần tử còn lại với xác suất $\frac{2}{2+1} = \frac{2}{3}$; bước 3 chỉ còn $\mathbf{y}_3$ nên xác suất là 1. Vậy xác suất cả thứ tự là $\frac{1}{2} \cdot \frac{2}{3} \cdot 1 = \frac{1}{3}$; lấy logarit thì tích biến thành tổng, đúng như Phương trình (10.60). Khi danh sách chỉ có 2 phần tử, công thức này trùng với mô hình Bradley-Terry: $\frac{e^{r_a}}{e^{r_a}+e^{r_b}} = \text{Sigmoid}(r_a - r_b)$.

Cũng có nhiều phương pháp theo cặp và theo danh sách khác để mô hình hóa thứ hạng, chẳng hạn RankNet [Burges và cộng sự, 2005] và ListNet [Cao và cộng sự, 2007]. Tất cả các phương pháp này có thể được xếp vào một họ lớn các phương pháp học để xếp hạng (*learning-to-rank*), và phần lớn trong số chúng đều áp dụng được cho bài toán mô hình hóa sở thích của con người. Tuy nhiên, việc thảo luận các phương pháp này nằm ngoài phạm vi của chương. Bạn đọc quan tâm có thể tham khảo các sách về chủ đề này để biết thêm chi tiết [Liu, 2009; Li, 2011].

Bên cạnh xếp hạng theo cặp và theo danh sách, các phương pháp theo điểm (*pointwise methods*) cung cấp một cách khác để huấn luyện mô hình phần thưởng và nắm bắt sở thích của con người. Khác với các phương pháp tập trung vào thứ hạng tương đối giữa các đầu ra khác nhau, phương pháp theo điểm xử lý từng đầu ra một cách độc lập. Chẳng hạn, các chuyên gia con người có thể gán điểm cho từng đầu ra riêng lẻ, ví dụ chấm theo thang năm bậc. Mục tiêu là điều chỉnh mô hình phần thưởng sao cho đầu ra của nó khớp với các điểm số này. Một cách đơn giản để huấn luyện theo điểm là dùng các kỹ thuật hồi quy (*regression*), trong đó phần thưởng của mỗi đầu ra được xem là một biến mục tiêu. Gọi $\varphi(\mathbf{x}, \mathbf{y})$ là điểm con người gán cho $\mathbf{y}$ khi cho trước $\mathbf{x}$. Mô hình phần thưởng theo điểm có thể được huấn luyện bằng cách cực tiểu hóa một hàm mất mát, thường dựa trên sai số bình phương trung bình (*mean squared error*) hoặc các hàm mất mát hồi quy khác, giữa phần thưởng dự đoán $r(\mathbf{x}, \mathbf{y})$ và phản hồi thực tế của con người $\varphi(\mathbf{x}, \mathbf{y})$. Chẳng hạn, hàm mất mát có thể là

$$\mathcal{L}_{\text{point}} \ = \ \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}_r}\left[\left(\varphi(\mathbf{x},\mathbf{y}) - r(\mathbf{x},\mathbf{y})\right)^2\right]. \tag{10.62}$$

Tuy phương pháp theo điểm đơn giản hơn về mặt khái niệm và có thể trực tiếp hướng dẫn mô hình phần thưởng dự đoán điểm số, chúng không phải lúc nào cũng là lựa chọn tốt nhất trong RLHF. Một vấn đề là các phương pháp này có thể gặp khó khăn với phương sai cao trong phản hồi của con người, đặc biệt khi các chuyên gia khác nhau cho những điểm số không nhất quán đối với các đầu ra tương tự nhau. Vì chúng tập trung khớp với điểm số tuyệt đối thay vì chênh lệch tương đối, sự không nhất quán trong chấm điểm có thể khiến mô hình hoạt động kém. Hơn nữa, việc khớp với các đầu ra được chấm điểm cụ thể có thể làm hại khả năng tổng quát hóa (*generalization*), nhất là khi dữ liệu huấn luyện trong RLHF thường rất hạn chế. Ngược lại, các phương pháp xét sở thích tương đối có thể thúc đẩy việc học những khuôn mẫu sở thích tổng quát hơn. Dù vậy, vẫn có những tình huống mà phương pháp theo điểm có thể phù hợp. Ví dụ, trong các nhiệm vụ có dữ liệu huấn luyện dồi dào và chi phí thu được các nhãn chính xác, nhất quán là thấp, phương pháp theo điểm có thể tỏ ra hiệu quả.

Chú giải của người dịch: Theo điểm hay theo cặp?

Thử nghĩ đến việc chấm bài văn. Hai giám khảo có thể "khó tính" khác nhau: một người hay cho 6–7 điểm, người kia hay cho 8–9 điểm cho cùng chất lượng bài. Nếu huấn luyện theo điểm (Phương trình 10.62), mô hình bị buộc phải khớp với cả hai thang "lệch" này. Nhưng nếu chỉ hỏi "bài A hay hơn bài B không?", hai giám khảo thường đồng ý với nhau hơn nhiều. Đó là lý do so sánh tương đối (theo cặp, theo danh sách) thường bền vững hơn với dữ liệu sở thích của con người.

Thực ra, để tín hiệu giám sát dùng huấn luyện mô hình phần thưởng bền vững hơn, ta cũng có thể đưa thêm các số hạng chính quy hóa (*regularization*) vào quá trình huấn luyện. Ví dụ, nếu ta xem số hạng thứ nhất $U_{\text{ppo-clip}}(\mathbf{x}, \mathbf{y}; \theta)$ trong Phương trình (10.54) như một loại phần thưởng tổng quát hóa, thì số hạng thứ hai (tức số hạng phạt) có thể được xem là một dạng chính quy hóa cho mô hình phần thưởng, chỉ khác là ở đây mục tiêu là huấn luyện chính sách (*policy*) chứ không phải mô hình phần thưởng. Một ví dụ khác là Eisenstein và cộng sự [2023] xây dựng một số hạng chính quy hóa dựa trên bình phương của tổng các phần thưởng, rồi cộng nó vào hàm mất mát so sánh theo cặp trong RLHF:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{pair}} - \lambda \mathbb{E}_{(\mathbf{x}, \mathbf{y}_a, \mathbf{y}_b) \sim \mathcal{D}_r} \left[\left(r(\mathbf{x}, \mathbf{y}_a) + r(\mathbf{x}, \mathbf{y}_b) \right)^2 \right]$$

$$= -\mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\left[\log \Pr_{\phi}(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\right] - \lambda\mathbb{E}_{(\mathbf{x},\mathbf{y}_a,\mathbf{y}_b)\sim\mathcal{D}_r}\left[(r(\mathbf{x},\mathbf{y}_a) + r(\mathbf{x},\mathbf{y}_b))^2\right], \tag{10.63}$$

trong đó λ là trọng số của số hạng chính quy hóa. Tối ưu hóa với số hạng chính quy hóa này có thể giúp giảm nhẹ tính bất định (*underdetermination*) của mô hình phần thưởng[8].

Chú giải của người dịch: Vì sao mô hình phần thưởng "bất định"?

Mô hình Bradley-Terry chỉ quan tâm đến *hiệu* $r(\mathbf{x},\mathbf{y}_a) - r(\mathbf{x},\mathbf{y}_b)$. Nếu cộng cùng một hằng số vào mọi phần thưởng (ví dụ đổi $r_a = 3, r_b = 1$ thành $r_a = 103, r_b = 101$), hiệu vẫn là 2 và mất mát không đổi. Như vậy có vô số mô hình "tốt như nhau" với mức phần thưởng trôi nổi tùy ý. Số hạng $(r_a + r_b)^2$ nhằm kéo tổng các phần thưởng về gần 0, giúp "neo" thang đo lại: trong ví dụ trên, cặp $r_a = 1, r_b = -1$ sẽ được ưa chuộng hơn.

2. Phần thưởng thưa và phần thưởng dày

Như đã bàn ở Mục 10.3, phần thưởng do phản hồi của con người cung cấp thường *thưa*: chúng chỉ được quan sát ở cuối chuỗi, chứ không liên tục trong suốt quá trình sinh. Xử lý phần thưởng thưa (*sparse rewards*) từ lâu đã là một mối quan tâm trong học tăng cường, và là một trong những thách thức của nhiều ứng dụng thực tế. Ví dụ, trong robot học, hàm phần thưởng thường cần được "nắn" lại để dễ tối ưu hơn thay vì chỉ dựa vào phần thưởng ở cuối chuỗi. Nhiều phương pháp đã được phát triển để giải quyết vấn đề này. Một cách phổ biến là định hình phần thưởng (*reward shaping*), trong đó hàm ban đầu được sửa đổi để bao gồm các phần thưởng trung gian, nhờ đó cung cấp phản hồi tức thời hơn. Ngoài ra, ta có thể áp dụng học theo giáo trình (*curriculum learning*) để sắp xếp các nhiệm vụ sao cho độ phức tạp của chúng tăng dần. Điều này giúp mô hình thành thạo các nhiệm vụ đơn giản trước và dần chuẩn bị cho những thử thách phức tạp hơn. Có nhiều phương pháp như vậy có thể làm giảm tác động của phần thưởng thưa, chẳng hạn động lực nội tại (*intrinsic motivation*) và phát lại kinh nghiệm nhìn lại (*hindsight experience replay*). Phần lớn các phương pháp này mang tính tổng quát và được thảo luận rộng rãi trong tài

[8] Một mô hình được gọi là bất định (*underdetermined*) nếu có nhiều bộ tham số khác nhau cùng có thể đạt được một mục tiêu như nhau.

liệu về học tăng cường nói chung, như cuốn sách giáo khoa kinh điển của Sutton và Barto [2018].

Dù ở đây chúng ta không bàn chi tiết các phương pháp giảm nhẹ phần thưởng thưa, một câu hỏi thú vị nảy sinh: tại sao phần thưởng thưa lại thành công đến vậy trong RLHF? Nhớ lại từ Mục 10.3.1 rằng tín hiệu giám sát nhận được tại mỗi bước thời gian t không phải là phần thưởng của hành động hiện tại, mà là một dạng phần thưởng tích lũy từ t cho đến bước thời gian cuối cùng. Những tín hiệu giám sát như vậy là *dày* trên chuỗi, vì phần thưởng nhận được ở cuối chuỗi có thể được truyền ngược về bước thời gian đó, bất kể đó là bước nào. Nói cách khác, phần thưởng thưa đã được biến đổi thành tín hiệu giám sát dày. Hơn nữa, từ góc độ định hình phần thưởng, Ng và cộng sự [1999] chỉ ra rằng phần thưởng tại t được tính bằng

$$r'(s_t, a_t, s_{t+1}) \;=\; r(s_t, a_t, s_{t+1}) + f(s_t, a_t, s_{t+1}), \tag{10.64}$$

trong đó $r'(\cdot)$ là hàm phần thưởng đã biến đổi, $r(\cdot)$ là hàm phần thưởng ban đầu, và $f(\cdot)$ là hàm phần thưởng định hình (*shaping reward function*). Để đảm bảo tính tối ưu của chính sách dưới hàm phần thưởng đã biến đổi, hàm phần thưởng định hình có thể được cho ở dạng

$$f(s_t, a_t, s_{t+1}) \;=\; \gamma\Phi(s_{t+1}) - \Phi(s_t), \tag{10.65}$$

trong đó $\Phi(s)$ được gọi là giá trị thế năng (*potential value*) của trạng thái s. Nếu ta định nghĩa $\Phi(s)$ là hàm giá trị như trong Phương trình (10.15) và thế Phương trình (10.65) vào Phương trình (10.64), ta thu được

$$r'(s_t, a_t, s_{t+1}) \;=\; r(s_t, a_t, s_{t+1}) + \gamma V(s_{t+1}) - V(s_t). \tag{10.66}$$

Thật thú vị khi thấy biểu thức này tương đương với sai số chênh lệch thời gian (*temporal-difference error*) một bước được dùng trong PPO. Điều này liên hệ các phương pháp dựa trên lợi thế (*advantage*) với định hình phần thưởng: lợi thế về bản chất là một phần thưởng đã được định hình.

Chú giải của người dịch: Định hình phần thưởng dựa trên thế năng

Tên gọi "thế năng" gợi nhớ đến thế năng trong Vật lý: công của trọng lực khi đi từ A đến B chỉ phụ thuộc vào hiệu thế năng ở hai đầu, không phụ thuộc

đường đi. Tương tự, nếu cộng $f = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ ở mọi bước rồi tính tổng dọc theo cả chuỗi (tạm lấy $\gamma = 1$), các số hạng ở giữa triệt tiêu nhau: $(\Phi(s_2) - \Phi(s_1)) + (\Phi(s_3) - \Phi(s_2)) + \cdots = \Phi(s_{\text{cuối}}) - \Phi(s_1)$. Vì vậy phần thưởng bổ sung không làm thay đổi việc chuỗi hành động nào là tốt nhất, nhưng lại cho tác tử những "gợi ý" ở từng bước, giúp học nhanh hơn.

Mặt khác, một trong những lý do dùng phần thưởng ở cuối chuỗi nằm ở bản chất của các nhiệm vụ RLHF. Khác với các môi trường học tăng cường truyền thống, nơi tác tử (*agent*) tương tác với một môi trường động, các nhiệm vụ RLHF thường liên quan đến việc ra quyết định phức tạp dựa trên các quá trình ngôn ngữ hoặc các quá trình nhận thức cấp cao khác. Những quá trình này không dễ tạo ra các phần thưởng trung gian thường xuyên và có ý nghĩa, vì chất lượng và mức độ phù hợp của các hành động chỉ có thể được đánh giá đầy đủ sau khi quan sát tác động của chúng trong bối cảnh lớn hơn của toàn bộ chuỗi hoặc nhiệm vụ. Trong trường hợp này, tín hiệu phần thưởng dựa trên phản hồi của con người, tuy rất thưa, thường lại rất giàu thông tin và chính xác. Do đó, tính thưa này, cùng với lượng thông tin lớn và độ chính xác cao của phản hồi từ con người, có thể khiến việc học vừa bền vững vừa hiệu quả.

3. Phần thưởng chi tiết

Với nhiều ứng dụng, mục tiêu của chúng ta sẽ phức tạp hơn việc chỉ đánh giá toàn bộ một văn bản. Chẳng hạn, trong phân tích cảm xúc (*sentiment analysis*), ta thường không chỉ xác định cảm xúc của một văn bản, mà cần phân tích cảm xúc chi tiết hơn bằng cách gắn nó với các khía cạnh cụ thể của chủ đề được bàn đến trong văn bản. Xét câu "The camera of the phone is excellent, but the battery life is disappointing" (Camera của điện thoại thì tuyệt vời, nhưng thời lượng pin thì đáng thất vọng). Trong ví dụ này, ta cần phân tích riêng cảm xúc được bày tỏ về camera và về pin. Kiểu phân tích này, gọi là phân tích cảm xúc theo khía cạnh (*aspect-based sentiment analysis*), giúp hiểu nhận xét của khách hàng ở mức chi tiết hơn so với phân tích cảm xúc thông thường.

Với bài toán mô hình hóa phần thưởng, ta thường cần mô hình hóa các phần khác nhau của một chuỗi ở mức chi tiết tương tự. Một cách đơn giản để làm điều này là chia chuỗi thành các đoạn riêng biệt và tính phần thưởng cho từng đoạn một cách độc lập [Wu và cộng sự, 2023b]. Giả sử

một chuỗi token đầu ra $\mathbf{y}$ có thể được chia thành n_s đoạn $\{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_s}\}$ dựa trên một tiêu chí nào đó. Khi đó ta có thể dùng mô hình phần thưởng để đánh giá từng đoạn. Bằng cách xét đầu vào $\mathbf{x}$, toàn bộ đầu ra $\mathbf{y}$ và đoạn cụ thể $\bar{\mathbf{y}}_k$, điểm thưởng cho đoạn thứ k là

$$r^k \;=\; r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k). \tag{10.67}$$

Tổng điểm thưởng cho toàn bộ chuỗi đầu ra khi đó chỉ đơn giản là tổng của các phần thưởng cấp đoạn này

$$r(\mathbf{x}, \mathbf{y}) \;=\; \sum_{k=1}^{n_s} r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k). \tag{10.68}$$

Phần thưởng $r(\mathbf{x}, \mathbf{y})$ này sau đó có thể được dùng để huấn luyện chính sách bằng các thuật toán học tăng cường tiêu chuẩn như PPO.

Một vấn đề của cách tiếp cận này là huấn luyện mô hình phần thưởng ở cấp đoạn không đơn giản như học từ sở thích của con người trên toàn bộ văn bản, vì rất khó thu thập dữ liệu sở thích của con người ở cấp đoạn. Với các bài toán dạng chấm điểm (ví dụ, ta chấm một đoạn theo mức độ chứa thông tin sai lệch), một cách đơn giản là gán điểm cho từng đoạn và huấn luyện mô hình phần thưởng bằng phương pháp theo điểm. Chẳng hạn, ta có thể dùng một LLM mạnh để chấm các chuỗi $\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_{k-1}$ và $\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_k$, thu được các điểm $s(\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_{k-1})$ và $s(\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_k)$. Khi đó, ta có thể định nghĩa điểm của đoạn $\bar{\mathbf{y}}_k$ là hiệu giữa $s(\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_k)$ và $s(\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_{k-1})$

$$s(\bar{\mathbf{y}}_k) \;=\; s(\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_k) - s(\bar{\mathbf{y}}_1...\bar{\mathbf{y}}_{k-1}). \tag{10.69}$$

Với các điểm cấp đoạn này, ta có thể huấn luyện mô hình phần thưởng bằng một hàm mất mát hồi quy

$$\mathcal{L}_{\text{rating}} \;=\; \mathbb{E}_{(\mathbf{x},\mathbf{y},\bar{\mathbf{y}}_k)\sim\mathcal{D}_r}\left[(s(\bar{\mathbf{y}}_k) - r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k))^2\right]. \tag{10.70}$$

Chú giải của người dịch: Điểm của một đoạn bằng "phần đóng góp thêm"

Giống như tính điểm đóng góp của từng cầu thủ bằng cách xem tỉ số thay đổi thế nào khi họ vào sân. Ví dụ, một câu trả lời gồm 3 đoạn; LLM chấm phần $\bar{\mathbf{y}}_1$ được 6 điểm, phần $\bar{\mathbf{y}}_1\bar{\mathbf{y}}_2$ được 8 điểm, phần $\bar{\mathbf{y}}_1\bar{\mathbf{y}}_2\bar{\mathbf{y}}_3$ được 5 điểm. Theo Phương trình (10.69), đoạn 2 có điểm $8 - 6 = +2$ (làm câu trả lời tốt hơn),

còn đoạn 3 có điểm $5 - 8 = -3$ (có lẽ chứa thông tin sai, làm câu trả lời tệ đi).

Đôi khi, việc căn chỉnh có thể được xem như một bài toán phân loại (*classification*), chẳng hạn ta đánh giá xem một đoạn có vấn đề về đạo đức hay không. Trong trường hợp này, đoạn văn có thể được gán nhãn là có đạo đức hoặc phi đạo đức, bởi con người hoặc bằng các bộ phân loại bổ sung. Với các nhãn rời rạc này, ta có thể huấn luyện mô hình phần thưởng bằng các hàm mất mát phân loại. Giả sử nhãn đúng là $\hat{r} \in \{1, -1\}$, trong đó -1 chỉ một đoạn phi đạo đức và 1 chỉ một đoạn có đạo đức, và gọi $r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k)$ là đầu ra vô hướng do mô hình phần thưởng tạo ra[9]. Khi đó mô hình phần thưởng có thể được huấn luyện bằng hàm mất mát bản lề (*hinge loss*):

$$\mathcal{L}_{\text{hinge}} \;=\; \mathbb{E}_{(\mathbf{x},\mathbf{y},\bar{\mathbf{y}}_k)\sim\mathcal{D}_r}\big[\max(0, 1 - r(\mathbf{x}, \mathbf{y}, \bar{\mathbf{y}}_k) \cdot \hat{r})\big], \tag{10.71}$$

trong đó $\hat{r} \in \{1, -1\}$ ký hiệu nhãn đúng.

Chú giải của người dịch: Hàm mất mát bản lề

Đặt $m = r \cdot \hat{r}$. Nếu mô hình đoán đúng dấu và "tự tin" ($m \geq 1$) thì $\max(0, 1 - m) = 0$: không bị phạt. Ví dụ đoạn có đạo đức ($\hat{r} = 1$) và mô hình cho $r = 2$: mất mát $\max(0, 1 - 2) = 0$. Nếu mô hình cho $r = 0.3$ (đúng dấu nhưng chưa đủ chắc): mất mát 0.7. Nếu cho $r = -1$ (sai dấu): mất mát 2. Đồ thị của hàm này gãy khúc tại $m = 1$, trông giống một chiếc bản lề – vì vậy mà có tên gọi.

Vấn đề còn lại ở đây là cách chia $\mathbf{y}$ thành các đoạn. Một cách là định nghĩa phép chia có độ dài cố định, trong đó $\mathbf{y}$ được chia thành các khúc dài bằng nhau. Tuy nhiên, cách này không phải lúc nào cũng lý tưởng, vì nội dung của chuỗi có thể không khớp tốt với các ranh giới cố định. Một cách khác là chia $\mathbf{y}$ dựa trên các dấu hiệu ngôn ngữ hoặc ngữ nghĩa cụ thể, như ranh giới câu, sự chuyển chủ đề, hoặc các cấu trúc có ý nghĩa khác trong văn bản. Phép chia như vậy có thể thực hiện bằng các hệ thống phân đoạn ngôn ngữ hoặc bằng cách dùng prompt để LLM xác định các điểm ngắt tự nhiên trong chuỗi. Một cách tiếp cận khác nữa là dùng các phương

[9]Để mô hình phần thưởng có thể đưa ra xác suất phân loại hoặc logit, ta có thể điều chỉnh tầng tuyến tính mô tả ở Mục 10.3.2 cho phù hợp.

pháp phân đoạn động dựa trên độ phức tạp của chuỗi. Ví dụ, ranh giới đoạn có thể được đặt ở những chỗ điểm thưởng thay đổi đáng kể, điều có thể tương ứng với sự chuyển đổi trong nhiệm vụ đang được mô hình hóa.

4. Kết hợp các mô hình phần thưởng

Một mô hình phần thưởng có thể được xem là một đại diện thay thế (*proxy*) cho môi trường. Vì môi trường thật thường quá phức tạp hoặc không biết trước, nói chung không thể xây dựng một đại diện hoàn hảo cho môi trường. Kết quả là, việc căn chỉnh LLM quá mức theo đại diện không hoàn hảo này có thể làm giảm hiệu năng, hiện tượng được gọi là **vấn đề tối ưu hóa quá mức** (*overoptimization problem*) [Stiennon và cộng sự, 2020; Gao và cộng sự, 2023a][10]. Ta cũng có thể giải thích điều này qua định luật Goodhart (*Goodhart's law*), phát biểu rằng: *khi một thước đo trở thành mục tiêu, nó không còn là một thước đo tốt nữa* [Goodhart, 1984].

Giải quyết vấn đề tối ưu hóa quá mức không hề dễ, và hiện chưa có giải pháp nào hoàn toàn thỏa đáng. Cách lý tưởng có lẽ là xây dựng một mô hình phần thưởng "tiên tri" (*oracle*) có thể nắm bắt hoàn hảo mục tiêu thật của nhiệm vụ và ngăn tác tử "gian lận". Tuy nhiên, tạo ra một mô hình như vậy là cực kỳ khó do độ phức tạp của môi trường thực tế, cũng như thách thức trong việc định nghĩa mọi yếu tố liên quan góp phần tạo nên kết quả mong muốn. Thay vào đó, một cách thực tế hơn là kết hợp nhiều mô hình phần thưởng, nhờ đó giảm sự lệch giữa mục tiêu huấn luyện và mục tiêu thật vốn nảy sinh khi chỉ dùng một mô hình phần thưởng duy nhất [Coste và cộng sự, 2024].

Kết hợp một tập các mô hình phần thưởng là việc đơn giản về mặt khái niệm và thường có thể được phát biểu như một bài toán học kết hợp (*ensemble learning*). Một cách đơn giản nhưng phổ biến là kết hợp đầu ra

[10] Vấn đề này còn được gọi là **lách phần thưởng** (*reward hacking*) hay **lợi dụng phần thưởng** (*reward gaming*) [Krakovna và cộng sự, 2020; Skalse và cộng sự, 2022; Pan và cộng sự, 2022], chỉ hiện tượng tác tử cố "đánh lừa" mô hình phần thưởng nhưng không làm cho hành động của mình khớp với mục tiêu thực sự của nhiệm vụ. Hãy tưởng tượng một học sinh được giao bài tập về nhà và được thưởng điểm hoặc lời khen khi hoàn thành. Học sinh đó có thể tìm cách hoàn thành bài tập với ít công sức nhất để tối đa hóa phần thưởng, chẳng hạn sao chép lời giải từ internet hoặc từ các bài tập trước, thay vì tự giải các bài toán.

của các mô hình này để có được ước lượng phần thưởng đáng tin cậy hơn:

$$r_{\text{combine}} = \sum_{k=1}^{K} w_k \cdot r_k(\mathbf{x}, \mathbf{y}), \tag{10.72}$$

trong đó $r_k(\cdot)$ là mô hình phần thưởng thứ k trong tập kết hợp, w_k là trọng số của $r_k(\cdot)$, và K là số mô hình phần thưởng. Phần thưởng kết hợp này sau đó có thể được dùng để giám sát việc huấn luyện một chính sách. Thực ra, có nhiều cách để kết hợp các mô hình khác nhau, chẳng hạn có thể dự đoán bằng lấy trung bình mô hình Bayes (*Bayesian model averaging*) hoặc xây dựng một mạng hợp nhất để học cách kết hợp dự đoán từ các mô hình khác nhau. Một lựa chọn khác là phát biểu nhiệm vụ này như một bài toán tối ưu hóa đa mục tiêu (*multi-objective optimization*) và dùng nhiều mô hình phần thưởng để huấn luyện chính sách đồng thời. Các phương pháp này đã được thảo luận rộng rãi trong tài liệu về tối ưu hóa và học máy [Miettinen, 1999; Bishop, 2006].

Chú giải của người dịch: Vì sao kết hợp nhiều "giám khảo"?

Phương trình (10.72) giống cách tính điểm trung bình môn có hệ số: mỗi mô hình phần thưởng là một "giám khảo", w_k là hệ số của giám khảo đó. Ví dụ, hai mô hình cho điểm $r_1 = 4$, $r_2 = 2$ với $w_1 = 0.7$, $w_2 = 0.3$ thì $r_{\text{combine}} = 0.7 \cdot 4 + 0.3 \cdot 2 = 3.4$. Mỗi giám khảo đều có những "điểm mù" riêng mà chính sách có thể lợi dụng; nhưng khó tìm được một mẹo lừa được *tất cả* giám khảo cùng lúc, nên lách phần thưởng trở nên khó hơn.

Ngoài các phương pháp kết hợp mô hình, một vấn đề quan trọng khác là làm sao thu thập hoặc xây dựng nhiều mô hình phần thưởng khác nhau. Một trong những cách đơn giản nhất là dùng các kỹ thuật học kết hợp, chẳng hạn xây dựng các mô hình phần thưởng đa dạng từ các tập con khác nhau của một tập dữ liệu cho trước hoặc từ nhiều nguồn dữ liệu khác nhau. Với RLHF, cũng có thể xây dựng các mô hình phần thưởng dựa trên việc xem xét các khía cạnh khác nhau của căn chỉnh. Ví dụ, ta có thể xây dựng một mô hình phần thưởng để đánh giá độ chính xác về mặt sự kiện của đầu ra và một mô hình phần thưởng khác để đánh giá tính đầy đủ của đầu ra. Hai mô hình này bổ sung cho nhau và có thể được kết hợp để cải thiện việc đánh giá tổng thể đầu ra. Một cách tiếp cận khác là dùng các LLM có sẵn khác nhau làm mô hình phần thưởng. Cách này đơn giản và thực tế, vì có nhiều LLM đã được tiền huấn luyện mạnh sẵn có và ta chỉ

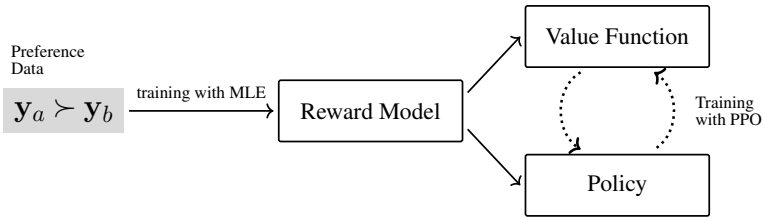
cần dùng chúng với ít hoặc không cần sửa đổi. Điều này đặt ra một câu hỏi thú vị: liệu một LLM được căn chỉnh theo các LLM khác có thể vượt qua chính các LLM đó không? Thoạt nhìn có lẽ là không. Theo trực giác, người ta có thể cho rằng một LLM đích chỉ bắt chước các LLM khác dựa trên sự giám sát hạn chế sẽ khó nắm bắt đầy đủ những hành vi tinh tế của các "người giám sát" mình. Tuy nhiên, với khả năng tổng quát hóa mạnh của LLM, cách tiếp cận này thực ra có thể khá hữu ích. Ví dụ, dùng các LLM mã nguồn mở hoặc thương mại làm mô hình phần thưởng đã cho thấy hiệu năng mạnh trong việc căn chỉnh LLM, thậm chí đạt kết quả tiên tiến nhất trên một số nhiệm vụ phổ biến [Lambert và cộng sự, 2024].

10.4.2 Tối ưu hóa sở thích trực tiếp

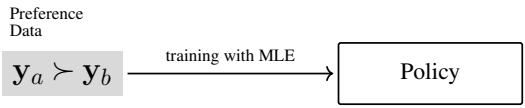
Mặc dù học mô hình phần thưởng là một bước tiêu chuẩn trong học tăng cường, nó khiến toàn bộ quá trình huấn luyện phức tạp hơn nhiều so với huấn luyện có giám sát thông thường. Bản thân việc huấn luyện một mô hình phần thưởng đáng tin cậy đã không dễ, và một mô hình phần thưởng được huấn luyện kém có thể ảnh hưởng lớn đến kết quả học chính sách. Giờ ta xét một phương pháp căn chỉnh thay thế, gọi là **tối ưu hóa sở thích trực tiếp** (*direct preference optimization*) (**DPO**), phương pháp này đơn giản hóa khung huấn luyện bằng cách loại bỏ nhu cầu mô hình hóa phần thưởng một cách tường minh [Rafailov và cộng sự, 2024]. Phương pháp này tối ưu hóa trực tiếp chính sách dựa trên sở thích của người dùng, thay vì huấn luyện một mô hình phần thưởng riêng. Nhờ đó, ta có thể đạt được việc căn chỉnh theo sở thích của con người theo cách giống như học có giám sát. Hình 10.10 so sánh phương pháp RLHF tiêu chuẩn và phương pháp DPO.

Trước khi suy ra mục tiêu của DPO, hãy cùng điểm lại mục tiêu huấn luyện chính sách được dùng trong RLHF. Như đã bàn ở Mục 10.3.3, chính sách thường được huấn luyện bằng cách tối ưu một hàm mất mát có số hạng phạt. Phương pháp DPO giả định một hàm mất mát đơn giản, trong đó chất lượng của đầu ra $\mathbf{y}$ khi cho đầu vào $\mathbf{x}$ được đánh giá bởi mô hình phần thưởng $r(\mathbf{x}, \mathbf{y})$. Mục tiêu huấn luyện do đó là

$$\tilde{\theta} \;=\; \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}} \mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})} \Big[\underbrace{-r(\mathbf{x},\mathbf{y})}_{\text{mất mát}} + \beta \underbrace{\big(\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \big)}_{\text{hình phạt}} \Big]. \tag{10.73}$$



(a) RLHF (PPO)



(b) DPO

Hình 10.10: RLHF tiêu chuẩn (PPO) so với DPO. Trong RLHF, dữ liệu sở thích của con người được dùng để huấn luyện một mô hình phần thưởng (*Reward Model*), mô hình này sau đó được dùng trong việc huấn luyện chính sách (*Policy*) cũng như hàm giá trị (*Value Function*). Trong DPO, việc dùng dữ liệu sở thích của con người trực tiếp hơn, và chính sách được huấn luyện trên dữ liệu này mà không cần huấn luyện mô hình phần thưởng. (Nhãn trong hình: *Preference Data* – dữ liệu sở thích; *training with MLE* – huấn luyện bằng ước lượng hợp lý cực đại; *Training with PPO* – huấn luyện bằng PPO.)

Lưu ý rằng trong bài toán tối ưu này, chỉ có số hạng $\pi_\theta(\mathbf{y}|\mathbf{x})$ phụ thuộc vào chính sách đích $\pi_\theta(\cdot)$. Cả mô hình phần thưởng $r(\mathbf{x}, \mathbf{y})$ lẫn mô hình tham chiếu (*reference model*) $\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})$ đều được giả định là cố định khi cho $\mathbf{x}$ và $\mathbf{y}$. Đây là một giả định mạnh so với PPO, nhưng như sẽ thấy ở phần sau, nó đơn giản hóa bài toán và rất quan trọng để suy ra mục tiêu của DPO.

Vì θ là biến ta muốn tối ưu, ta sắp xếp lại vế phải của Phương trình (10.73) để làm rõ sự phụ thuộc vào θ:

$$
\begin{aligned}
&\tilde{\theta} \\
&= \arg\min_\theta \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\big[\beta \log \pi_\theta(\mathbf{y}|\mathbf{x}) - \beta \log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) - r(\mathbf{x}, \mathbf{y})\big] \\
&= \arg\min_\theta \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\Big[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \big(\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) + \frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\big)\Big] \\
&= \arg\min_\theta \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\Big[\underbrace{\log \pi_\theta(\mathbf{y}|\mathbf{x})}_{\text{phụ thuộc } \theta} \\
&\qquad - \underbrace{\log\Big(\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\big(\tfrac{1}{\beta} r(\mathbf{x}, \mathbf{y})\big)\Big)}_{\text{không phụ thuộc } \theta}\Big].
\end{aligned}
\tag{10.74}
$$

Phương trình này định nghĩa hàm mục tiêu là hiệu giữa log-xác suất của chính sách và một hàm khác của $\mathbf{y}$. Dạng hàm mục tiêu này chưa lý tưởng, vì ta thường muốn thấy hiệu giữa hai phân phối, để có thể diễn giải hiệu này như một loại độ phân kỳ giữa các phân phối. Một ý tưởng đơn giản là biến số hạng thứ hai (tức $\log \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}))$) thành một phân phối log-xác suất trên miền của $\mathbf{y}$. Nếu ta xem $\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y}))$ là một xác suất chưa chuẩn hóa của y, ta có thể biến nó thành xác suất đã chuẩn hóa bằng cách chia cho một hệ số chuẩn hóa (*normalization factor*):

$$
Z(\mathbf{x}) = \sum_{\mathbf{y}} \pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\big(\tfrac{1}{\beta} r(\mathbf{x}, \mathbf{y})\big). \tag{10.75}
$$

Do đó, ta có thể định nghĩa một phân phối xác suất bởi

$$
\pi^*(\mathbf{y}|\mathbf{x}) = \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x}) \exp\big(\frac{1}{\beta} r(\mathbf{x}, \mathbf{y})\big)}{Z(\mathbf{x})}. \tag{10.76}
$$

Sau đó ta viết lại Phương trình (10.74) thành

$$\begin{aligned}\tilde{\theta} &= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\Big[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})\exp\left(\frac{1}{\beta}r(\mathbf{x},\mathbf{y})\right)}{Z(\mathbf{x})} \\ &\qquad - \log Z(\mathbf{x})\Big] \\ &= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\Big[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi^*(\mathbf{y}|\mathbf{x}) - \log Z(\mathbf{x})\Big] \\ &= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\Big[\mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\Big[\log \pi_\theta(\mathbf{y}|\mathbf{x}) - \log \pi^*(\mathbf{y}|\mathbf{x})\Big] \\ &\qquad - \mathbb{E}_{\mathbf{y}\sim\pi_\theta(\cdot|\mathbf{x})}\big[\log Z(\mathbf{x})\big]\Big] \\ &= \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\Big[\underbrace{D_{\text{KL}}\big(\pi_\theta(\cdot|\mathbf{x}) \,\|\, \pi^*(\cdot|\mathbf{x})\big)}_{\text{độ phân kỳ KL}} - \underbrace{\log Z(\mathbf{x})}_{\text{hằng số đối với } \theta}\Big]. \end{aligned} \tag{10.77}$$

Vì $\log Z(\mathbf{x})$ không phụ thuộc vào θ, nó không ảnh hưởng đến kết quả của phép $\arg\min_\theta$, và có thể bỏ khỏi hàm mục tiêu. Giờ ta thu được một mục tiêu huấn luyện mới, tìm chính sách tối ưu π_θ bằng cách cực tiểu hóa độ phân kỳ KL (*KL divergence*) giữa $\pi_\theta(\cdot|\mathbf{x})$ và $\pi^*(\cdot|\mathbf{x})$

$$\tilde{\theta} \;=\; \arg\min_{\theta} \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\Big[D_{\text{KL}}\big(\pi_\theta(\cdot|\mathbf{x}) \,\|\, \pi^*(\cdot|\mathbf{x})\big)\Big]. \tag{10.78}$$

Rõ ràng, nghiệm của bài toán tối ưu này là

$$\begin{aligned}\pi_\theta(\mathbf{y}|\mathbf{x}) &= \pi^*(\mathbf{y}|\mathbf{x}) \\ &= \frac{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})\exp\left(\frac{1}{\beta}r(\mathbf{x},\mathbf{y})\right)}{Z(\mathbf{x})}. \end{aligned} \tag{10.79}$$

Chú giải của người dịch: Vì sao nghiệm "rõ ràng" là $\pi_\theta = \pi^*$?

Độ phân kỳ KL $D_{\text{KL}}(p \,\|\, q)$ đo mức "khác nhau" giữa hai phân phối xác suất p và q. Nó có hai tính chất quan trọng: luôn ≥ 0, và bằng 0 khi và chỉ khi $p = q$. Vậy giá trị nhỏ nhất có thể của Phương trình (10.78) là 0, đạt được đúng khi chính sách trùng với π^*. Về trực giác, π^* là mô hình tham chiếu được "nghiêng" về phía các câu trả lời có phần thưởng cao: mỗi câu trả lời được nhân thêm hệ số $\exp(r/\beta)$. Ví dụ, với $\beta = 1$, hai câu trả lời có $\pi_{\theta_{\text{ref}}} = 0.5$ và 0.5, phần thưởng $r = \ln 3$ và $r = \ln 1 = 0$: trọng số chưa chuẩn hóa là 1.5 và 0.5, $Z = 2$, nên $\pi^* = 0.75$ và 0.25. Hệ số β càng lớn

thì sự nghiêng càng ít, tức chính sách càng bám sát mô hình tham chiếu.

Từ phương trình này, ta có thể biểu diễn phần thưởng $r(\mathbf{x}, \mathbf{y})$ bằng mô hình đích $\pi_\theta(\mathbf{y}|\mathbf{x})$, mô hình tham chiếu $\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})$ và hệ số chuẩn hóa $Z(\mathbf{x})$:

$$r(\mathbf{x}, \mathbf{y}) \;=\; \beta\Big(\log \frac{\pi_\theta(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})} + \log Z(\mathbf{x})\Big). \tag{10.80}$$

Điều này thú vị ở chỗ ban đầu ta muốn học chính sách $\pi_\theta(\cdot)$ bằng mô hình phần thưởng $r(\mathbf{x}, \mathbf{y})$, nhưng cuối cùng lại thu được một biểu diễn của mô hình phần thưởng dựa trên chính sách. Với mô hình phần thưởng định nghĩa ở Phương trình (10.80), ta có thể áp dụng nó vào mô hình Bradley-Terry để tính xác suất sở thích (xem thêm Mục 10.3.2):

$$\begin{aligned}
\Pr_\theta(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) &= \text{Sigmoid}(r(\mathbf{x}, \mathbf{y}_a) - r(\mathbf{x}, \mathbf{y}_b)) \\
&= \text{Sigmoid}\Big(\beta\Big(\log \frac{\pi_\theta(\mathbf{y}_a|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a|\mathbf{x})} + \log Z(\mathbf{x})\Big) - \\
&\qquad \beta\Big(\log \frac{\pi_\theta(\mathbf{y}_b|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b|\mathbf{x})} + \log Z(\mathbf{x})\Big)\Big) \\
&= \text{Sigmoid}\Big(\beta \log \frac{\pi_\theta(\mathbf{y}_a|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_a|\mathbf{x})} - \beta \log \frac{\pi_\theta(\mathbf{y}_b|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}_b|\mathbf{x})}\Big).
\end{aligned} \tag{10.81}$$

Công thức này rất đẹp vì nó biến hiệu các phần thưởng thành hiệu các hàm tỉ số, và ta không cần tính giá trị của $Z(\mathbf{x})$. Nhờ đó, ta không cần mô hình phần thưởng nữa, mà chỉ cần chính sách đích và mô hình tham chiếu để tính xác suất sở thích. Cuối cùng, ta có thể huấn luyện chính sách đích bằng cách cực tiểu hóa hàm mất mát DPO sau

$$\mathcal{L}_{\text{dpo}}(\theta) \;=\; -\mathbb{E}_{(\mathbf{x}, \mathbf{y}_a, \mathbf{y}_b) \sim \mathcal{D}_r}\big[\log \Pr_\theta(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x})\big]. \tag{10.82}$$

Dạng của hàm mất mát này rất giống với hàm mất mát dùng để huấn luyện mô hình phần thưởng trong RLHF (xem Phương trình (10.36)). Nhưng cần lưu ý rằng hàm mất mát ở đây phụ thuộc vào tham số của chính sách (tức θ) chứ không phải tham số của mô hình phần thưởng (tức ϕ).

Chú giải của người dịch: DPO hoạt động thế nào, nói một cách giản dị

Đại lượng $\log \frac{\pi_\theta(\mathbf{y}|\mathbf{x})}{\pi_{\theta_{\text{ref}}}(\mathbf{y}|\mathbf{x})}$ cho biết mô hình đang huấn luyện đã *tăng* (nếu dương) hay *giảm* (nếu âm) xác suất sinh ra câu trả lời $\mathbf{y}$ bao nhiêu so với mô hình ban đầu. Cực tiểu hóa Phương trình (10.82) nghĩa là đẩy cho lượng tăng của câu trả lời được ưa thích $\mathbf{y}_a$ lớn hơn lượng tăng của câu trả lời bị chê $\mathbf{y}_b$. Ví dụ, nếu sau huấn luyện $\pi_\theta(\mathbf{y}_a|\mathbf{x})$ gấp đôi so với ban đầu còn $\pi_\theta(\mathbf{y}_b|\mathbf{x})$ chỉ còn một nửa, thì với $\beta = 1$ đối số của Sigmoid là $\ln 2 - \ln \frac{1}{2} = \ln 4 \approx 1.39$, cho $\Pr_\theta(\mathbf{y}_a \succ \mathbf{y}_b|\mathbf{x}) = \frac{4}{4+1} = 0.8$. Toàn bộ quá trình chỉ cần dữ liệu cặp và hai mô hình ngôn ngữ, giống như huấn luyện có giám sát thông thường.

Ưu điểm chính của DPO nằm ở sự đơn giản và hiệu quả. Mục tiêu của DPO rất trực tiếp — nó tối ưu trực tiếp theo phản hồi dựa trên sở thích, thay vì dựa vào các mô hình phần thưởng được xây dựng riêng. Hơn nữa, DPO nhìn chung hiệu quả hơn về mẫu, vì nó học từ một tập dữ liệu cố định mà không cần quá trình lấy mẫu tốn kém về tính toán như trong PPO. Điều này khiến DPO trở thành một phương pháp phổ biến để căn chỉnh theo sở thích của con người, đặc biệt khi việc xây dựng và áp dụng mô hình phần thưởng thông qua học tăng cường gặp nhiều khó khăn.

Nhìn chung, DPO có thể được xem là một phương pháp **học tăng cường ngoại tuyến** (*offline reinforcement learning*), trong đó dữ liệu huấn luyện được thu thập trước và cố định, và không có sự khám phá (*exploration*). Ngược lại, các phương pháp học tăng cường trực tuyến (*online reinforcement learning*) như PPO, vốn đòi hỏi khám phá các trạng thái mới thông qua tương tác với môi trường (dùng mô hình phần thưởng làm đại diện), cũng có những ưu điểm riêng. Một trong những lợi ích của học tăng cường trực tuyến là nó cho phép tác tử liên tục thích nghi với những thay đổi của môi trường bằng cách học từ phản hồi theo thời gian thực. Điều này có nghĩa là, khác với các phương pháp ngoại tuyến, phương pháp trực tuyến không bị ràng buộc bởi bản chất tĩnh của dữ liệu thu thập sẵn và có thể khám phá ra các chiến lược giải quyết vấn đề mới. Ngoài ra, việc khám phá có thể giúp tác tử bao quát được một phạm vi rộng hơn các cặp trạng thái–hành động, nhờ đó cải thiện khả năng tổng quát hóa. Đây có thể là một ưu điểm quan trọng đối với LLM, vì khả năng tổng quát hóa được xem là một khía cạnh then chốt khi ứng dụng những mô hình lớn như vậy.

10.4.3 Tự động sinh dữ liệu sở thích

Mặc dù học từ sở thích của con người là một phương pháp hiệu quả và phổ biến để căn chỉnh LLM, việc gán nhãn dữ liệu sở thích rất tốn kém. Học từ phản hồi của con người không chỉ hạn chế về khả năng mở rộng quy mô, mà còn có thể đưa vào thiên lệch, vì phản hồi của con người vốn mang tính chủ quan. Do đó, ta có thể chuyển sang các phương pháp phản hồi từ AI (*AI feedback*) để giải quyết các vấn đề về khả năng mở rộng và tính nhất quán này mà không gặp những hạn chế gắn với người gán nhãn.

Cũng như việc sinh dữ liệu cho tinh chỉnh theo chỉ dẫn, sinh dữ liệu sở thích bằng LLM là việc đơn giản. Cho một tập đầu vào, trước tiên ta dùng một LLM để sinh ra các cặp đầu ra. Sau đó, ta dùng prompt để yêu cầu LLM xác định đầu ra được ưa thích hơn trong mỗi cặp, khi biết đầu vào tương ứng. Dưới đây là một ví dụ dùng prompt để LLM sinh nhãn sở thích cho một cặp phản hồi chăm sóc khách hàng.

> Xét một tình huống chăm sóc khách hàng, trong đó một khách hàng đưa ra yêu cầu. Bạn sẽ xem xét hai phản hồi cho yêu cầu này. Hãy cho biết phản hồi nào được ưa thích hơn. Lưu ý rằng một phản hồi tốt cần lịch sự, rõ ràng và ngắn gọn. Nó nên giải quyết trực tiếp mối quan tâm của khách hàng, cung cấp thông tin hữu ích hoặc giải pháp, và giữ giọng điệu tích cực.
>
> **Yêu cầu:**
> Xin chào, tôi để ý rằng đơn hàng của tôi vẫn chưa đến, dù theo lịch đáng lẽ nó đã đến từ vài ngày trước. Bạn có thể cập nhật cho tôi tình trạng đơn hàng không? Cảm ơn!
> **Phản hồi A:**
> Tôi rất xin lỗi vì sự chậm trễ và hiểu rằng điều này có thể gây thất vọng thế nào. Chúng tôi đang cố gắng hết sức để nhanh chóng giải quyết việc này cho bạn.
> **Phản hồi B:**
> Ê, chuyện thường mà! Gói hàng của bạn tới lúc nào thì tới, khỏi phải căng thẳng.
>
> Phản hồi A được ưa thích hơn.

Khi đã thu thập được các nhãn sở thích như vậy, ta có thể dùng chúng, cùng với đầu vào và hai phản hồi, để huấn luyện mô hình phần thưởng. Tất nhiên, ta có thể cân nhắc cung cấp một vài ví dụ minh họa hoặc dùng

các kỹ thuật prompt tinh vi như CoT để cải thiện chất lượng gán nhãn. Chẳng hạn, ta có thể đưa vào prompt một ví dụ cho thấy cách thức và lý do một trong hai phản hồi được ưa thích hơn, dựa trên một lập luận theo kiểu chuỗi suy nghĩ (*chain of thought*).

Bên cạnh nhãn sở thích, ta cũng có thể lấy xác suất gắn với mỗi nhãn [Lee và cộng sự, 2023]. Một cách đơn giản là trích xác suất của các token nhãn, chẳng hạn "A" và "B", từ các xác suất do LLM đưa ra. Sau đó, ta có thể dùng hàm Softmax hoặc các kỹ thuật chuẩn hóa khác để chuẩn hóa lại các xác suất này thành một phân phối trên các nhãn. Các xác suất của nhãn được ưa thích này có thể đóng vai trò tín hiệu giám sát theo điểm để huấn luyện mô hình phần thưởng, như đã bàn ở Mục 10.4.1.

Chú giải của người dịch: Chuẩn hóa lại xác suất nhãn

Giả sử khi đọc prompt trên, LLM gán xác suất 0.6 cho token "A", 0.15 cho token "B", và phần còn lại cho các token khác (như "Phản", "Cả", …). Chỉ giữ hai nhãn và chia cho tổng $0.6 + 0.15 = 0.75$, ta được $\Pr(\text{A}) = 0.8$, $\Pr(\text{B}) = 0.2$. Con số 0.8 cho biết LLM khá (nhưng không tuyệt đối) chắc chắn rằng A tốt hơn – thông tin mềm này giàu hơn một nhãn cứng "A".

Mặc dù mở rộng quy mô sinh dữ liệu bằng AI là dễ dàng, việc đảm bảo độ chính xác và tính đa dạng của dữ liệu vẫn rất quan trọng. Ở đây, vấn đề chất lượng và đa dạng của dữ liệu không chỉ liên quan đến việc gán nhãn sở thích mà cả đầu vào và đầu ra của mô hình. Vì vậy, ta thường cần dùng nhiều kỹ thuật khác nhau để có được dữ liệu quy mô lớn, chất lượng cao. Ví dụ, có thể sinh ra các đầu ra và chú thích đa dạng của mô hình bằng cách dùng các LLM, prompt, ví dụ minh họa trong ngữ cảnh khác nhau, v.v. [Cui và cộng sự, 2024]. Dubois và cộng sự [2024] cho biết tính đa dạng trong dữ liệu sở thích theo cặp là quan trọng đối với việc huấn luyện LLM từ phản hồi của con người hay của AI.

Tuy học từ phản hồi của AI có khả năng mở rộng rất cao và nhìn chung nhất quán hơn phản hồi của con người, phương pháp này phù hợp hơn với các nhiệm vụ được định nghĩa rõ ràng, có sẵn các thước đo hiệu năng khách quan. Ngược lại, học từ phản hồi của con người có lợi thế hơn khi căn chỉnh các hệ thống AI với giá trị, sở thích của con người, và với các nhiệm vụ thực tế phức tạp đòi hỏi hiểu những bối cảnh tinh tế hoặc chủ quan. Các phương pháp này có thể được kết hợp để huấn luyện những

LLM tận dụng được cả hiểu biết của con người lẫn khả năng mở rộng của phản hồi từ AI.

10.4.4 Căn chỉnh từng bước

Cho đến giờ, thảo luận của chúng ta về căn chỉnh chủ yếu tập trung vào việc dùng mô hình phần thưởng để đánh giá toàn bộ các cặp chuỗi đầu vào–đầu ra. Những phương pháp này có thể dễ dàng thích ứng với các tình huống mà tính đúng đắn của đầu ra có thể được kiểm chứng dựa trên đáp án cuối cùng. Chẳng hạn, trong nhiệm vụ tính giá trị một biểu thức toán học, mô hình phần thưởng có thể đưa ra phản hồi tích cực nếu đáp án đúng và phản hồi tiêu cực nếu đáp án sai. Tuy nhiên, trong nhiều bài toán đòi hỏi lập luận (*reasoning*) phức tạp, chỉ xem xét tính đúng đắn của kết quả cuối cùng là chưa đủ để học. Hãy tưởng tượng một học sinh chỉ được cho đáp số cuối cùng của một bài toán khó. Biết đáp số cuối cùng đúng hay sai không giúp học sinh nhận ra mình đã sai ở đâu và làm thế nào để tính ra đáp án đúng. Một cách tốt hơn là hướng dẫn học sinh bằng cách phân tích từng bước của quá trình giải bài, và giúp học sinh hiểu các khái niệm và logic nền tảng đằng sau các bước đó.

Ở Chương 9, chúng ta đã học các phương pháp CoT, dùng prompt để LLM viết ra một cách tường minh các bước trung gian hoặc quá trình lập luận cần thiết để đi đến một kết luận hay giải một bài toán. Ta đã thấy rằng chia nhỏ bài toán thành các phần nhỏ hơn có thể giúp dễ hiểu đường đi đến lời giải hơn và làm tăng độ chính xác của đầu ra. Những phương pháp này có thể được mở rộng một cách tự nhiên sang việc căn chỉnh LLM, tức là ta giám sát mô hình trong các bước lập luận trung gian. Xét một nhiệm vụ lập luận trong đó LLM tạo ra một chuỗi các bước lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_s}\}$ cho đầu vào đã cho. Kết quả của quá trình lập luận được giả định là nằm trong bước cuối $\bar{\mathbf{y}}_{n_s}$ và có thể dễ dàng kiểm chứng. Với bài toán lập luận này, Uesato và cộng sự [2022] chia các cách tinh chỉnh LLM thành hai loại:

- **Các phương pháp dựa trên kết quả** (*Outcome-based Approaches*). Việc giám sát chỉ diễn ra khi kết quả cuối cùng được kiểm chứng. Đây là phương pháp tiêu chuẩn để học từ phản hồi của con người mà chúng ta đã bàn trong chương này. Ví dụ, LLM được tối ưu để cực đại hóa một dạng phần thưởng $r(\mathbf{x}, \mathbf{y})$ nào đó.

- **Các phương pháp dựa trên quá trình** (*Process-based Approaches*). Việc giám sát được áp dụng cho mọi bước trung gian bên cạnh bước cuối cùng. Để làm được điều này, ta cần xây dựng một mô hình phần thưởng đưa ra tín hiệu giám sát ở mỗi bước, và thiết kế các hàm mất mát có thể tận dụng những tín hiệu giám sát như vậy.

Câu hỏi (trích từ Olympic Toán học Thiếu niên Quốc tế):
Jessica có rất nhiều ứng dụng trên điện thoại. Số ứng dụng cô có chia hết cho 5. Nếu cô tải thêm 10 ứng dụng nữa, cô vẫn sẽ có ít hơn 50 ứng dụng. Hỏi số ứng dụng ban đầu lớn nhất mà Jessica có thể có là bao nhiêu?

Phản hồi 1:

$\bar{\mathbf{y}}_1$: x phải là bội của 5, tức là $x \equiv 0 \pmod 5$.

$\bar{\mathbf{y}}_2$: Sau khi tải 10 ứng dụng, tổng tổng số ứng dụng vẫn phải nhỏ hơn 50: $x + 10 < 50$. Giải bất phương trình này: $x < 40$.

$\bar{\mathbf{y}}_3$: Vì x phải là bội của 5 và nhỏ hơn 40, giá trị lớn nhất của x là 35.

$\bar{\mathbf{y}}_4$: Kiểm tra kết quả. Nếu $x = 35$ thì sau khi tải 10 ứng dụng, Jessica sẽ có $35 + 10 = 45$, thỏa mãn $45 < 50$.

$\bar{\mathbf{y}}_5$: Số ứng dụng ban đầu lớn nhất là $\underline{35}$ (*đúng*).

Phản hồi 2:

$\bar{\mathbf{y}}_1$: Số ứng dụng chia hết cho 5, nên x phải là bội của 5.

$\bar{\mathbf{y}}_2$: Nếu cô tải thêm 10 ứng dụng, tổng mới sẽ là $x+10$, và số này phải nhỏ hơn 50. Do đó $x + 10 < 50$. Rút gọn ta được $x < 40$.

$\bar{\mathbf{y}}_3$: x phải nhỏ hơn hoặc bằng bội lớn nhất của 5, tức là 40. có vấn đề

$\bar{\mathbf{y}}_4$: Nhưng $x + 10$ không được lớn hơn hoặc bằng 50. Nên ta cần trừ 5 từ 40. có vấn đề

$\bar{\mathbf{y}}_5$: Do đó, kết quả cuối cùng là $\underline{35}$ (*đúng*).

Hình 10.11: Hai phản hồi của LLM cho một bài toán. Ở phản hồi 1, cả kết quả cuối cùng lẫn mọi bước lập luận đều đúng. Ở phản hồi 2, kết quả cuối cùng đúng, nhưng có những lỗi trong quá trình lập luận (tô màu đỏ). Với các phương pháp dựa trên kết quả, cả hai phản hồi đều được coi là đúng. Với các phương pháp dựa trên quá trình, các lỗi trong phản hồi 2 có thể được tính đến trong mô hình hóa phần thưởng.

Hình 10.11 cho thấy hai đầu ra của LLM cho một bài toán ví dụ. Dù LLM đưa ra đáp án cuối cùng đúng trong cả hai trường hợp, nó mắc lỗi trong quá trình giải ở đầu ra thứ hai. Các phương pháp dựa trên kết quả bỏ qua những lỗi này và đưa ra phản hồi tích cực cho toàn bộ lời giải. Ngược lại, các phương pháp dựa trên quá trình có thể tính đến những lỗi này và cung cấp thêm hướng dẫn về các bước lập luận chi tiết.

Chú giải của người dịch: Lỗi ở phản hồi 2 nằm ở đâu?

Bước $\bar{\mathbf{y}}_3$ của phản hồi 2 nói $x \leq 40$, mâu thuẫn với bước trước ($x < 40$, tức x không thể bằng 40). Bước $\bar{\mathbf{y}}_4$ "trừ 5 từ 40" là một cách sửa chữa mang tính mò mẫm, không có lập luận chặt chẽ, chỉ tình cờ cho đáp số đúng 35. Giống như bài kiểm tra ở trường: giáo viên chấm cả lời giải chứ không chỉ đáp số, vì đáp số đúng từ lập luận sai thì lần sau rất dễ sai.

Một thách thức then chốt của các phương pháp dựa trên quá trình là ta cần có phản hồi ở cấp từng bước trong một đường lập luận (có thể rất) dài. Ta có thể thu thập hoặc sinh các đường lập luận ứng với các bài toán từ những tập dữ liệu có sẵn. Sau đó, các chuyên gia con người gán cho mỗi bước một nhãn về tính đúng đắn. Các chú thích này có thể được dùng để huấn luyện trực tiếp LLM hoặc làm phần thưởng trong mô hình hóa phần thưởng. Tuy nhiên, trong thực tế, người ta thường dùng các chú thích phong phú hơn [Lightman và cộng sự, 2024]. Ngoài các nhãn *đúng* và *sai*, một bước còn có thể được gán nhãn *trung tính* để chỉ rằng tuy bước đó có thể đúng về mặt kỹ thuật, nó vẫn có thể có vấn đề trong toàn bộ quá trình lập luận. Hơn nữa, để nâng cao hiệu quả gán nhãn dữ liệu, có thể áp dụng các kỹ thuật như học chủ động (*active learning*). Việc chỉ ra các lỗi hiển nhiên thường không đóng góp đáng kể vào việc học từ các lỗi suy luận. Thay vào đó, gán nhãn những bước mà mô hình tự tin cho là đúng nhưng thực ra có vấn đề thường hiệu quả hơn.

Với một tập các đường lập luận đã được gán nhãn ở cấp bước cùng các đầu vào tương ứng, ta có thể huấn luyện một mô hình phần thưởng để cung cấp phản hồi giám sát việc học chính sách. Mô hình phần thưởng có thể được xem là một mô hình phân loại, nên kiến trúc của nó có thể là một bộ giải mã Transformer với một tầng Softmax đặt ở trên cùng. Tại bước k, mô hình phần thưởng nhận cả mô tả bài toán (ký hiệu $\mathbf{x}$) lẫn các bước lập luận đã sinh ra cho đến lúc đó (ký hiệu $\bar{\mathbf{y}}_{\leq k}$) làm đầu vào, và đưa ra một phân phối xác suất trên tập nhãn $\{\text{đúng}, \text{sai}\}$ hoặc $\{\text{đúng}, \text{sai}, \text{trung tính}\}$. Sau đó, mô hình phần thưởng đã học được dùng để đánh giá các đường lập luận bằng cách đánh giá tính đúng đắn của từng bước. Một cách đơn giản để mô hình hóa tính đúng đắn là đếm số bước được phân loại là đúng:

$$r(\mathbf{x}, \mathbf{y}) \;=\; \sum_{k=1}^{n_s} \delta(\text{đúng}, C(\mathbf{x}, \bar{\mathbf{y}}_{\leq k})), \tag{10.83}$$

trong đó $C(\mathbf{x}, \bar{\mathbf{y}}_{\leq k})$ ký hiệu nhãn có xác suất lớn nhất. Ta cũng có thể dùng log-xác suất phân loại để định nghĩa phần thưởng của toàn bộ đường lập luận

$$r(\mathbf{x}, \mathbf{y}) \;=\; \sum_{k=1}^{n_s} \log \Pr(\text{đúng} | \mathbf{x}, \bar{\mathbf{y}}_{\leq k}), \tag{10.84}$$

trong đó $\Pr(\text{đúng}|\mathbf{x}, \bar{\mathbf{y}}_{\leq k})$ ký hiệu xác suất của nhãn "đúng" do mô hình phần thưởng đưa ra. Điểm thưởng $r(\mathbf{x}, \mathbf{y})$ sau đó có thể được dùng để huấn luyện chính sách trong RLHF như thường lệ.

Chú giải của người dịch: Tính thử hai cách cho điểm

Hàm $\delta(a, b)$ bằng 1 nếu $a = b$ và bằng 0 nếu ngược lại. Với phản hồi 2 ở Hình 10.11 (5 bước), giả sử mô hình phần thưởng cho xác suất "đúng" ở các bước lần lượt là $0.9,\ 0.9,\ 0.2,\ 0.3,\ 0.8$. Theo Phương trình (10.83), có 3 bước được phân loại là đúng (xác suất > 0.5), nên $r = 3$; còn phản hồi 1 với mọi bước đúng sẽ được $r = 5$. Theo Phương trình (10.84), $r = \ln 0.9 + \ln 0.9 + \ln 0.2 + \ln 0.3 + \ln 0.8 \approx -3.33$: một bước rất đáng ngờ (xác suất nhỏ) sẽ kéo tổng xuống mạnh, vì ln của số gần 0 rất âm.

Tuy chúng ta giới hạn thảo luận ở các bài toán toán học, các phương pháp mô tả ở đây mang tính tổng quát và có thể áp dụng cho rất nhiều nhiệm vụ liên quan đến suy luận và ra quyết định nhiều bước. Hơn nữa, ta có thể xét nhiều khía cạnh khi đánh giá chất lượng một bước, chứ không chỉ tính đúng đắn của nó. Ví dụ, trong các hệ thống hội thoại, phản hồi không chỉ phải chính xác mà còn phải phù hợp với ngữ cảnh qua nhiều lượt hội thoại. Nếu một mô hình đưa ra phản hồi đúng nhưng không duy trì được sự mạch lạc trong ngữ cảnh của cuộc hội thoại đang diễn ra, phản hồi ở cấp bước có thể giúp mô hình nhận ra và sửa những sự lệch lạc như vậy. Cũng cần lưu ý rằng các phương pháp dựa trên quá trình có liên hệ với các phương pháp mô hình hóa phần thưởng chi tiết đã bàn ở Mục 10.4.1. Tất cả các phương pháp này về bản chất đều nhằm cung cấp sự giám sát chi tiết hơn cho LLM bằng cách chia đầu ra của chúng thành các bước nhỏ hơn, dễ xử lý hơn. Tuy nhiên, phản hồi dựa trên quá trình tập trung hơn vào việc đánh giá tính đúng đắn của một bước dựa trên các bước đứng trước nó, còn các phương pháp ở Mục 10.4.1 nhấn mạnh việc đánh giá từng bước một cách độc lập.

Ý tưởng căn chỉnh LLM từng bước có tiềm năng ứng dụng to lớn, đặc

biệt khi xét đến xu hướng gần đây chuyển sang các nhiệm vụ lập luận phức tạp hơn trong việc sử dụng LLM. Ví dụ, cả hai mô hình GPT o1 và o3 đều được thiết kế với các kỹ thuật suy luận tiên tiến hơn (như CoT nội tại dài) để giải các bài toán thách thức như suy luận khoa học và toán học [OpenAI, 2024]. Những nhiệm vụ này thường dựa trên các đường lập luận dài và phức tạp, do đó việc đưa vào các tín hiệu giám sát chi tiết trong quá trình lập luận có vẻ là thiết yếu. Hơn nữa, từ góc độ thực tiễn, giám sát hiệu quả trên các đường lập luận dài không chỉ cải thiện hiệu năng lập luận mà còn giúp mô hình loại bỏ những bước lập luận không cần thiết, nhờ đó giảm độ phức tạp của suy luận và tăng hiệu quả.

10.4.5 Căn chỉnh tại thời điểm suy luận

Từ đầu mục này đến giờ, chúng ta đã tìm hiểu nhiều phương pháp căn chỉnh mô hình theo sở thích và chú thích của con người. Tuy nhiên, một trong những hạn chế đáng kể của nhiều phương pháp như vậy là LLM phải được tinh chỉnh. Với RLHF và các biến thể của nó, huấn luyện LLM bằng mô hình phần thưởng có thể tốn kém về tính toán và không ổn định, làm tăng độ phức tạp và chi phí khi áp dụng các phương pháp này. Trong trường hợp đó, ta có thể cân nhắc thực hiện căn chỉnh tại thời điểm suy luận (*inference*), nhờ đó tránh được độ phức tạp và công sức phát sinh thêm.

Một cách đơn giản để căn chỉnh tại thời điểm suy luận là dùng mô hình phần thưởng để chọn đầu ra tốt nhất trong N đầu ra thay thế do LLM sinh ra, phương pháp được gọi là **lấy mẫu tốt nhất trong N** (*Best-of-N sampling*) (**lấy mẫu BoN**). Ta có thể xem lấy mẫu BoN như một dạng xếp hạng lại (*reranking*). Thực ra, các phương pháp xếp hạng lại đã được dùng rộng rãi trong các nhiệm vụ NLP, chẳng hạn dịch máy (*machine translation*), từ lâu. Chúng thường được áp dụng trong những tình huống mà việc huấn luyện các mô hình phức tạp tốn kém. Trong những trường hợp như vậy, việc xếp hạng lại trực tiếp các đầu ra cho phép tích hợp các mô hình phức tạp này với chi phí rất thấp[11].

[11] Các phương pháp xếp hạng lại cũng có thể giúp ta khảo sát cái gọi là *lỗi mô hình* và *lỗi tìm kiếm*, dù những vấn đề này ít được bàn đến trong bối cảnh LLM. Ví dụ, giả sử ta có một mô hình cũ và một mô hình mới mạnh hơn. Ta có thể dùng mô hình mới để chọn đầu ra tốt nhất từ danh sách N tốt nhất của mô hình cũ làm đầu ra "tiên tri" (*oracle*). Chênh lệch hiệu năng giữa đầu ra tiên tri và đầu ra đứng đầu (top-1) của danh sách N tốt nhất ban đầu phản

Trong quá trình lấy mẫu BoN, LLM nhận chuỗi đầu vào $\mathbf{x}$ và sinh ra N chuỗi đầu ra khác nhau $\{\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_N\}$:

$$\{\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_N\} \;=\; \underset{\mathbf{y}}{\text{argTopN}} \left[\Pr(\mathbf{y}|\mathbf{x}) \right], \tag{10.85}$$

trong đó phép toán argTopN trả về N đầu ra đứng đầu làm cực đại hàm $\Pr(\mathbf{y}|\mathbf{x})$. Các đầu ra này có thể được sinh ra theo nhiều cách khác nhau, tùy thuộc vào thuật toán tìm kiếm mà mô hình dùng (ví dụ lấy mẫu (*sampling*) hoặc tìm kiếm chùm (*beam search*)). Khi đã sinh ra N ứng viên đầu ra tốt nhất, mô hình phần thưởng được dùng để đánh giá và chọn ra ứng viên tốt nhất:

$$\hat{\mathbf{y}}_{\text{best}} \;=\; \underset{\hat{\mathbf{y}} \in \{\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_N\}}{\arg\max} \; r(\mathbf{x}, \hat{\mathbf{y}}). \tag{10.86}$$

Chú giải của người dịch: Lấy mẫu BoN giống "chọn bài dự thi"

Hãy hình dung một học sinh viết nháp 4 bài văn cho cùng một đề ($N = 4$), rồi nhờ giáo viên (mô hình phần thưởng) chấm điểm để chọn ra bài tốt nhất nộp đi. Bản thân học sinh không học thêm gì (mô hình không được tinh chỉnh), nhưng bài nộp vẫn tốt hơn. Cái giá phải trả là thời gian: phải viết N bài và chấm N lần mỗi khi trả lời một câu hỏi.

Cần lưu ý rằng kết quả của lấy mẫu BoN cũng chịu ảnh hưởng bởi tính đa dạng của danh sách N tốt nhất. Đây là một vấn đề chung của hầu hết các phương pháp xếp hạng lại. Lý tưởng nhất là các ứng viên đầu ra trong danh sách N tốt nhất có chất lượng tương đối cao nhưng đủ khác biệt với nhau. Trong nhiều hệ thống sinh văn bản, các đầu ra trong danh sách N tốt nhất rất giống nhau, thường chỉ khác nhau một hai từ. Vấn đề đa dạng còn khó khăn hơn với LLM, vì các đầu ra N tốt nhất do LLM sinh ra có thể khác nhau về cách diễn đạt, nhưng ý nghĩa thường khá giống nhau. Trong thực tế, ta có thể điều chỉnh các siêu tham số (*hyperparameter*) của mô hình và/hoặc dùng các LLM khác nhau để sinh ra các ứng viên đầu ra đa dạng hơn cho việc xếp hạng lại. Dù vậy, như với nhiều hệ thống thực tế, ta cần cân bằng giữa việc chọn các ứng viên chất lượng cao và việc đảm bảo đủ sự biến thiên trong các đầu ra được sinh.

ánh mức tăng hiệu năng do mô hình mới mang lại. Nếu mức tăng này đáng kể, ta có thể nói rằng mô hình cũ có nhiều lỗi mô hình hơn. Nếu mức tăng nhỏ, điều đó có thể cho thấy vấn đề nằm ở lỗi tìm kiếm, vì các ứng viên tốt nhất đã không được tìm thấy.

Lấy mẫu BoN cũng có thể được dùng để huấn luyện LLM. Một phương pháp có liên quan chặt chẽ là **lấy mẫu loại bỏ** (*rejection sampling*). Trong phương pháp này, trước tiên ta chọn các đầu ra "tốt nhất" từ các danh sách N tốt nhất bằng mô hình phần thưởng, rồi lấy các đầu ra được chọn này để tinh chỉnh LLM. Bằng cách này, ta có thể đưa sở thích của con người vào việc huấn luyện LLM bằng một cách tiếp cận đơn giản hơn nhiều so với RLHF. Nhiều LLM đã áp dụng lấy mẫu loại bỏ để tinh chỉnh [Nakano và cộng sự, 2021; Touvron và cộng sự, 2023b].

10.5 Tóm tắt

Trong chương này, chúng ta đã tìm hiểu một loạt kỹ thuật để căn chỉnh LLM. Cụ thể, chúng ta đã bàn về các phương pháp tinh chỉnh (*fine-tuning*) giúp LLM làm theo chỉ dẫn và căn chỉnh chúng theo sở thích của con người. Một trong những lợi ích của việc tinh chỉnh LLM là hiệu quả tính toán. Khác với tiền huấn luyện (*pre-training*) dựa trên tối ưu hóa mạng nơ-ron quy mô lớn, tinh chỉnh là một bước hậu huấn luyện (*post-training*) nên ít tốn kém về tính toán hơn. Hơn nữa, nó phù hợp hơn để giải quyết những vấn đề khó giải quyết trong tiền huấn luyện, chẳng hạn căn chỉnh theo giá trị của con người. Sự quan tâm rộng rãi đến việc căn chỉnh AI cũng dẫn đến sự bùng nổ các bài báo nghiên cứu về chủ đề này, khiến việc bao quát đầy đủ mọi kỹ thuật gần đây trở nên khó khăn. Tuy nhiên, chúng tôi đã cố gắng giới thiệu tương đối chi tiết các cách tiếp cận cơ bản cho việc căn chỉnh, chẳng hạn tinh chỉnh theo chỉ dẫn (*instruction fine-tuning*) và RLHF.

Dù chương này tập trung vào các kỹ thuật căn chỉnh LLM, thuật ngữ *căn chỉnh AI* là một khái niệm rộng. Nó thường chỉ quá trình đảm bảo hành vi của một hệ thống AI phù hợp với giá trị, mục tiêu và kỳ vọng của con người. Ý tưởng căn chỉnh AI có thể được truy ngược về những ngày đầu của AI. Một mô tả về căn chỉnh AI được trích dẫn rộng rãi đến từ một bài viết của nhà toán học và nhà khoa học máy tính Norbert Wiener [Wiener, 1960]. Câu trích dẫn như sau

> *Nếu, để đạt được mục đích của mình, chúng ta sử dụng một tác nhân cơ học mà ta không thể can thiệp một cách hiệu quả vào hoạt động của nó … thì tốt hơn hết ta phải thật chắc chắn*

rằng mục đích được đưa vào cỗ máy chính là mục đích mà ta thực sự mong muốn.

Vào thời điểm đó, căn chỉnh chưa phải là mối quan tâm trung tâm trong nghiên cứu AI. Nhưng ngày nay, nó ảnh hưởng rất lớn đến việc thiết kế nhiều hệ thống AI khác nhau. Ví dụ, trong robot học, căn chỉnh là yếu tố then chốt để đảm bảo các robot tự hành tương tác an toàn với con người và môi trường xung quanh. Trong lái xe tự động, xe không chỉ phải tuân thủ luật giao thông mà còn phải đưa ra những quyết định phức tạp theo thời gian thực, ưu tiên an toàn của con người, tránh tai nạn và xử lý các tình huống khó xử về đạo đức.

Trong nghiên cứu AI hiện nay, căn chỉnh thường được tiếp cận bằng cách xây dựng một mục tiêu thay thế tương tự với mục tiêu thực sự và lái hệ thống AI hướng tới mục tiêu này. Tuy nhiên, thiết kế mục tiêu của căn chỉnh AI là rất khó. Một lý do là giá trị của con người rất đa dạng và thường phụ thuộc vào bối cảnh, khiến việc chắt lọc chúng thành một hàm mục tiêu duy nhất, áp dụng được cho mọi trường hợp trở nên khó khăn. Ngoài ra, độ phức tạp của môi trường thực tế, nơi các giá trị và mục tiêu thường xung đột hoặc thay đổi theo thời gian, càng làm phức tạp thêm nỗ lực căn chỉnh. Ngay cả khi ta có thể định nghĩa một mục tiêu phù hợp, các hệ thống AI vẫn có thể tìm ra những cách ngoài dự kiến để đạt được nó, dẫn đến những kết quả "lệch chuẩn" (*misaligned*), về mặt kỹ thuật vẫn thỏa mãn mục tiêu nhưng theo cách có hại hoặc phản tác dụng.

Chú giải của người dịch: Ví dụ về kết quả "lệch chuẩn"

Đây chính là ý của định luật Goodhart ở Mục 10.4.1. Một ví dụ dễ hình dung: nếu ta thưởng cho robot dọn nhà theo "số lượng rác không còn nhìn thấy", robot có thể học cách *giấu* rác dưới thảm thay vì đổ rác – mục tiêu đo được vẫn đạt, nhưng điều ta thực sự muốn (nhà sạch) thì không.

Những thách thức này tiếp tục thúc đẩy nghiên cứu AI hướng tới các hệ thống được căn chỉnh tốt hơn, hoặc thông qua phát triển những cơ chế mới để nhận thức thế giới, hoặc thông qua các phương pháp hiệu quả hơn và có khả năng tổng quát hóa cao hơn để thích ứng các hệ thống này với những nhiệm vụ cho trước. Quan trọng hơn, khi các hệ thống AI ngày càng mạnh mẽ và thông minh hơn, đặc biệt khi những tiến bộ gần đây của LLM đã cho thấy năng lực đáng kinh ngạc trong việc xử lý nhiều bài

toán thách thức, nhu cầu căn chỉnh AI càng trở nên cấp bách. Các nhà nghiên cứu đã bắt đầu quan tâm đến an toàn AI (*AI safety*) và cảnh báo cộng đồng rằng cần phát triển và phát hành các hệ thống AI hết sức thận trọng để ngăn những hệ thống này bị lệch chuẩn [Russell, 2019; Bengio và cộng sự, 2024].

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.